

Encrypted RAM Delegation: Homomorphic NIZKs, Malicious MPC with Optimal Communication Complexity, and more*

Abtin Afshar UW–Madison abtin@cs.wisc.edu	Jiaqi Cheng UW–Madison jiaqicheng@cs.wisc.edu	Rishab Goyal UW–Madison rishab@cs.wisc.edu [†]
Aayush Yadav GMU ayadav5@gmu.edu [‡]	Saikumar Yadugiri UW–Madison saikumar@cs.wisc.edu	

Abstract

We construct fully-homomorphic NIZKs from a broader range of cryptographic assumptions, including LWE , $O(1)\text{-LIN}$, and DDH (without circular security assumptions). Prior to this work, homomorphic NIZKs were only known based on pairing-based techniques. A key ingredient that we develop in this work is a new notion of a partially-hiding succinct delegation system, that we call encrypted RAM (eRAM) delegation. In addition to homomorphic NIZKs, we show that eRAM delegation is also useful for turning a semi-malicious multiparty computation (MPC) protocol into a full maliciously secure protocol in a communication-complexity-preserving manner (i.e., malicious and semi-malicious protocols have identical asymptotic communication complexity). Prior MPC compilers with similar efficiency required strong non-falsifiable assumptions. Furthermore, eRAM delegation also is useful for designing composable zero-knowledge batch arguments and multi-hop homomorphic signatures with strong-context-hiding.

*Previous version contained the same results with a different presentation.

[†]Support for this research was provided by OVCRGE at UW-Madison with funding from the Wisconsin Alumni Research Foundation.

[‡]Work done while at UW–Madison. Funded by NSF #2143287.

Contents

1	Introduction	1
1.1	Overview	2
2	Preliminaries	11
2.1	RAM Programs	11
2.2	Somewhere Extractable (Zero-Knowledge) Batch Arguments	12
2.3	Hash Tree	14
2.4	Non-Interactive Zero-Knowledge Arguments (NIZK)	15
2.5	Rate-1 Message Encoding	16
2.6	Public-Key Encryption System	17
2.7	RAM Delegation Scheme	17
3	Defining (Reusable) Encrypted RAM Delegation	18
3.1	Encrypted RAM Delegation	18
3.2	Reusable Encrypted RAM Delegation	19
4	Constructing (Reusable) Encrypted RAM Delegation	22
4.1	Encrypted RAM Delegation	22
4.1.1	Security Analysis.	23
4.2	Reusable Encrypted RAM Delegation	28
4.2.1	Security Analysis.	29
5	Homomorphic Non-Interactive Zero-Knowledge	32
5.1	Definitions: Homomorphic NIZKs	32
5.2	Tools Needed: Optimally Efficient NIZK and Rate-1 Message Encodings	33
5.2.1	Security Analysis	35
5.3	Boolean Circuit Decomposition	37
5.4	Constructing Homomorphic NIZKs	38
5.5	Security Analysis	40
6	Complexity-Preserving Semi-Malicious to Malicious Compiler for mrNISC	42
6.1	Definition	43
6.2	Construction	44
6.3	Security Analysis of Construction 6.4	46
6.4	Semi-Maliciously Secure mrNISC with Compact Communication	51
7	Multi-Hop zkBARG with Somewhere-Extraction	54
7.1	Definition	55
7.2	Construction	57
7.3	Security Analysis	58
8	Publicly-Verifiable Succinct Delegation of Committed Programs	62
8.1	Definition	63
8.2	Construction	65

A	Additional Definitions	75
A.1	Somewhere Extractable Hash Families	75
A.2	Multi-Key Fully Homomorphic Encryption	76
B	Split Memory RAM Delegation	77
B.1	Split Memory RAM Delegation Construction	77
B.2	Security Analysis	80

1 Introduction

Homomorphism has long been regarded as a “holy grail” concept in cryptography, enabling computation over cryptographic data without compromising its privacy or integrity. Since the introduction and groundbreaking realization of fully homomorphic encryption [RAD⁺78, Gen09], the cryptographic community has studied homomorphism across a broad range of primitives, including signatures [JMSW02, AB09, BFKW09, GVV15b], secret sharing [BGI15, BGI16], and zero-knowledge proofs [CKLM12, CKLM13b, ADKL19]. These primitives have unlocked numerous applications and fundamentally reshaped how we think about secure protocol design and data privacy.

Although we have many diverse constructions for homomorphic encryption (from lattices, codes, obfuscation) [RAD⁺78, Gen09, BV11b, BV11a, BGV12, GSW13, CLTV15, CHKV25], homomorphic signatures (from lattices, pairings, obfuscation, batch proofs) [JMSW02, ABC⁺07, AB09, BFKW09, BF11b, GVV15b, BFS14, FMNP16, WW24, GU24, ACG24], and homomorphic secret sharing (from lattices, codes, DDH, DCR, obfuscation, and more) [BGI15, BGI16, BGI17, BCG⁺17, BKS19, FGJS17, OSY21, RS21, ADOS22, BCG⁺19, CM21, AJLA⁺12, CM15, MW16, DHRW16, BGG⁺18, BGI⁺18, DIJL23], a notable gap persists in the space of non-interactive zero-knowledge proofs (NIZKs). In particular, the only known approach to constructing homomorphic NIZKs remains narrowly tied to pairing-based techniques [CKLM12, CKLM13a, CKLM13b, AN11, AGP14, ACJ17, AGM18, ADKL19]. It leaves homomorphic NIZKs as the only primitive, in the family of homomorphic cryptosystems, that does not have a quantum-safe construction. Given the vast number of applications of homomorphic proofs to verifiable shuffles [CKLM12], delegatable anonymous credentials [BCC⁺09, CKLM13a], non-interactive blind signatures [Han23, BCGY24], leakage-resilient proof systems [AGP14], private incremental proofs [Val08], this lack of diversity in the context of homomorphic NIZKs is both technically limiting and conceptually unsatisfying.

In this work, we develop a new approach for designing homomorphic NIZKs, and diversify the class of assumptions that suffice for them. Our work closes the existing gap and brings homomorphic NIZKs on similar footing (in terms of assumptions) as homomorphic encryption, signatures, and secret sharing. As a key ingredient of our approach, we introduce a new notion of “partially-hiding” succinct delegation system, called encrypted RAM (eRAM) delegation, and design it under a wide variety of standard cryptographic assumptions such as LWE , $O(1)\text{-LIN}$, or DDH assumption. In addition to serving as an important building block for homomorphic NIZKs, we show eRAM delegation is useful for designing maliciously-secure multi-party computation protocols with optimal communication complexity [Yao86, GMW87, AJLA⁺12]. Our main results can be summarized as follows.

Theorem 1 (informal). Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists a homomorphic NIZK scheme for NP supporting all polynomial-sized homomorphic computations.

Theorem 2 (informal). Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists a compiler to upgrade any MPC protocol with semi-malicious security to full malicious security while incurring only additive $\text{poly}(\lambda)$ blowup in communication per message.

Our compiler takes a semi-malicious MPC protocol and upgrades it to a maliciously-secure protocol with identical (asymptotic) communication complexity. As an immediate corollary, we obtain a maliciously-secure multi-party reusable non-interactive secure computation (mrNISC) protocol [BL20] from LWE , where the first-round (reusable) message length is the input length plus

$\text{poly}(\lambda)$, and second-round (online) message length is the final output length times a fixed $\text{poly}(\lambda)$ factor. In particular, the total communication complexity is independent of the size of the multi-party computation function. Additionally, we provide many other immediate applications of eRAM such as realizing strong context hiding in multi-hop homomorphic signatures [GVW15b] and designing composable zero-knowledge non-interactive batch arguments [DGKV22, PP22]. These are summarized below.

Theorem 3 (informal). Assuming sub-exponential hardness of $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{DDH}\}$, there exists a multi-key multi-hop homomorphic signature scheme with strong context hiding for all polynomial-sized homomorphic computations.

Theorem 4 (informal). Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists a multi-hop zero-knowledge non-interactive batch argument (zk-BARG) for NP with somewhere extraction.

Roadmap. The rest of the paper is organized as follows. In the next section, we provide an overview of our results and main techniques. In Sections 3 and 4, we formally introduce our notion of eRAM delegation and construct it from a variety of standard assumptions. In Section 5, we design extractable NIZKs with optimal efficiency, and use them to build fully homomorphic NIZKs. In Section 6, we present our semi-malicious to malicious compiler for mrNISC and related MPC protocols. We provide some useful preliminaries in Section 2. And, later in Sections 7 and 8, we provide additional results such as composable zero-knowledge batch arguments and delegation protocols for committed programs.

Concurrent Work. In a concurrent work, Branco, Döttling, and Srinivasan [BDS25] studied the problem of rate-1 NIZK arguments from standard assumptions. A central focus of our work is to design homomorphic NIZKs from standard assumptions. Along the way, we also provide a new construction for rate-1 NIZK arguments from standard assumptions. The main overlap between our work and [BDS25] is that both works design rate-1 NIZK arguments from $\text{LWE}/O(1)\text{-LIN}/\text{sub-exponential DDH}$ assumption, and they rely on similar building blocks such as non-interactive batch arguments. In terms of differences, our work designs the first homomorphic NIZK scheme from non-pairing assumptions, and provides a communication complexity preserving compiler for mrNISC and related MPC protocols, and other results such as homomorphic signatures with strong context hiding, multi-hop zk-BARGs. On the other hand, [BDS25] only study and design rate-1 NIZKs. In further detail, another difference between our work and [BDS25] is that we prove computational zero-knowledge for our rate-1 NIZK, while [BDS25] prove statistical zero-knowledge. Additionally, our proof size is truly rate-1 (i.e., $\pi = |\omega| + \text{poly}(\lambda)$), while theirs is slightly larger (i.e., $\pi = |\omega| + |\omega|^\epsilon \cdot \text{poly}(\lambda)$) and also requires sub-exponential hardness.

1.1 Overview

A fully-homomorphic NIZK system [ADKL19] is a special-purpose NIZK [BDSMP91], where an arbitrary sequence of independently generated proofs can be homomorphically combined to generate a fresh NIZK proof certifying validity of the evaluated instance-witness pair. To formally define the homomorphic evaluation process, consider the following NP language — each instance x contains a pair of circuit and boolean output (C, b) , and a witness ω is an input string such that $C(\omega) = b$. Homomorphic NIZKs have a special *proof evaluation* algorithm, Eval , that takes k tuples of the

form $\{(x_i = (C_i, b_i), \pi_i)\}_{i \in [k]}$ and a boolean circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$. The goal of the evaluation algorithm is to generate a NIZK proof for the statement (C', b') , where $b' = C(b_1, \dots, b_k)$ and C' is the circuit C composed with circuit C_i on its i -th input wire. Such a composition maintains the invariant that (C', b') will be a valid instance if every (C_i, b_i) is a valid instance. This is because $C'(\omega_1, \dots, \omega_k)$ is same as $C(C_1(\omega_1), \dots, C_k(\omega_k))$, by definition.

Unlike other homomorphic primitives, it is not necessary to define an additional compactness property for evaluated proofs. In fact, homomorphic NIZKs with compact evaluated proofs face strong barriers under standard falsifiable assumptions [GW11, CGKS23].¹ Thus, a more natural approach is to study and design homomorphic NIZKs with basic efficiency. Although designing other homomorphic primitives, such as encryption and signatures, without compactness is usually far easier, this is not the case for homomorphic NIZKs.

To further illustrate the difference, consider the following naive homomorphic signature construction. An evaluator sets the evaluated proof to be the input instance-proof pairs $\{(x_i = (C_i, b_i), \pi_i)\}_i$ along with evaluation circuit C . A verifier follows a two-step approach: first, it verifies the individual NIZK proofs $\pi_1, \dots, \pi_k$; second, it checks that $C(b_1, \dots, b_k)$ matches the boolean output in the instance. The problem with the naive construction is that it does not preserve zero-knowledge. An evaluated proof leaks intermediate wire values during homomorphic evaluation. E.g., if C is a tree of OR gates, then (ideally) the evaluated proof should hide which/ how many of the input bits from b_1 to b_k are 1. However, the naive approach leaks that in the clear. More broadly, the above evaluated proof does not resemble a fresh ‘honestly generated proof’ for the evaluated instance (C', b') , where $C' = C(C_1(\cdot), \dots, C_k(\cdot))$ and $b' = C(b_1, \dots, b_k)$. As noted in earlier works [CKLM12, CKLM13b, ADKL19], such trivial homomorphic evaluations are not interesting or useful for applications.

To eliminate such naive designs and support interesting applications, Ananth et al. [ADKL19] proposed an unlinkability property (in addition to zero-knowledge) for homomorphic NIZKs. Unlinkability states that an evaluated proof must be indistinguishable from any honestly generated proof for the same instance. Combining the unlinkability property with the zero-knowledge property, we obtain that evaluated proofs are also indistinguishable from simulated proofs.

The current state-of-the-art for fully-homomorphic NIZKs is the pairing-based construction by Ananth et al. [ADKL19]. Their construction builds on the seminal NIZK construction by Groth-Ostrovsky-Sahai (GOS) [GOS06]. At a very high level, during each homomorphic evaluation, they follow a two-step approach: (1) use a commitment to hide the output wires of each input proof, and (2) generate a fresh NIZK proving that it knows openings to the output wire commitments such that they satisfy the circuit being evaluated. To ensure unlinkability, the evaluator also randomizes all incoming wire commitments as well as re-randomizes incoming NIZK proofs to match these randomized commitments. In a sentence, Ananth et al. used intrinsic malleability properties of GOS NIZKs to support homomorphism. For more details, readers are encouraged to refer [GOS06, ADKL19].

Quite clearly, we need to hide the incoming NIZK proofs during homomorphic evaluation, and without relying on proof malleability, hiding an incoming proof seems difficult. To the best of our knowledge, GOS-style NIZKs are the only available NIZKs that are malleable [CKLM12, CKLM13b]. Therefore, existing design templates do not appear useful for diversifying the class of assumptions sufficient for homomorphic NIZKs. To get around this, we propose a new approach.

¹Compact homomorphic NIZKs imply succinct non-interactive arguments [Kil92, Mic94a], which are inhibited by strong infeasibility results [GW11, CGKS23].

New approach for homomorphic NIZKs. We draw some inspiration from the classic Gentry’s bootstrapping approach for homomorphic encryption [Gen09], and identify a simpler building block that could be bootstrapped to fully-homomorphic NIZKs. Recall that for a regular NIZK proof system, the proof size as well as the verifier running time can scale polynomially with the time it takes to verify the actual NP relation. That is, let T denote the circuit size of an NP relation $\mathcal{R}_{\mathcal{L}}$ that outputs 1 for any valid instance-witness pair $(x, \omega) \in \mathcal{L}$. For regular NIZKs, the proof size, $|\pi|$, as well as the verifier’s running time grows polynomially with T . This is the case for all standard model NIZK systems built from standard falsifiable assumptions. This also includes the FHE-based rate-1 NIZK construction by Gentry et al. [GGI⁺15], where the verifier homomorphically evaluates the relation $\mathcal{R}_{\mathcal{L}}$ on an FHE ciphertext during verification.

Our observation is that an optimally efficient NIZK proof system with three salient features is one such useful building block. To put it formally, if a NIZK system — (i) has an *optimal-proof-rate*, which basically means the proof length is asymptotically equal to the witness length, i.e. $|\pi| = |\omega| + \text{poly}(\lambda)$, (ii) has a straight-line knowledge extractor, which means we can extract a valid witness from any accepting proof, and (iii) the verifier’s run-time is independent of T , actual NP verification time, — then it suffices for designing homomorphic NIZKs.

Optimal-NIZKs to Homomorphism. Given a sequence of instances $\{(C_i, b_i)\}_{i \in [k]}$ and proofs $\{\pi_i\}_{i \in [k]}$, we homomorphically evaluate a circuit C as follows:

- Step 1.** Commit the incoming wire values $b_1, \dots, b_k$ to $\text{com}_1, \dots, \text{com}_k$, and create k NIZK proofs $\pi'_1, \dots, \pi'_k$ to prove i -th commitment com_i opens to a boolean value b_i such that (C_i, b_i) is a valid instance. To prove this, the evaluator uses commitment randomness for com_i along with each incoming proof π_i as the new witness.
- Step 2.** Create an additional NIZK proof π'' to prove that $\text{com}_1, \dots, \text{com}_k$ commit to boolean values $b_1, \dots, b_k$ such that $C(b_1, \dots, b_k)$ matches the boolean value in the evaluated instance.
- Step 3.** Release $\{\text{com}_i\}_i$, $\{\pi'_i\}_i$, and π'' as the evaluated proof.

Note that the evaluated proof can be verified using a piece-by-piece approach. By which we mean that first all π'_i are individually verified and then π'' is verified. Next, observe that if the underlying NIZK proof system is optimally efficient (it has optimal-proof-rate and has fast verification time), then each evaluated proof can be further evaluated many times. This is because the proof length grows by only an additive $\text{poly}(\lambda)$ factor after each evaluation step due to optimal-proof-rate property. And, the evaluator’s running time also can be bounded by a fixed polynomial since the underlying NIZK verifier run-time is fixed.

Moving onto the security guarantees of the above template, zero-knowledge and soundness are fairly straightforward. For zero-knowledge, one can rely on the underlying NIZK being zero-knowledge and the commitment scheme being hiding in tandem. For soundness, we can use the straight-line knowledge extractor to extract wire values for each intermediate wire. This will either give us a satisfying witness, or break binding of the commitment. Finally, moving on to the desirable “unlinkability” property, we face a problem.

The issue is that one can trivially distinguish an evaluated proof from a freshly generated proof by inspecting the proof length. An evaluated proof would be much larger than a fresh proof since during each homomorphic evaluation we pay an additive $\text{poly}(\lambda)$ cost in proof length. While one might try padding to fix this issue, it is unclear whether that could work without putting artificial constraints (such as statically fix the circuit size and topology that can be homomorphically evaluated).

Our idea is to harmonize the prover and evaluator algorithm, and essentially have them run the underlying NIZK prover the same number of times. In particular, we “decompose” each circuit C into non-overlapping groups of “atomic circuits”. And, to generate a NIZK proof, a prover would first decompose each circuit and use homomorphic evaluation over all atomic components to build the proof in a step-by-step manner. The important part is each homomorphic operation only evaluates an atomic circuit. To ensure the decomposition is deterministic and consistent, we view each circuit as a directed acyclic graph, and identify the largest set of vertices that can be safely ‘cut’ to partition the circuit into smaller sub-circuits, where each sub-circuit cannot be further decomposed.

Later in the main body, we formally provide our algorithms for atomic circuit decomposition, and plug them in our base template to design homomorphic NIZKs. For ease of writing in the main body, we merge all individual NIZK proofs created during evaluation into a single proof. We write them individually above for ease of explanation. We remark that our homomorphic NIZK construction supports unbounded homomorphism, and is truly *fully* homomorphic without relying on any circular security type assumptions.

We believe our template should also generalize to get non-trivial notions of succinctness for restricted types of homomorphic NIZKs (such as batched succinctness during homomorphism), and leave it as an exciting problem to study. As noted in a prior work [CG25], any non-trivial notion of succinctness for fully-homomorphic NIZKs faces strong barriers. On a separate note, we view optimally-efficient NIZKs as an analogue of bootstrappable homomorphic encryption [Gen09], since they both serve the same purpose. Next, we describe how to design such NIZKs via a new tool called eRAM.

Introducing encrypted RAM delegation. As mentioned earlier, a key ingredient that we use is a new notion of RAM delegation [Mic94b, KR09, KRR14, BHK17, PR17], called *encrypted RAM* delegation.

A traditional RAM delegation system has two algorithms, *Prove* and *Verify*.² Given a large input x , *Prove* outputs a short proof π certifying the machine $\mathcal{M}$ accepts x . *Verify* reads (a short digest of) input x and proof π , and outputs a bit to signal validity of computation. To make this non-trivial, one requires verifier’s run-time and proof length to be independent of machine $\mathcal{M}$ ’s run-time. Moreover, the system should be sound, as in an adversary cannot produce an input x and proof π such that *Verify* accepts, yet $\mathcal{M}(x)$ rejects.

Encrypted RAM (eRAM) generalizes traditional RAM delegation to guarantee that an input x can be “partially hidden” from the verifier. To define partial hiding, we give x in two disjoint pieces, x_{pb} and x_{pr} , to the *Prove* algorithm. Here x_{pb} is viewed as a *large public* input, while x_{pr} is a *small private* input. We want the verifier to check validity of the computation, $\mathcal{M}(x) = \mathcal{M}(x_{\text{pb}}, x_{\text{pr}})$, given only the proof π and (the digest of) x_{pb} . Notably, *Verify* does not get x_{pr} . In all applications of eRAM that we study in this work, x_{pr} is always of fixed polynomial length, thus we only require a slightly relaxed compactness property. In particular, we require the verifier’s run-time and proof length to be $\text{poly}(\log T, \log |x_{\text{pb}}|, |x_{\text{pr}}|)$, where T is machine $\mathcal{M}$ ’s runtime. That is, polynomial dependence on the length of the private input is acceptable.

For security, we require eRAM to satisfy *partial* knowledge soundness and private input hiding properties. Partial knowledge soundness states that there exists an efficient *straight-line* extractor that, given an accepting proof π for public input x_{pb} , can extract a satisfying private input x_{pr}^* .

²It additionally requires a trusted setup to sample a CRS, but we ignore it for simplicity.

Observe that since we allow the proof size to grow with $|x_{\text{pr}}|$, thus straight-line partial extraction is indeed feasible in the standard model. The private input hiding property states that a proof π corresponding to input $(x_{\text{pb}}, x_{\text{pr}})$ should be efficiently simulatable, given only x_{pb} . Basically, any leakage about x_{pr} in π can be simulated by a zero-knowledge style simulator.

Next, we describe how eRAM can be used to design an optimally efficient NIZKs as needed for homomorphic NIZKs. Later, we discuss key ideas behind our eRAM construction from standard assumptions.

Making NIZKs optimally-efficient. The core idea is pretty straightforward. We will set the NIZK proof to be an eRAM proof, where the original NIZK witness ω will be encrypted and included in the public portion of eRAM input, while the decryption key will be used as the private eRAM input. Concretely, our new NIZK prover starts by sampling a random λ -bit PRG seed sd . It then uses sd to one-time pad encrypt ω as $\hat{\omega} = \omega \oplus \text{PRG}(\text{sd})$, and put the encryption $\hat{\omega}$ in the public eRAM input. Basically, it runs the eRAM prover algorithm with $x_{\text{pb}} = (x, \hat{\omega})$ and $x_{\text{pr}} = \text{sd}$, where the RAM machine first decrypts $\hat{\omega}$ using x_{pr} and then runs the RAM machine for corresponding NP language. The final NIZK proof contains the above eRAM proof and the encrypted witness $\hat{\omega}$.

Given an eRAM proof and encrypted witness, the NIZK verifier can simply run the eRAM verifier. This relies on the eRAM property that eRAM verifier only needs x_{pb} which contains instance x and $\hat{\omega}$. The zero-knowledge of the above design follows directly from the partial input hiding property of eRAM, as the eRAM proof can be simulated without $x_{\text{pr}} = \text{sd}$ and once it is simulated $\hat{\omega}$ can also be turned to garbage. Next, let us quickly verify that the above design satisfies all properties desired by our homomorphic NIZK template.

Optimal-proof-rate. Since x_{pr} is λ bits long, and $|\hat{\omega}| = |\omega|$, thus by the relaxed compactness of eRAM, we get the resulting NIZK proof is of optimal length.

Straight-line extractor. This follows by using the eRAM partial extractor to extract sd and then decrypting $\hat{\omega}$ using extracted sd .

Fast verification. Given eRAM verifier's run-time is $\text{poly}(\log T, \log |x_{\text{pb}}|, |x_{\text{pr}}|)$, thus the verifier's run-time is independent of the actual NP verification time.

In summary, eRAM gives us a homomorphic NIZK by plugging in the above NIZK in our new template. To finish up the construction of our homomorphic NIZKs, we provide a construction for eRAM under a wide variety of standard assumptions (such as LWE , $O(1)\text{-LIN}$, DDH).

Constructing eRAM from standard assumptions. The starting point for our eRAM construction is the Choudhuri-Jain-Jin (CJJ) [CJJ22, CJJ21] RAM delegation construction based on non-interactive batch arguments (BARGs). Briefly, a BARG lets a prover take a batch of, say, k instance witness pairs $\{(x_i, \omega_i)\}_i$, and output a short proof π , where $|\pi| = \text{poly}(\lambda, |\omega_i|, \log k)$, yet it can certify validity of all k instances $x_1, \dots, x_k$. The CJJ RAM delegation scheme used BARGs to prove consistency of a table of RAM computation states with a proof size that is independent of the total computation time. At a high level, their delegation scheme for machine $\mathcal{M}$ can be summarized as follows.

Let cf_0 denote the initial configuration (including the input x), and cf_i denote the machine $\mathcal{M}$'s configuration after i computation steps. The idea underlying the CJJ RAM delegation scheme is to split the global consistency check between the initial and final configuration, to a sequence of local consistency checks between any two consecutive configurations cf_{i-1} and cf_i . Namely, the prover

generates a short hash digest c of all configurations $(cf_0, \dots, cf_T)$, then generates a BARG where the i -th instance certifies: (a) $cf_i = \text{nxt-cnfg}(cf_{i-1})$, where nxt-cnfg is the step function for $\mathcal{M}$, and (b) cf_{i-1} and cf_i are $(i-1)$ -th and i -th configurations as per the digest c .

Although the proof size in the above does not grow with the total number of configurations, it does scale with a single configuration size which could be larger than $\mathcal{M}$'s running time or the input length. CJJ fixed this by hashing down each configuration before creating the digest. That is, each cf_i is first hashed to hcf_i , and then all $\{hcf_i\}_i$ are hashed together to obtain the final digest c . With this change, the prover generates the BARG proof where the i -th instance certifies that the $(i-1)$ -th and i -th hashed configurations, hcf_{i-1} and hcf_i , are consistent as per nxt-cnfg . Note that any two consecutive RAM machine configurations are identical except at a few places, thus their consistency can be checked efficiently by opening hcf_{i-1} and hcf_i for those few locations. The proof of soundness relies on a very delicate strategy of combining somewhere extractability of BARGs [CJJ22, CJJ21] with a strong binding property for the underlying hash function [HW15, OPWW15]. We refer to [CJJ22] for more details.

Is CJJ already partially hiding? Unfortunately, the above approach leaks the input. There are two main reasons. First, the configuration digest c is a deterministic hash value, thus it could leak the starting configuration that contains x . Second, standard BARGs do not hide the witnesses, thus the batch proof could leak information about the bits read/written during a single computation step.

In order to control this leakage, we generalize the CJJ template and rely on additional standard tools. Our idea is to encrypt each hashed configuration hcf_i and use a zero-knowledge BARG (zkBARG) for proving local consistency. However, a straightforward execution of the above idea will not work. The reason is that the verifier does not know x_{pr} , thus it cannot check whether the final digest c was correctly computed. This is an essential step in the CJJ template. In eRAM, however, the verifier only gets x_{pb} and not x_{pr} . Therefore, we need the eRAM verifier to be able to verify the accuracy of digest c given only x_{pb} .

To address this, we partition each RAM configuration into two parts — first part contains only x_{pb} , while second part contains x_{pr} and rest of the RAM configuration (such as work tape). Next, we hash these two components separately and only encrypt the second part. This gives us a deterministic hash of x_{pb} and randomized hash of everything else (including x_{pr} , work tape). We view these two hashes jointly as the new hashed configuration hcf , which are hashed as before to create the final digest c . Now the eRAM prover uses a zkBARG to prove local correctness à la CJJ, with one modification that it additionally uses encryption randomness as a witness. Finally, the prover outputs the zkBARG proof, digest c , and a hash opening to the digest of x_{pb} w.r.t. c . Given all of this, a verifier creates a digest for x_{pb} , and checks the validity of its opening w.r.t. digest c . The rest of zkBARG proof verification proceeds as before. This seems to resolve the previous issue of eRAM verifier struggling to verify digest c given only x_{pb} . However, we do not know how to prove security of the above design. The reason is that to prove soundness CJJ crucially relied on the reduction algorithm (verifier) knowing the full input x . We don't have that in case of eRAM, thus the CJJ proof strategy does not directly apply.

Our intuition is that this final problem can be resolved if the reduction algorithm can extract x_{pr} from a valid proof by using some trapdoor. To execute this, we make one last change. We no longer hash x_{pr} before encryption, but simply encrypt it as is along with the hash of the work tape (and rest of the RAM configuration). The advantage of the above is that the reduction algorithm can use the secret key for the encryption system as the trapdoor to extract x_{pr} and use that to

implement a CJJ-style proof strategy for proving soundness. We point out that since we no longer hash down x_{pr} , thus the final proof size scales at least linearly with $|x_{\text{pr}}|$. However, it grows only poly-logarithmically with $|x_{\text{pb}}|$ and run-time of $\mathcal{M}$, thereby satisfies relaxed compactness property for eRAM. Finally, partial input hiding property follows from semantic security of encryption and zero-knowledge property of zkBARGs.

In the main body, rather than fully opening up the CJJ construction, we provide a more direct semi-generic construction for eRAM. We prove that the CJJ RAM construction can be extended to work in a ‘split memory’ model and use this model to directly build eRAM. Additionally, we introduce a *reusability* notion for eRAM, and describe how our eRAM construction can be easily turned into a reusable eRAM scheme. To understand reusable eRAM, consider the following scenario — a user wants to create multiple eRAM proofs π_1, π_2 (and so on) for inputs x_1, x_2 (and so on), where the private portion of each input is identical (i.e. $x_{1,\text{priv}} = x_{2,\text{priv}} \dots$). A reusable eRAM lets us amortize the cost of x_{pr} across many proofs. Basically, using reusable eRAM a user can encode x_{pr} just once, and this encoding can be reused in all proofs that share the same private input. The added advantage of generating this encoding is that the non-reusable/fresh part in each new proof is very short, and no longer grows with even $|x_{\text{pr}}|$ anymore. As we explain next, reusable eRAM is very useful in MPC settings. More details are provided in Section 4.

Complexity-Preserving Semi-Malicious to Malicious MPC. Multiparty computation (MPC) [Yao86, GMW87] protocols are a staple in modern cryptography. They allow mutually distrusting parties to jointly and securely compute a function f over distributed inputs. The most desirable notion of security for MPC protocols is of full malicious security, where adversaries can behave arbitrarily to learn the honest parties’ inputs. Typically, designing a maliciously secure MPC protocol is a difficult task. Therefore, a popular approach in the literature is to first design an MPC that is secure in a weaker semi-malicious model, and later compile it to the (full) malicious model. One such compiler was originally presented by Asharov et al. [AJLA⁺12], but it was not very efficient. In particular, if they start with a semi-malicious protocol with optimal communication complexity, the compiled malicious protocol is very far from optimal communication complexity. While it is known that under a strong non-falsifiable assumption (such as zk-SNARKs) the blowup in communication complexity can be avoided [DGMR21], we do not yet have any approaches to achieve the same under standard falsifiable assumptions.

We show that reusable eRAM is a great tool to build (optimal) complexity-preserving compilers from semi-malicious MPC to malicious MPC. By complexity-preserving, we mean that the communication overhead added by the compiler is independent of the size of the MPC functionality, inputs, or the number of parties. To the best of our knowledge, our work provides the *first* complexity-preserving compiler for semi-malicious to malicious security from standard falsifiable assumptions.

For ease of exposition, we describe our compiler in the context of multiparty reusable non-interactive secure computation (mrNISC) protocols [BL20]. In a mrNISC protocol, each party (say P_i) encodes its private inputs (say x_i) using a special **Com** algorithm such that this encoding can be reused across many different protocol runs. Each encoding, $\hat{x}_i$, is widely accessible via a public bulletin board. Consider any set S of parties, say $\{P_i\}_{i \in S}$, that want to jointly evaluate a function f on their private inputs $\{x_i\}_{i \in S}$. A mrNISC protocol allows each party, P_i , to individually (and without interaction) create an evaluated encoding $\hat{f}_i$ using the **FuncEnc** algorithm. **FuncEnc** only requires the input encodings $\{\hat{x}_j\}_{j \in S}$ and the *de-commitment* information corresponding to $\hat{x}_i$.

All these encodings together can be used to learn $f(x_1, \dots, x_n)$, but nothing more. Essentially, mrNISC makes the first round of the protocol to be independent of the function f and the set S of parties. Thus, it brings down the cost of computing k functions to just $k + 1$ rounds. Over the last few years, multiple constructions for mrNISC with semi-maliciously security from standard assumptions have been proposed [BL20, BJKL21, AJJM21, Shi22]. Next, we boost their security to malicious security with identical (asymptotic) communication complexity.

Malicious security via reusable eRAM. During the input encoding process, each party creates a commitment $\hat{x}_i$ for its private input x_i using the semi-malicious protocol and, additionally, it generates a reusable eRAM encoding $\hat{\pi}_i$ for x_i and commitment randomness as the reusable eRAM *private* input. Together $\hat{\pi}_i$ and $\hat{x}_i$ correspond to the new input encoding for our malicious protocol.

To generate a function encoding, each party first generates the semi-malicious function encoding $\hat{f}_i$. Next, it generates an eRAM proof using the semi-malicious function encoding algorithm as the RAM machine and $\hat{f}_i$ and all the encodings $\{\hat{x}_j\}_{j \in S}$ as the eRAM *public* input. Basically, the RAM machine checks that $\hat{f}_i$ was correctly computed given input x_i , its commitment randomness, and public encodings of rest of the parties. Each party outputs the eRAM proof and the semi-malicious function encoding as the new function encoding. An evaluator runs eRAM verifier to check each proof is valid and then uses the semi-malicious evaluator to learn the output of the computation.

The correctness of the malicious protocol follows in a straightforward way. Moreover, we can rely on the partial knowledge soundness and input hiding property for our reusable eRAM to reduce any successful attacker on our new protocol to a successful semi-malicious attacker on the base protocol. The proof relies on the fact that partial knowledge soundness can be used to extract private inputs for malicious parties, while eRAM input hiding property and semi-malicious security of underlying MPC can be used to simulate the honest parties' messages.

Interestingly, the above compiler preserves the function class for the underlying mrNISC protocol, but it is not fully complexity-preserving yet. The reason is that it turns out the commitment randomness sampled by each party at the start must be as long as each input x_i . Thus, even when our reusable eRAM provides best possible succinctness, the final input encoding will be at least *two* times as large (i.e., a blowup of at least a multiplicative factor of 2).

Thus, to compile a semi-malicious protocol in a truly complexity-preserving manner, we need a few more ideas. At a very high level, our insight is to use another level of delegation within our compiler, and take out each parties' private input x_i and the corresponding commitment randomness from the eRAM private input, and move it to the public input. While this could possibly get around the multiplicative overhead issue, we need to ensure x_i (and the commitment randomness) is still hidden.

To this end, our final trick is to have the i -th party sample a short random seed sd_i and use it to encrypt its input x_i as $\hat{x}_i = x_i \oplus \text{PRG}(\text{sd}_i)$. Now rather than using x_i as the private eRAM input, each party uses sd_i as the private eRAM input and includes $\hat{x}_i$ in the public portion of the eRAM input. Thus, party P_i sets $\hat{x}_i$ in addition to the semi-malicious input encoding and an eRAM encoding for sd_i as the final input encoding for the malicious protocol. Observe that this encoding is now really complexity-preserving as each sd is of fixed polynomial length, and the actual input x_i is encrypted using a one-time pad.

Now, to make sure the correctness of the protocol stays intact, we need to slightly adjust the function encoding process as well. In particular, instead of generating function encodings for actual function f , we generate it for a new function g . This new function g has the original function f as well as the one-time input encryptions $\hat{x}_1, \dots, \hat{x}_n$ hardwired, and it takes as input a sequence of

PRG seeds, and outputs the following:

$$g_{[f, \hat{x}_1, \dots, \hat{x}_n]}(\text{sd}_1, \dots, \text{sd}_n) = f(x_1, \dots, x_n)$$

To prove security of the resulting protocol, we can rely on similar ideas as our previous protocol, except we need to additionally rely on pseudorandomness of the PRG. Later, in Section 6, we provide our full construction and discuss its extensions to other MPC protocols.

Additionally, we describe a new maliciously-secure mrNISC protocol (assuming the hardness of circularly secure **LWE**) where the input encoding length is $|x| + \text{poly}(\lambda)$ (i.e., optimal-rate) and the function encoding length is $\ell \cdot \text{poly}(\lambda)$ where ℓ denotes the function output length. Thus, our function encoding length is independent of the computation size. Prior works [AJJM21] informally sketch out method in which function encodings grow with ℓ as well as depth of the function. However, we show that by relying on multi-key FHE [LATV12, CM15, MW16], we can avoid this dependence and realize function encodings with $\ell \cdot \text{poly}(\lambda)$ size in the **crs** model (required for security against malicious adversaries). At a high level, the idea is to delegate the computation of f to multi-key FHE and generate function encodings for multi-key FHE’s decryption function which only depends on ℓ . We provide more details in Section 6. We leave it as an interesting problem to extend this to **SXDH** as well as further reduce the function encoding length making it fully-optimal.

Other applications: composable zkBARGs, strong context hiding in multi-hop homomorphic signatures, etc. As mentioned earlier, the notion of eRAM turns out to be immediately useful for many more applications. We briefly discuss some of them below and leave it as an interesting future research direction to discover more such applications.

Context-Hiding Multi-Hop Multi-Key Homomorphic Signatures. Homomorphic signatures [JMSW02, AB09, BFKW09, BF11a, GVW15b] enable computations on *secretly* signed data. Given a signature σ for database D , they let anybody publicly compute a signature for $f(D)$, for any efficiently computable function f . A highly desirable property for homomorphic signatures is the ability to compute on evaluated signatures (called multi-hop evaluation) and a guarantee that any evaluated signature looks just like a fresh signature (called strong context hiding) [GVW15b].

By plugging our notion of eRAM delegation in prior BARG-based homomorphic signature schemes [ACG24], we obtain a homomorphic signature scheme satisfying general multi-hop homomorphic evaluation and strong context hiding, under sub-exponential hardness of **LWE/DDH**/ $O(1)$ -**LIN**. Furthermore, the homomorphic signature scheme readily supports the multi-key model [GVW15b, FMNP16] as well. Prior to our work, homomorphic signatures satisfying strong multi-hop context hiding were not known from standard falsifiable assumptions. We refer the reader to [ACG24] for a detailed discussion about limitations of prior works.

Composable zkBARGs. Composable or multi-hop batch arguments were recently introduced by Devasdas et al. [DGKV22] as a new tool to design incrementally verifiable computation protocols [Val08] and aggregate signatures with unbounded aggregation [BGLS03]. Informally, a multi-hop BARG allows succinct recursive composition of multiple BARG proofs.

In this work, we provide a simple construction for a multi-hop *zero-knowledge* BARG. Our construction essentially combines the techniques of [DGKV22] with our eRAM delegation object to get the desired zero-knowledge property. As an immediate application, this gives a privacy-preserving incrementally verifiable computation protocol, where by privacy-preserving we mean the incrementally computed proof can hide information about the intermediate computation steps as well as starting input. For completeness, we provide more details in Section 7.

Succinct Delegation of Committed Programs. In a recent work, Ghosal et al. [GSW23] introduced a new notion for SNARGs called semi-trusted SNARGs. Their goal was to assist users that wish to delegate the responsibility of hosting a program to a server. As an immediate application, we show that reusable eRAM immediately gives semi-trusted SNARGs with a stronger zero-knowledge guarantee. For completeness, we give our reusable eRAM based construction in Section 8.

Related work. The concept of eRAM delegation introduced in this work can be viewed as a “partially hiding” generalization of vanilla RAM delegation [Mic94b, KR09, KRR14, BHK17, PR17]. In this work, we do not require strong succinctness from eRAM (which would require proof size to be independent of private input length too). A recent work [CG25] shows any extractable argument system with non-trivial succinctness faces strong barriers under standard falsifiable assumptions [GW11]. Since we require our eRAM system to have partial extractability, thus this recent infeasibility result [CG25] also rules out any efficiency improvement to our eRAM delegation construction under standard falsifiable assumptions.

More broadly, the intuition behind our eRAM notion weakly resembles partially-hiding functional encryption (PHFE) introduced by Gorbunov-Vaikuntanathan-Wee [GVW15a]. PHFE generalizes vanilla FE, where the input to the encryption algorithm is split into two parts (public and private). Additionally, in PHFE, “large” computation can only be performed on public input and only a “small” computation can be carried on secret input. It is known [GVW15a, AJL⁺19, BSW11] that PHFE is far easier to design than general-purpose FE. Drawing inspiration from PHFE, eRAM generalizes (zero-knowledge) RAM delegation in a similar manner, where the size of public input can be large, while the size of private input must be small.

2 Preliminaries

Notation. We denote the security parameter by λ . By PPT, we denote a probabilistic polynomial-time. All polynomials denoted by $\text{poly}(\cdot)$ are positive polynomials. For any finite set $\mathcal{S}$, $x \leftarrow \mathcal{S}$ denotes a uniformly random element $x \in \mathcal{S}$. Similarly for any distribution $\mathcal{D}$, $x \leftarrow \mathcal{D}$ denotes an element x drawn from distribution $\mathcal{D}$. We denote the set of all positive integers up to n as $[n] := \{1, \dots, n\}$. Also, we use $[m, n]$ where $n \geq m$ to denote the set of all integers from m to n , i.e., $[m, n] := \{m, \dots, n\}$. By $\text{negl}(\lambda)$, we define negligible functions. A function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}$ is a negligible function if for every $c \in \mathbb{N}$ and for large enough λ , $\text{negl}(\lambda) < \lambda^{-c}$.

2.1 RAM Programs

A RAM program is typically defined as a deterministic machine $\mathcal{M}$ represented using a fixed polynomial-sized set of states that has random access to a large memory (where the memory includes an explicit input and rest of it is initialized as zeros). In this work, we use the following representation for RAM machines as it enables a simpler exposition of our main ideas.

Any RAM machine $\mathcal{M}$ that receives n bits of explicit input $x = \{0, 1\}^n$, is associated with a work tape of size $S = S(n)$ and a fixed set of states Q that the machine can be in at any point during machine execution. Formally, a machine $\mathcal{M}$ is associated with following components:

- A set of machine states Q .

- A memory M of size $n + S$. Without loss of generality, we assume that the explicit input $x \in \{0, 1\}^n$ is written in the first n cells of the memory (i.e., $x = (M_1, \dots, M_n)$), and the rest of the memory (i.e., $(M_{n+1}, \dots, M_{n+S})$) corresponds to the work-tape of the machine. For ease of exposition, we also use W_i (for $i \in [S]$) to denote the work-tape. In our model, we view the work tape to be initialized as all zeros.
- A state transition function δ with the following syntax:

$$\delta : Q \times \{0, 1\} \rightarrow Q \times \{0, 1\}^{\log(n+S)} \times \{0, 1\}^{\log S} \times \{0, 1\}$$

Here the transition function defines the execution of machine $\mathcal{M}$ at each time step.

There is a circuit $C_{\mathcal{M}}$ that computes the next step as follows:

$$C_{\mathcal{M}}(q, \text{rbit}) = (q', \text{ridx}', \text{widx}, \text{wbit})$$

We use cf_i for the configuration of the machine when run for i steps, and let cf_0 be the initial configuration of the machine. We denote by $\mathcal{M}(z; 1^t)$ running the machine $\mathcal{M}$ starting from cf_0 on input z for t steps that outputs cf_t . The language of machine $\mathcal{M}$ is defined as follows:

$$\mathcal{L}_{\mathcal{M}, T} = \{(z, \text{out}) : \text{cf}_0 = \text{initial configuration} \wedge \mathcal{F}(\text{cf}_T) = \text{out} \wedge \mathcal{M}(z; 1^T) = \text{cf}_T\} \quad (1)$$

where $\mathcal{F}(\cdot)$ is a deterministic function that computes the output of the machine from the final configuration.

In this work, we focus on RAM machines that output a single bit 0/1. In this case, the language would be as follows:

$$\mathcal{L}_{\mathcal{M}, T} = \{(z, b) : \text{cf}_0 = \text{initial configuration} \wedge \mathcal{F}(\text{cf}_T) = b \wedge \mathcal{M}(z; 1^T) = \text{cf}_T\} \quad (2)$$

2.2 Somewhere Extractable (Zero-Knowledge) Batch Arguments

Syntax. A (publicly verifiable and non-interactive) somewhere extractable batch argument scheme **seBARG** for an NP language $\mathcal{L}$ consists of the following polynomial time algorithms:

Setup $(1^\lambda, k, n, i^*) \rightarrow \text{crs}$. This is a probabilistic setup algorithm that takes as input a security parameter 1^λ , number of instances k , input length n , and an index $i^* \in [k]$. It runs in time at most $\text{poly}(\lambda, n, \log k)$ and outputs a common reference string crs .

Prove $(\text{crs}, x_1, \dots, x_k, w_1, \dots, w_k) \rightarrow \pi$. This is a prover algorithm takes as input a crs , k instances $x_1, \dots, x_k$ and corresponding witnesses $w_1, \dots, w_k$, and outputs a proof π .

Verify $(\text{crs}, x_1, \dots, x_k, \pi) \rightarrow 0/1$. The verification algorithm takes as input a common reference string crs , k instances x_i for $i \in [k]$, and a proof π . It outputs 0 (reject) or 1 (accept).

Definition 2.1 (**seBARG**). A somewhere-extractable batch argument scheme **seBARG** = (**Setup**, **Prove**, **Verify**) for $\mathcal{L}$ is required to satisfy the following properties:

Efficiency. The size of the CRS and the proof is at most $\text{poly}(\lambda, \log k, n, m)$, where m is the witness length.

Completeness. For any $\lambda \in \mathbb{N}$, and any $k = k(\lambda)$, $n = n(\lambda)$ of size at most 2^λ , any k instances $x_1, \dots, x_k \in \mathcal{L}$, and their corresponding witnesses $w_1, \dots, w_k \in \{0, 1\}^m$, and any index $i^* \in [k]$,

$$\Pr \left[\text{Verify}(\text{crs}, x_1, \dots, x_k, \pi) = 1 : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, k, n, i^*), \\ \pi \leftarrow \text{Prove}(\text{crs}, x_1, \dots, x_k, w_1, \dots, w_k) \end{array} \right] = 1.$$

Index hiding. For any PPT adversary $\mathcal{A}$, any polynomials $k = k(\lambda)$ and $n = n(\lambda)$, and any indices $i_0, i_i \in [k]$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[b \leftarrow \mathcal{A}(\text{crs}) : \begin{array}{l} b \leftarrow \{0, 1\}, \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, k, n, i_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Somewhere Extraction. There exists a stateful PPT extractor $\mathcal{E}$ such that for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for any polynomials $k = k(\lambda)$ and $n = n(\lambda)$, and any index $i^* \in [k]$, for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{ll} \text{Verify}(\text{crs}, x_1, \dots, x_k, \pi) = 1 & (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, k, n, i^*) \\ \wedge w^* \text{ is not a valid} & : (x_1, \dots, x_k, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ \text{witness for } x_{i^*} \in \mathcal{L} & w^* \leftarrow \mathcal{E}(\text{td}, \{x_i\}_{i \in [k]}, \pi) \end{array} \right] \leq \text{negl}(\lambda).$$

Remark 2.2. We note that the somewhere extraction property implies the following *semi-adaptive soundness* property which asserts that for any PPT adversary $\mathcal{A}$, any polynomials $k = k(\lambda)$ and $n = n(\lambda)$, and any index $i^* \in [k]$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, x_1, \dots, x_k, \pi) = 1 \\ \wedge x_{i^*} \notin \mathcal{L} \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, k, n, i^*) \\ (x_1, \dots, x_k, \pi) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition 2.3 (zkBARG). A somewhere-extractable zero-knowledge batch argument scheme **zkBARG** = (Setup, Prove, Verify) for $\mathcal{L}$ is required to satisfy the following property in addition to all properties under Definition 2.1:

Zero-knowledge. There exists a PPT simulator $\mathcal{S}$ such that for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for any polynomials $k = k(\lambda)$ and $n = n(\lambda)$, and any index $i^* \in [k]$, for every $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[1 \leftarrow \mathcal{A}(\text{crs})^{\text{Prove}(\text{crs}, \cdot, \cdot)} : \text{crs} \leftarrow \text{Setup}(1^\lambda, k, n, i^*) \right] - \Pr \left[1 \leftarrow \mathcal{A}(\text{crs})^{\mathcal{O}^{\mathcal{S}}(\text{crs}, \cdot, \cdot)} : \text{crs} \leftarrow \mathcal{S}(1^\lambda, k, n) \right] \right| \leq \text{negl}(\lambda).$$

where $\mathcal{O}^{\mathcal{S}}((x_1, \dots, x_k), (w_1, \dots, w_k))$ outputs $\mathcal{S}(x_1, \dots, x_k)$ if for every $i \in [k]$, w_i is a valid witness for $x_i \in \mathcal{L}$ and $\perp$ otherwise.

Definition 2.4 (Rate-1 seBARG). An seBARG scheme (Setup, Prove, Verify) is said to be rate-1 if the proof generated by $\text{Prove}(\text{crs}, x_1, \dots, x_k, w_1, \dots, w_k)$ is of length $m + O(m/\lambda) + \text{poly}(\lambda)$.

Theorem 2.5 ([DGKV22, PP22, KLVW23]). Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists rate-1 seBARG.

2.3 Hash Tree

Syntax. A hash tree consists of the following polynomial time algorithms:

$\text{Gen}(1^\lambda) \rightarrow \text{hk}$. This is a probabilistic key generation algorithm that takes as input a security parameter 1^λ , and outputs a hash key hk .

$\text{Hash}(\text{hk}, x) \rightarrow \text{h}$. This is the hashing algorithm that takes as input the hash key hk and an input $x \in \{0, 1\}^*$, and outputs a hash value $\text{h} \in \{0, 1\}^{\text{poly}(\lambda)}$.

$\text{Open}(\text{hk}, x, i) \rightarrow (b, \text{op})$. This is the hash opening algorithm that takes as input the hash key hk , an input $x \in \{0, 1\}^*$ and an index $i \in [|x|]$, and outputs a bit $b \in \{0, 1\}$ and an opening $\text{op} \in \{0, 1\}^{\leq \text{poly}(\lambda)}$.

$\text{Write}(\text{hk}, x, i, b) \rightarrow (\text{h}', \text{op}')$. This is the writing algorithm that takes as input the hash key hk , an input $x \in \{0, 1\}^*$ an index $i \in [|x|]$ and a bit $b \in \{0, 1\}$, and outputs a hash value $\text{h}' \in \{0, 1\}^{\text{poly}(\lambda)}$ an opening $\text{op}' \in \{0, 1\}^{\leq \text{poly}(\lambda)}$.

$\text{VerifyRead}(\text{hk}, \text{h}, i, b, \text{op}) \rightarrow 0/1$. This is a read-verification algorithm that takes as input the hash key hk , a hash value $\text{h} \in \{0, 1\}^{\text{poly}(\lambda)}$, an index $i \in [|x|]$, a bit $b \in \{0, 1\}$ and an opening $\text{op} \in \{0, 1\}^{\leq \text{poly}(\lambda)}$, and outputs 0 (reject) or 1 (accept).

$\text{VerifyWrite}(\text{hk}, \text{h}, i, b, \text{h}', \text{op}') \rightarrow 0/1$. This is a write-verification algorithm that takes as input the hash key hk , a hash value $\text{h} \in \{0, 1\}^{\text{poly}(\lambda)}$, an index $i \in [|x|]$, a bit $b \in \{0, 1\}$, a hash value $\text{h}' \in \{0, 1\}^{\text{poly}(\lambda)}$ and an opening $\text{op}' \in \{0, 1\}^{\leq \text{poly}(\lambda)}$, and outputs 0 (reject) or 1 (accept).

Definition 2.6 (Hash Tree). A hash tree $\text{HT} = (\text{Gen}, \text{Hash}, \text{Open}, \text{Write}, \text{VerifyRead}, \text{VerifyWrite})$ is required to satisfy the following properties:

Efficiency. The size of hash key hk and hash value h is at most $\text{poly}(\lambda)$ and the size of openings is at most $\text{poly}(\lambda, \log n)$ where n is the input size.

Reading Soundness (Collision Resistance w.r.t. Opening). For any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{VerifyRead}(\text{hk}, \text{h}, i, b_i^1, \text{op}_i^1) = 1, \\ \text{VerifyRead}(\text{hk}, \text{h}, i, b_i^2, \text{op}_i^2) = 1, \\ b_i^1 \neq b_i^2 \end{array} : \begin{array}{l} (\text{hk}) \leftarrow \text{Gen}(1^\lambda), \\ (\text{h}, i, b_i^1, \text{op}_i^1, b_i^2, \\ \text{op}_i^2) \leftarrow \mathcal{A}(\text{hk}) \end{array} \right] \leq \text{negl}(\lambda).$$

Writing Soundness. For any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{VerifyWrite}(\text{hk}, \text{h}, i, b, \text{h}_1, \text{op}_1) = 1, \\ \text{VerifyWrite}(\text{hk}, \text{h}, i, b, \text{h}_2, \text{op}_2) = 1, \\ \text{h}_1 \neq \text{h}_2 \end{array} : \begin{array}{l} (\text{hk}) \leftarrow \text{Gen}(1^\lambda), \\ (\text{h}, i, b, \text{h}_1, \text{op}_1, \text{h}_2, \text{op}_2) \\ \leftarrow \mathcal{A}(\text{hk}) \end{array} \right] \leq \text{negl}(\lambda).$$

Remark 2.7. Reading soundness implies collision resistance of the hash tree.

Remark 2.8 ([Mer87]). Hash trees can be constructed using any family $\mathcal{F} : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ of collision-resistance hash functions.

2.4 Non-Interactive Zero-Knowledge Arguments (NIZK)

Consider an NP language $\mathcal{L} = \{x \mid \exists w : \mathcal{R}(x, w) = 1\}$ defined w.r.to relation $\mathcal{R}$.

Syntax. A non-interactive zero-knowledge (NIZK) argument consists of the following polynomial time algorithms:

Setup $(1^\lambda, 1^{n_x}, 1^{n_w}) \rightarrow \text{crs}$. The probabilistic setup algorithm takes as input a security parameter λ , an instance length n_x , an witness length n_w , and outputs a common reference string crs .

Prove $(\text{crs}, x, w) \rightarrow \pi$. The prover algorithm takes as input a common reference string crs , an instance x , and a witness w and outputs a proof π .

Verify $(\text{crs}, x, \pi) \rightarrow 0/1$. The verifier algorithm takes as input a common reference string crs , an instance x , and a proof π . It outputs 0 (reject) or 1 (accept).

Definition 2.9 (NIZK). A non-interactive zero-knowledge proof (Setup, Prove, Verify) for $\mathcal{L}$ is required to satisfy the following properties:

Completeness. For all $\lambda, n_x, n_w \in \mathbb{N}$ and $(x, w) \in \mathcal{R}$ where $|x| = n_x$ and $|w| = n_w$ we have:

$$\Pr[\text{Verify}(\text{crs}, x, \pi) = 1 : \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^{n_x}, 1^{n_w}), \pi \leftarrow \text{Prove}(\text{crs}, x, w)] = 1.$$

Adaptive Soundness. For any PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n_x \in \mathbb{N}$:

$$\Pr \left[\text{Verify}(\text{crs}, x, \pi) = 1 \wedge x \notin \mathcal{L} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^{n_x}, 1^{n_w}), \\ (x, \pi) \leftarrow \mathcal{A}(\text{crs}), |x| = n_x \end{array} \right] \leq \text{negl}(\lambda)$$

Zero-Knowledge. There exists a stateful PPT simulator $\mathcal{S}$ such that for any PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n_x, n_w \in \mathbb{N}$:

$$\begin{aligned} & \left| \Pr[\mathcal{A}^{\text{Prove}(\text{crs}, \cdot, \cdot)}(\text{crs}) = 1 : \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^{n_x}, 1^{n_w})] - \right. \\ & \quad \left. \Pr[\mathcal{A}^{\mathcal{O}^\mathcal{S}(\cdot, \cdot)}(\text{crs}) = 1 : \text{crs} \leftarrow \mathcal{S}(1^\lambda, 1^{n_x}, 1^{n_w})] \right| \leq \text{negl}(\lambda) \end{aligned}$$

where $\mathcal{O}^\mathcal{S}(x, w)$ outputs $\mathcal{S}(x)$ if $x \in \mathcal{L}$ and $\perp$ otherwise.

Straight-line Extractor. There exists a stateful PPT extractor $\mathcal{E}$ such that for any non-uniform PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n_x, n_w \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\overline{\text{crs}}, x, \pi) = 1, \\ \wedge (\mathcal{R}(x, w) = 0 \vee |w| > n_w) \end{array} : \begin{array}{l} (\overline{\text{crs}}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, 1^{n_x}, 1^{n_w}), \\ (x, \pi) \leftarrow \mathcal{A}(\overline{\text{crs}}), \\ |x| = n_x, \\ w \leftarrow \mathcal{E}(\text{td}, x, \pi) \end{array} \right] \leq \text{negl}(\lambda).$$

and $\overline{\text{crs}}$ and $\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^{n_x}, 1^{n_w})$ are computationally indistinguishable.

Remark 2.10 ([CW23, BWW24, BKP⁺24]). Assuming seBARGs there exists NIZKs.

Definition 2.11 (NIZK with strong extraction). A NIZK scheme $(\text{Setup}, \text{Prove}, \text{Verify})$ for language $\mathcal{L}$ is said to be a NIZK scheme with strong extraction if it satisfies Definition 2.9 and the following property:

Strong Extractor. There exists a stateful PPT extractor $\mathcal{E}$ such that for any non-uniform PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n_x, n_w \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\overline{\text{crs}}, x, \pi) = 1, \\ \wedge (\mathcal{R}(x, w) = 0 \vee |w| > n_w) \end{array} : \begin{array}{l} (\overline{\text{crs}}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, 1^{n_x}, 1^{n_w}), \\ (x, \pi) \leftarrow \mathcal{A}(\overline{\text{crs}}, \text{td}), \\ |x| = n_x, \\ w \leftarrow \mathcal{E}(\text{td}, x, \pi) \end{array} \right] \leq \text{negl}(\lambda).$$

Note that this property implies straight-line extraction property from Definition 2.9.

Remark 2.12 (NIZK with strong extraction). Strong extraction is an additional property which can be achieved using a PKE scheme (Definition 2.16) in conjunction with soundness of NIZK scheme.

Definition 2.13 (Optimally Efficient NIZK). A NIZK scheme $(\text{Setup}, \text{Prove}, \text{Verify})$ for language $\mathcal{L}$ is said to be an optimally efficient NIZK if it satisfies Definition 2.9 and the size of the proof π is $|w| + \text{poly}(\lambda, \log |x|, \log |w|)$, i.e., rate-1 proof size and running time of Verify is independent of size of $\mathcal{R}$. Moreover, we require that the setup of suzh NIZK scheme be oblivious to the language; i.e., the CRS is universal and independent of the language, and in fact, the language description can depend on the CRS.

2.5 Rate-1 Message Encoding

A rate-1 message encoding is a one-time encryption system, with the special guarantee that the secret key length is independent of message length. Such an encoding follows directly by combining a one-time pad with a pseudorandom generator/function. Since we also use such encodings in our MPC compiler, we decided to abstract it out. Below is the formal definition.

Definition 2.14 (r1Enc). A rate-1 message encoding r1Enc consists of two algorithms: $(\text{Encode}, \text{Decode})$. Encode takes as input a security parameter λ , a string $x \in \{0, 1\}^*$, and outputs an encoded value ρ and secret key sk . Decode takes ρ and sk as inputs, and outputs a string x . Together they must satisfy the following:

Completeness and compactness. For any $\lambda \in \mathbb{N}$, $x \in \{0, 1\}^*$, $\text{Decode}(\rho, \text{sk}) = x$, $|\rho| = |x|$, and $|\text{sk}| = \text{poly}(\lambda)$, where $(\rho, \text{sk}) \leftarrow \text{Encode}(1^\lambda, x)$.

Security. For any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \mathcal{A}(\rho_b) = b \end{array} : \begin{array}{l} x \leftarrow \mathcal{A}(1^\lambda), b \leftarrow \{0, 1\} \\ (\rho_0, \text{sk}) \leftarrow \text{Encode}(1^\lambda, x), \\ \rho_1 \leftarrow \{0, 1\}^{|x|} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Remark 2.15. A r1Enc scheme can be constructed using any family of pseudorandom functions such that $\text{PRF}_\lambda : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}$ used in counter mode. In addition, sk will be a λ -bit string.

2.6 Public-Key Encryption System

Syntax. A public key encryption (PKE) scheme for the message space $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial time algorithms.

$\text{Setup}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$. The probabilistic setup algorithm takes as input a security parameter 1^λ and outputs the public and secret key pair (pk, sk) .

$\text{Enc}(\text{pk}, m) \rightarrow \text{ct}$. The probabilistic encryption algorithm takes as input the public key pk , a message $m \in \mathcal{M}_\lambda$, and outputs the ciphertext ct .

$\text{Dec}(\text{sk}, \text{ct}) \rightarrow m'$. The decryption algorithm takes as input secret key sk , ciphertext ct , and outputs m' .

Definition 2.16 (PKE). A public-key encryption system $(\text{Setup}, \text{Enc}, \text{Dec})$ for $m \in \mathcal{M}_\lambda$ is required to satisfy the following properties:

Correctness. For any $\lambda \in \mathbb{N}$, $m \in \mathcal{M}_\lambda$, we have that $\text{Dec}(\text{sk}, \text{ct}) = m$ where $\text{ct} \leftarrow \text{Enc}(\text{pk}, m)$ and $(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda)$.

Security. For any stateful PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \mathcal{A}^{\text{Enc}(\text{pk}, \cdot)}(1^\lambda, \text{pk}) \right] - \Pr \left[1 \leftarrow \mathcal{A}^{\text{Enc}(\text{pk}, 0^{|m|})}(1^\lambda, \text{pk}) \right] \right| \leq \text{negl}(\lambda)$$

where $(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda)$.

2.7 RAM Delegation Scheme

Syntax. A publicly verifiable non-interactive delegation scheme for RAM machine $\mathcal{M}$ w.r.t. a hash tree HT with hash key ht.hk consists of the following PPT algorithms:

$\text{Setup}(1^\lambda, T) \rightarrow \text{crs}$: The setup algorithm takes as input security parameter λ and running time bound T . It outputs crs .

$\text{Digest}(\text{crs}, \text{cf}) \rightarrow \text{h}$: This is a deterministic polynomial time algorithm that takes as input a crs , and a configuration cf and outputs a digest h .

$\text{Prove}(\text{crs}, \text{cf}_0) \rightarrow (b, \pi)$: The prover algorithm takes as input a common reference string crs , and an initial configuration cf_0 , and outputs a bit b and a proof π .

$\text{Verify}(\text{crs}, \text{h}_0, b, \pi) \rightarrow \{0, 1\}$: The verifier algorithm takes as input crs , a digest of the initial configuration h_0 , a bit b , and a proof π , and outputs either 0 or 1.

Definition 2.17 (RAM Delegation). A publicly verifiable non-interactive RAM delegation scheme $(\text{Setup}, \text{Digest}, \text{Prove}, \text{Verify})$ for $\mathcal{M}$ with setup time T_S and proof length ℓ_π is required to satisfy the following properties:

Completeness. For every polynomial $\lambda, T \in \mathbb{N}$ s.t. $T \leq 2^\lambda$, and $\text{cf}_0 \in \{0, 1\}^*$ such that $\text{cf}_0 \in \mathcal{L}_{\mathcal{M}, T}$ it holds that

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, \text{h}_0, b, \pi) = 1 \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, T) \\ \text{h}_0 \leftarrow \text{Digest}(\text{crs}, \text{cf}_0) \\ (b, \pi) = \text{Prove}(\text{crs}, \text{cf}_0) \end{array} \right] = 1.$$

Efficiency. In the completeness experiment above,

- Setup runs in time T_S .
- Digest on input cf runs in time $|cf| \cdot \text{poly}(\lambda)$ and outputs a digest of length λ .
- Prove runs in time $\text{poly}(\lambda, T, |cf|)$ and output a proof of length ℓ_π .
- Verify runs in time $\mathcal{O}(\ell_\pi) + \text{poly}(\lambda)$.

Collision Resistance. For every PPT adversary $\mathcal{A}$ and pair of polynomials $T = T(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} cf \neq cf' \\ \text{Digest}(crs, cf) = \text{Digest}(crs, cf') \end{array} : \begin{array}{l} crs \leftarrow \text{Setup}(1^\lambda, T) \\ (cf, cf') \leftarrow \mathcal{A}(crs) \end{array} \right] \leq \text{negl}(\lambda).$$

Soundness. For every PPT adversary $\mathcal{A}$ and pair of polynomials $T = T(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(crs, h_0, b, \pi) = 1 \\ cf_0 \notin \mathcal{L}_{\mathcal{M}, T} \end{array} : \begin{array}{l} crs \leftarrow \text{Setup}(1^\lambda, T) \\ (cf_0, b, \pi) \leftarrow \mathcal{A}(crs) \\ h_0 \leftarrow \text{Digest}(crs, cf_0) \end{array} \right] \leq \text{negl}(\lambda).$$

Strong Soundness. For every PPT adversary $\mathcal{A}$ and pair of polynomials $T = T(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(crs, h_0, 0, \pi^{(0)}) = 1 \\ \text{Verify}(crs, h_0, 1, \pi^{(1)}) = 1 \end{array} : \begin{array}{l} crs \leftarrow \text{Setup}(1^\lambda, T) \\ (h_0, \pi^{(0)}, \pi^{(1)}) \leftarrow \mathcal{A}(crs) \end{array} \right] \leq \text{negl}(\lambda).$$

Remark 2.18 ([CJJ22]). Assuming seBARGs and SEH there exists RAM delegation.

3 Defining (Reusable) Encrypted RAM Delegation

In this section we formally define the notions of encrypted RAM delegation and reusable encrypted RAM delegation.

3.1 Encrypted RAM Delegation

Syntax. An encrypted RAM delegation scheme for a RAM machine $\mathcal{M}$ consists of the following PPT algorithms:

Setup $(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \rightarrow crs$. The probabilistic setup algorithm takes as input a security parameter 1^λ , a public input length n_{pb} , a private input length n_{pr} , a max configuration size S for the RAM machine, and the running time T of the machine. It outputs a common reference string crs .

Prove $(crs, x_{\text{pb}}, x_{\text{pr}}) \rightarrow (b, \pi)$. The proving algorithm takes as input the common reference string crs , a public input value x_{pb} and a private input value x_{pr} . It outputs a bit b and a proof π .

Verify $(crs, h_{\text{pb}}, b, \pi) \rightarrow 0/1$. The verification algorithm takes as input the common reference string crs , a digest h_{pb} of the public input, a bit b and a proof π . It outputs 0 (reject) or 1 (accept).

Definition 3.1 (Encrypted RAM Delegation). An encrypted RAM delegation scheme ($\text{Setup}, \text{Prove}, \text{Verify}$) for $\mathcal{M}$ associated with a hash tree HT with hash key ht.hk has to satisfy the following property:

Completeness. For every $\lambda, T, S, n_{\text{pb}}, n_{\text{pr}} \in \mathbb{N}$ s.t. $T, S, n_{\text{pb}} \leq 2^\lambda$, $n_{\text{pr}} = \text{poly}(\lambda)$, $x_{\text{pb}} \in \{0, 1\}^{n_{\text{pb}}}$, $x_{\text{pr}} \in \{0, 1\}^{n_{\text{pr}}}$, $(x_{\text{pb}}, x_{\text{pr}}) \in \mathcal{L}_{\mathcal{M}, T}$ it holds that:

$$\Pr \left[\text{Verify}(\text{crs}, h_{\text{pb}}, b, \pi) = 1 \quad : \quad \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \\ (b, \pi) \leftarrow \text{Prove}(\text{crs}, x_{\text{pb}}, x_{\text{pr}}) \end{array} \right] = 1.$$

Efficiency. The crs and proof size, and the setup and the verifier running time are $\text{poly}(\lambda, \log T, \log n_{\text{pb}}, n_{\text{pr}})$, and the prover's running time is $\text{poly}(\lambda, T, n_{\text{pb}}, n_{\text{pr}})$.

Input Hiding. There exists a stateful PPT simulator $\mathcal{S}$ such that for any PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n_{\text{pb}}, n_{\text{pr}}, S, T \in \mathbb{N}$:

$$\left| \Pr[1 \leftarrow \mathcal{A}^{\text{Prove}(\text{crs}, \cdot, \cdot)}(\text{crs}) : \text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)] - \Pr[1 \leftarrow \mathcal{A}^{\mathcal{O}^{\mathcal{S}}(\cdot, \cdot)}(\text{crs}) : \text{crs} \leftarrow \mathcal{S}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)] \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}^{\mathcal{S}}(x_{\text{pb}}, x_{\text{pr}})$ outputs $\mathcal{S}(x_{\text{pb}})$ if $\mathcal{M}(x_{\text{pb}}, x_{\text{pr}}) = 1$ and $\perp$ otherwise.

Soundness. For any stateful PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, h_{\text{pb}}, b, \pi) = 1, \\ \wedge \forall x_{\text{pr}} : (x_{\text{pb}}, x_{\text{pr}}) \notin \mathcal{L}_{\mathcal{M}, T} \end{array} : \begin{array}{l} (n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \leftarrow \mathcal{A}(1^\lambda), \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T), \\ (x_{\text{pb}}, b, \pi) \leftarrow \mathcal{A}(\text{crs}), \\ h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}}) \end{array} \right] \leq \text{negl}(\lambda) .$$

We remark that the above soundness property is implied by the straight-line extraction property.

Straight-line Extraction. There exists a stateful PPT extractor $\mathcal{E}$ such that for any stateful PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, h_{\text{pb}}, b, \pi) = 1 \\ \wedge (x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M}, T} \end{array} : \begin{array}{l} (n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T), \\ (x_{\text{pb}}, b, \pi) \leftarrow \mathcal{A}(\text{crs}), \\ h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}}), \\ x_{\text{pr}}^* \leftarrow \mathcal{E}(\text{td}, \pi) \end{array} \right] \leq \text{negl}(\lambda) .$$

such that $\text{crs}_1 \leftarrow \mathcal{E}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ and $\text{crs}_2 \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ are indistinguishable.

3.2 Reusable Encrypted RAM Delegation

Syntax. A reusable encrypted RAM delegation scheme for a RAM machine $\mathcal{M}$ consists of the following PPT algorithms:

$\text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \rightarrow \text{crs}$. This is the same as non-reusable encrypted RAM delegation.

$\text{PrivEnc}(\text{crs}, x_{\text{pr}}) \rightarrow (\text{eh}_{\text{pr}}, \pi_{\text{enc}}, \text{aux})$. The private-input encoding algorithm takes as input the common reference string crs and a private input value x_{pr} . It outputs an encrypted digest eh_{pr} , a proof π_{enc} and some auxiliary value aux .

$\text{Prove}(\text{crs}, x_{\text{pb}}, \text{aux}) \rightarrow (b, \pi_{\text{edel}})$. The proving algorithm takes as input the common reference string crs , a public input value x_{pb} and an auxiliary value aux . It outputs a bit b and a proof π_{edel} .

$\text{VerifyEnc}(\text{crs}, \text{eh}_{\text{pr}}, \pi_{\text{enc}}) \rightarrow 0/1$. The encoding verification algorithm takes as input the common reference string crs , an encrypted digest eh_{pr} and a proof π_{enc} . It outputs 0 (reject) or 1 (accept).

$\text{Verify}(\text{crs}, h_{\text{pb}}, \text{eh}_{\text{pr}}, b, \pi_{\text{edel}}) \rightarrow 0/1$. The verification algorithm takes as input the common reference string crs , a digest h_{pb} of the public input, an encrypted digest eh_{pr} of the private input, a bit b , and a proof π_{edel} . It outputs 0 (reject) or 1 (accept).

Definition 3.2 (Reusable Encrypted RAM Delegation). A reusable encrypted RAM delegation scheme (Setup , PrivEnc , Prove , VerifyEnc , Verify) for $\mathcal{M}$ associated with a hash tree HT with hash key ht.hk has to satisfy the following property:

Completeness. For every $\lambda, T, S, n_{\text{pb}}, n_{\text{pr}} \in \mathbb{N}$ s.t. $T, S, n_{\text{pb}} \leq 2^\lambda$, $n_{\text{pr}} = \text{poly}(\lambda)$, $x_{\text{pb}} \in \{0, 1\}^{n_{\text{pb}}}$, $x_{\text{pr}} \in \{0, 1\}^{n_{\text{pr}}}$, $(x_{\text{pb}}, x_{\text{pr}}) \in \mathcal{L}_{\mathcal{M}, T}$ it holds that:

$$\Pr \left[\begin{array}{l} \text{VerifyEnc}(\text{crs}, \text{eh}_{\text{pr}}, \pi_{\text{enc}}) = 1 \wedge \\ \text{Verify}(\text{crs}, h_{\text{pb}}, \text{eh}_{\text{pr}}, b, \pi_{\text{edel}}) = 1 \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \\ (\text{eh}_{\text{pr}}, \pi_{\text{enc}}, \text{aux}) \leftarrow \text{PrivEnc}(\text{crs}, x_{\text{pr}}) \\ (b, \pi_{\text{edel}}) \leftarrow \text{Prove}(\text{crs}, x_{\text{pb}}, \text{aux}) \end{array} \right] = 1.$$

Efficiency. For the completeness experiment above,

- The size of the crs and setup running time are $\text{poly}(\lambda, \log T, \log n_{\text{pb}}, n_{\text{pr}})$.
- The size of π_{enc} , the private encoding and its verification time are $\text{poly}(\lambda, n_{\text{pr}})$.
- Prover's running time is $\text{poly}(\lambda, T, n_{\text{pb}}, n_{\text{pr}})$.
- The size of π_{edel} and the verification time are $\text{poly}(\lambda, \log T, \log n_{\text{pb}}, \log n_{\text{pr}})$.

Input Hiding. There exists a stateful PPT simulator $\mathcal{S}$ such that for any PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n_{\text{pb}}, n_{\text{pr}}, S, T \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \text{Expt}_0^{\mathcal{C}, \mathcal{A}}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \right] - \Pr \left[1 \leftarrow \text{Expt}_1^{\mathcal{S}, \mathcal{A}}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \right] \right| \leq \text{negl}(\lambda)$$

where definitions of $\text{Expt}_0^{\mathcal{C}, \mathcal{A}}$ and $\text{Expt}_1^{\mathcal{S}, \mathcal{A}}$ are provided in Figure 1.

Soundness. For any stateful PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{VerifyEnc}(\text{crs}, \text{eh}_{\text{pr}}, \pi_{\text{enc}}) = 1 \\ \wedge \text{Verify}(\text{crs}, h_{\text{pb}}, \text{eh}_{\text{pr}}, b, \pi_{\text{edel}}) = 1 \\ \wedge \forall x_{\text{pr}} : (x_{\text{pb}}, x_{\text{pr}}) \notin \mathcal{L}_{\mathcal{M}, T} \end{array} : \begin{array}{l} (n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \leftarrow \mathcal{A}(1^\lambda), \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T), \\ (x_{\text{pb}}, \text{eh}_{\text{pr}}, \pi_{\text{enc}}, b, \pi_{\text{edel}}) \leftarrow \mathcal{A}(\text{crs}), \\ h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}}) \end{array} \right] \leq \text{negl}(\lambda) .$$

$\text{Expt}_0^{\mathcal{C}, \mathcal{A}}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$. This is the real experiment parameterized by an honest challenger $\mathcal{C}$. $\mathcal{A}$ receives $\text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from $\mathcal{C}$. $\mathcal{A}$ makes the following queries in an adaptive manner. After this, $\mathcal{A}$ outputs guess b' . Output b' .

- **PRIVATE INPUT ENCODING:** $\mathcal{A}$ sends x_{pr} and receives $(\text{eh}_{\text{pr}}, \pi_{\text{enc}}, i)$ where $(\text{eh}_{\text{pr}}, \pi_{\text{enc}}, \text{aux}_i) \leftarrow \text{PrivEnc}(\text{crs}, x_{\text{pr}})$ and i is an index that after each query of this type is incremented.
- **PROVE:** $\mathcal{A}$ sends (x_{pb}, i) and receives π_{edel} where $(b, \pi_{\text{edel}}) \leftarrow \text{Prove}(\text{crs}, x_{\text{pb}}, \text{aux}_i)$.

$\text{Expt}_1^{\mathcal{S}, \mathcal{A}}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$. This is the ideal experiment parameterized by a stateful simulator $\mathcal{S}$. $\mathcal{A}$ receives $\text{crs} \leftarrow \mathcal{S}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from $\mathcal{C}$. $\mathcal{A}$ makes the following queries in an adaptive manner. After this, $\mathcal{A}$ outputs guess b' . Output b' .

- **PRIVATE INPUT ENCODING:** $\mathcal{A}$ sends x_{pr} and receives $(\text{eh}_{\text{pr}}, \pi_{\text{enc}}, i)$ where $(\text{eh}_{\text{pr}}, \pi_{\text{enc}}) \leftarrow \mathcal{S}(1^{|x_{\text{pr}}|}, i)$ and i is an index that after each query of this type is incremented.
- **PROVE:** $\mathcal{A}$ sends (x_{pb}, i) , and if $(x_{\text{pb}}, x_{\text{pr}}) \in \mathcal{L}_{\mathcal{M}, T}$ (for x_{pr} corresponding to i) receives π_{edel} where $\pi_{\text{edel}} \leftarrow \mathcal{S}(x_{\text{pb}}, i)$, otherwise, receives $\perp$.

Figure 1: Real and ideal experiments for Input Hiding property of reusable encrypted RAM delegation.

We remark that the above soundness property is implied by the straight-line extraction property explained further below.

Strong Soundness. For any stateful PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, h_{\text{pb}}, \text{eh}_{\text{pr}}, 0, \pi_{\text{edel}}^{(0)}) = 1 \\ \wedge \text{Verify}(\text{crs}, h_{\text{pb}}, \text{eh}_{\text{pr}}, 1, \pi_{\text{edel}}^{(1)}) = 1 \end{array} : \begin{array}{l} (n_{\text{pb}}, n_{\text{pr}}, S, T) \leftarrow \mathcal{A}(1^\lambda), \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, n_{\text{pr}}, S, T), \\ (h_{\text{pb}}, \text{eh}_{\text{pr}}, \pi_{\text{edel}}^{(0)}, \pi_{\text{edel}}^{(1)}) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda) .$$

Straight-line Extraction. There exists a stateful PPT extractor $\mathcal{E}$ such that for any stateful PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n_x \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{VerifyEnc}(\text{crs}, \text{eh}_{\text{pr}}, \pi_{\text{enc}}) = 1 \\ \wedge \text{Verify}(\text{crs}, h_{\text{pb}}, \text{eh}_{\text{pr}}, b, \pi_{\text{edel}}) = 1 \\ \wedge (x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M}, T} \end{array} : \begin{array}{l} (n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T), \\ (\text{eh}_{\text{pr}}, \pi_{\text{enc}}) \leftarrow \mathcal{A}(\text{crs}), \\ x_{\text{pr}}^* \leftarrow \mathcal{E}(\text{td}, \pi_{\text{enc}}), \\ (x_{\text{pb}}, b, \pi_{\text{edel}}) \leftarrow \mathcal{A}(\text{crs}), \\ h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}}) \end{array} \right] \leq \text{negl}(\lambda) .$$

such that $\text{crs}_1 \leftarrow \mathcal{E}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ and $\text{crs}_2 \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ are indistinguishable.

4 Constructing (Reusable) Encrypted RAM Delegation

In this section, we first informally define a split memory RAM delegation scheme and then show how to combine this object with PKEs and NIZKs to construct a (reusable) encrypted RAM delegation.

Informally speaking, a split memory RAM delegation scheme allows one to split the memory of a RAM computation into several partitions, then prove the correctness of a RAM computation w.r.t. the partitioned memory and verify the proof w.r.t. independent partial digests of such partitions. Namely, the digest algorithm uses a hash tree to independently generate a digest h_i of each partition P_i and we let the final digest to be $h = (h_1, \dots, h_n)$. Syntactically, this object is the same as a RAM delegation scheme except that the setup algorithm takes as input a partitioning $\mathbf{P} = (P_1, \dots, P_n)$ of the memory, and generates a crs w.r.t. this partitioning. Note that the partitioning of the memory can be determined by the partition sizes, hence, in this work we use $\mathbf{s} = (s_1, \dots, s_n)$ to define a partitioning $\mathbf{P} = (P_1, \dots, P_n)$ where $|P_i| = s_i$ and $P_i = [\sum_{j=1}^{i-1} s_j + 1, \dots, \sum_{j=1}^i s_j]$. We refer the readers to Appendix B for a formal definition and a construction based on [CJJ22].

4.1 Encrypted RAM Delegation

Language $\mathcal{L}_{\text{enc}}$	Language $\mathcal{L}_{\text{edel}}$
Hardwired: $\text{ht.hk}, \text{pke.pk}$	Hardwired: $\text{ht.hk}, \text{pke.pk}$
Instance: $x_{\text{enc}} = \text{eh}_{\text{pr}}$	Instance: $x_{\text{edel}} = (\text{h}_{\text{pb}}, \text{eh}_{\text{pr}}, \text{h}_{\Sigma}, b)$
Witness: $w_{\text{enc}} = (x_{\text{pr}}, \text{h}_{\text{pr}}, r)$	Witness: $w_{\text{edel}} = (\text{h}_{\text{pr}}, r, \text{del}.\pi)$
Output: $(x_{\text{enc}}, w_{\text{enc}}) \in \mathcal{L}_{\text{enc}}$ if the following hold:	Output: $(x_{\text{edel}}, w_{\text{edel}}) \in \mathcal{L}_{\text{edel}}$ if the following hold:
– $\text{h}_{\text{pr}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pr}})$. – $\text{eh}_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, \text{h}_{\text{pr}}; r)$.	– $\text{eh}_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, \text{h}_{\text{pr}}; r)$. – $\text{Del.Verify}(\text{del.crs}, h, b, \text{del}.\pi) = 1$, where $h = (\text{h}_{\text{pb}}, \text{h}_{\text{pr}}, \text{h}_{\Sigma})$.

Parameters. In the following construction we use $n_{\text{enc}} = |\text{eh}_{\text{pr}}| = |\text{PKE.Enc}(\text{pke.pk}, 1^\lambda)| = \text{poly}(\lambda)$, $n'_{\text{enc}} = |x_{\text{pr}}| + |\text{h}_{\text{pr}}| + |r| = n_{\text{pr}} + \text{poly}(\lambda)$, $n_{\text{edel}} = |\text{h}_{\text{pb}}| + |\text{eh}_{\text{pr}}| + |\text{h}_{\Sigma}| = \text{poly}(\lambda)$, and $n'_{\text{edel}} = |\text{h}_{\text{pr}}| + |r| + |\text{del}.\pi| = \text{poly}(\lambda)$.

Construction 4.1. [Encrypted RAM Delegation] Let $\text{PKE} = (\text{PKE.Gen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a public-key encryption scheme, $\text{NIZK} = (\text{NIZK.Setup}, \text{NIZK.Prove}, \text{NIZK.Verify})$ be a non-interactive zero-knowledge scheme, and $\text{Del} = (\text{Del.Setup}, \text{Del.Digest}, \text{Del.Prove}, \text{Del.Verify})$ be a split memory RAM delegation scheme associated with a hash tree HT with hash key ht.hk . We construct our encrypted RAM delegation scheme $\mathcal{EDel} = (\text{Setup}, \text{Prove}, \text{Verify})$ as follows:

Setup $(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \rightarrow \text{crs}$. Let $\mathbf{s} = (n_{\text{pb}}, n_{\text{pr}}, S)$ be a partitioning of the memory. Sample a split memory RAM delegation scheme $\text{del.crs} \leftarrow \text{Del.Setup}(1^\lambda, T, \mathbf{s})$, a PKE scheme $(\text{pke.pk}, \text{pke.sk}) \leftarrow \text{PKE.Gen}(1^\lambda)$, a NIZK scheme for $\mathcal{L}_{\text{enc}}$ as $(\text{zk.crs}_{\text{enc}}, \text{zk.td}_{\text{enc}}) \leftarrow \text{NIZK.Setup}(1^\lambda, 1^{n_{\text{enc}}}, 1^{n'_{\text{enc}}})$, and finally another NIZK scheme for $\mathcal{L}_{\text{edel}}$ as $(\text{zk.crs}_{\text{edel}}, \text{zk.td}_{\text{edel}}) \leftarrow \text{NIZK.Setup}(1^\lambda, 1^{n_{\text{edel}}}, 1^{n'_{\text{edel}}})$. Let $\text{crs} = (\text{del.crs}, \text{pke.pk}, \text{zk.crs}_{\text{enc}}, \text{zk.crs}_{\text{edel}}, S)$.

Prove $(\text{crs}, x_{\text{pb}}, x_{\text{pr}}) \rightarrow (b, \pi)$. This poly-time algorithm does the following:

1. Parse $\text{crs} = (\text{del.crs}, \text{pke.pk}, \text{zk.crs}_{\text{enc}}, \text{zk.crs}_{\text{edel}}, S)$.

2. Compute a hash digest of the private input $h_{pr} \leftarrow \text{HT.Hash}(\text{ht.hk}, x_{pr})$, sample a randomness r , and compute an encryption of h_{pr} as $eh_{pr} \leftarrow \text{PKE.Enc}(\text{pke.pk}, h_{pr}; r)$.
3. Compute a NIZK proof for validity of eh_{pr} as $zk.\pi_{enc} \leftarrow \text{NIZK.Prove}(zk.crs_{enc}, eh_{pr}, (x_{pr}, h_{pr}, r))$.
4. Let $cf_0 = (x_{pb}, x_{pr}, 0^S)$ be the initial memory of the RAM machine and compute its partial digests $h_{pb} \leftarrow \text{HT.Hash}(\text{ht.hk}, x_{pb})$ and $h_\Sigma \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S)$.
5. Compute a split memory RAM delegation proof $(b, \text{del}.\pi) \leftarrow \text{Del.Prove}(\text{del.crs}, cf_0)$.
6. Compute a NIZK proof $zk.\pi_{edel} \leftarrow \text{NIZK.Prove}(zk.crs_{edel}, (h_{pb}, eh_{pr}, h_\Sigma, b), (h_{pr}, r, \text{del}.\pi))$.
7. Output $(b, \pi = (eh_{pr}, zk.\pi_{enc}, zk.\pi_{edel}))$.

$\text{Verify}(\text{crs}, h_{pb}, b, \pi) \rightarrow 0/1$. Parse crs as above and compute $h_\Sigma \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S)$. Output 1 if $\text{NIZK.Verify}(zk.crs_{enc}, eh_{pr}, zk.\pi_{enc}) = 1$ and $\text{NIZK.Verify}(zk.crs_{edel}, (h_{pb}, eh_{pr}, h_\Sigma, b), zk.\pi_{edel}) = 1$; otherwise output 0.

Theorem 4.2. Assuming split memory RAM delegations, PKEs, and NIZKs, the Construction 4.1 is an encrypted RAM delegation (Definition 3.1) with strong soundness, straight-line extractor, and input hiding.

Completeness. The completeness follows by the construction and the completeness of the underlying Del and NIZK and the correctness of the underlying PKE.

Efficiency. The crs consists of $(\text{del.crs}, \text{pke.pk}, zk.crs_{enc}, zk.crs_{edel}, S)$, and the followings hold (1) Del setup time and crs size are bounded by $\text{poly}(\lambda, \log T, \log n_{pb}, \log n_{pr})$, (2) PKE key generation time and pk size are bounded by $\text{poly}(\lambda)$, (3) NIZK setup time and crs size are bounded by (a) $\text{poly}(\lambda, n_{pr})$ for $\mathcal{L}_{enc}$, and (b) $\text{poly}(\lambda, \log T, \log n_{pb}, \log n_{pr})$ for $\mathcal{L}_{edel}$. Thus the crs size and setup time of our construction is bounded by $\text{poly}(\lambda, \log T, \log n_{pb}, \log n_{pr}) + \text{poly}(\lambda, n_{pr})$.

The h_{pb} (resp. h_{pr}) size is λ (resp. λ), and the generation and verification time is $\text{poly}(\lambda, n_{pb})$ (resp. $\text{poly}(\lambda, n_{pr})$).

Given h_{pr} , the eh_{pr} size, generation time and verification time are all $\text{poly}(\lambda)$

The $zk.\pi_{enc}$ size, its generation and verification time all depend on the NIZK for $\mathcal{L}_{enc}$, which depends on eh_{pr} and h_{pr} , hence are bounded by $\text{poly}(\lambda, n_{pr})$.

The $zk.\pi_{edel}$ size, its generation and verification time all depend on the NIZK for $\mathcal{L}_{edel}$, which is in return is dependent on the Del scheme. Note that the $\text{del}.\pi$ size and verification time are $\text{poly}(\lambda, \log T, \log n_{pb}, \log n_{pr})$, and generation time is $\text{poly}(\lambda, T, n_{pb}, n_{pr})$. Hence the same holds in our construction.

Hence, in our construction, the proof size and the verification time are $\text{poly}(\lambda, \log T, \log n_{pb}, \log n_{pr}) + \text{poly}(\lambda, n_{pr})$ and the generation time is $\text{poly}(\lambda, T, n_{pb}, n_{pr})$

Note that since we have $T \leq 2^\lambda$ and $n_{pb} + n_{pr} \leq T$, we can get rid of all the log terms in our analysis.

4.1.1 Security Analysis.

Here we prove the straight-line extractor and input hiding properties of our construction.

Proof of Straight-Line Extractor. Let the extractor $\mathcal{E}$ do the following – (1) to generate (crs, td) proceed the same as normal setup except that use $\text{NIZK}.\mathcal{E}_1$ to sample $(zk.crs_{enc}, zk.td_{enc})$ and

$(\text{zk.crs}_{\text{edel}}, \text{zk.td}_{\text{edel}})$, then construct crs similar to setup and let $\text{td} = (\text{pke.sk}, \text{zk.td}_{\text{enc}}, \text{zk.td}_{\text{edel}})$, (2) upon receiving $\pi = (\text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}, \text{zk.}\pi_{\text{edel}})$ extract $(x_{\text{pr}}^*, h_{\text{pr},1}^*, r_1^*) = \text{NIZK.E}_1(\text{zk.td}_{\text{enc}}, \text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}})$ and output x_{pr}^* .

First note that the crs indistinguishability is directly implied by the crs indistinguishability of the underlying NIZK schemes.

Now define the experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ as follows – the adversary $\mathcal{A}$ on input 1^λ sends $(n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ to the challenger $\mathcal{C}$, $\mathcal{C}$ runs $(\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ and sends crs to $\mathcal{A}$, then $\mathcal{A}$ outputs (x_{pb}, b, π) . Parse $\pi = (\text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}, \text{zk.}\pi_{\text{edel}})$, and let $h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}})$, and $h_\Sigma \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S)$. Then extract the values $(x_{\text{pr}}^*, h_{\text{pr},1}^*, r_1^*) = \text{NIZK.E}_1(\text{zk.td}_{\text{enc}}, \text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}})$, and $(h_{\text{pr},2}^*, r_2^*, \text{del.}\pi^*) = \text{NIZK.E}_2(\text{zk.td}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), \text{zk.}\pi_{\text{edel}})$. Namely:

$$\text{Exp}_{\mathcal{E}\text{Del}} = \left[\begin{array}{l} (n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T), \\ (x_{\text{pb}}, b, \pi) \leftarrow \mathcal{A}(\text{crs}), \\ h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}}), \\ h_\Sigma \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S), \\ \text{Let } \pi = (\text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}, \text{zk.}\pi_{\text{edel}}), \\ (x_{\text{pr}}^*, h_{\text{pr},1}^*, r_1^*) \leftarrow \text{NIZK.E}_1(\text{zk.td}_{\text{enc}}, \text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}), \\ (h_{\text{pr},2}^*, r_2^*, \text{del.}\pi^*) = \text{NIZK.E}_2(\text{zk.td}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), \text{zk.}\pi_{\text{edel}}) \end{array} \right]$$

For an adversary $\mathcal{A}$ we define the output of the above experiment, $\text{Exp}_{\mathcal{E}\text{Del}}(\mathcal{A})$, to be 1 if it holds that $\text{NIZK.Verify}(\text{zk.crs}_{\text{enc}}, \text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}) = 1$ and $\text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), \text{zk.}\pi_{\text{edel}}) = 1$ and $(x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T}$. Namely $\mathcal{A}$ wins if:

$$\left[\begin{array}{l} (x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T} \wedge \\ \text{NIZK.Verify}(\text{zk.crs}_{\text{enc}}, \text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}) = 1 \wedge \\ \text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), \text{zk.}\pi_{\text{edel}}) = 1 \end{array} : \text{Exp}_{\mathcal{E}\text{Del}} \right]$$

Let $(x_{\text{enc}}, w_{\text{enc}}) = (\text{eh}_{\text{pr}}, (x_{\text{pr}}^*, h_{\text{pr},1}^*, r_1^*))$ and $(x_{\text{edel}}, w_{\text{edel}}) = ((h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), (h_{\text{pr},2}^*, r_2^*, \text{del.}\pi^*))$. Define the following variables:

$$\epsilon_1 = \Pr[\text{Exp}_{\mathcal{E}\text{Del}}(\mathcal{A}) = 1] - \Pr \left[\begin{array}{l} (x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T} \wedge (x_{\text{enc}}, w_{\text{enc}}) \in \mathcal{L}_{\text{enc}} \wedge \\ \text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, x_{\text{edel}}, \text{zk.}\pi_{\text{edel}}) = 1 \end{array} : \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (3)$$

$$\epsilon_2 = \Pr \left[\begin{array}{l} (x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T} \wedge (x_{\text{enc}}, w_{\text{enc}}) \in \mathcal{L}_{\text{enc}} \wedge \\ \text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, x_{\text{edel}}, \text{zk.}\pi_{\text{edel}}) = 1 \end{array} : \text{Exp}_{\mathcal{E}\text{Del}} \right] - \\ \Pr \left[(x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T} \wedge (x_{\text{enc}}, w_{\text{enc}}) \in \mathcal{L}_{\text{enc}} \wedge (x_{\text{edel}}, w_{\text{edel}}) \in \mathcal{L}_{\text{edel}} : \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (4)$$

$$\epsilon_3 = \Pr \left[(x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T} \wedge (x_{\text{enc}}, w_{\text{enc}}) \in \mathcal{L}_{\text{enc}} \wedge (x_{\text{edel}}, w_{\text{edel}}) \in \mathcal{L}_{\text{edel}} : \text{Exp}_{\mathcal{E}\text{Del}} \right] - \\ \Pr \left[\begin{array}{l} (x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T} \wedge h_{\text{pr},2}^* = \text{HT.Hash}(\text{ht.hk}, x_{\text{pr}}^*) \\ \text{Del.Verify}(\text{del.crs}, (h_{\text{pb}}, h_{\text{pr},2}^*, h_\Sigma), b, \text{del.}\pi) = 1 \end{array} : \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (5)$$

$$\epsilon_4 = \Pr \left[\begin{array}{l} (x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T} \wedge h_{\text{pr},2}^* = \text{HT.Hash}(\text{ht.hk}, x_{\text{pr}}^*) \\ \text{Del.Verify}(\text{del.crs}, (h_{\text{pb}}, h_{\text{pr},2}^*, h_\Sigma), b, \text{del.}\pi) = 1 \end{array} : \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (6)$$

Now since $\Pr[\text{Exp}_{\mathcal{E}\text{Del}}(\mathcal{A}) = 1] = \epsilon_1 + \epsilon_2 + \epsilon_3 + \epsilon_4$ we only need to show that for $i \in [4]$, ϵ_i is negligible.

Case 1. Suppose towards the contradiction that ϵ_1 is non-negligible, then by Eq. (3) it holds that:

$$\Pr[\text{NIZK.Verify}(\text{zk.crs}_{\text{enc}}, \text{eh}_{\text{pr}}, \text{zk}.\pi_{\text{enc}}) = 1 \wedge (x_{\text{enc}}, w_{\text{enc}}) \notin \mathcal{L}_{\text{enc}}] \geq \epsilon_1$$

Now we construct adversary $\mathcal{B}_{\text{zk}}^{\mathcal{E}}$ that breaks the straight-line extractor property of the underlying NIZK scheme as follows:

1. Receive $(n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from $\mathcal{A}$.
2. Receive $\text{zk.crs}_{\text{enc}}^*$ from $\mathcal{C}$.
3. Construct crs similar to experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ where $\text{zk.crs}_{\text{enc}} = \text{zk.crs}_{\text{enc}}^*$, and send it to $\mathcal{A}$.
4. Receive (x_{pb}, b, π) from $\mathcal{A}$.
5. Parse $\pi = (\text{eh}_{\text{pr}}, \text{zk}.\pi_{\text{enc}}, \text{zk}.\pi_{\text{edel}})$.
6. Send $(\text{eh}_{\text{pr}}, \text{zk}.\pi_{\text{enc}})$ to $\mathcal{C}$.

Note that $\mathcal{B}_{\text{zk}}^{\mathcal{E}}$ perfectly simulates the experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ for $\mathcal{A}$, thus has the same advantage ϵ_1 in breaking the straight-line extractor property of the underlying NIZK scheme.

Case 2. Suppose towards the contradiction that ϵ_2 is non-negligible, then by Eq. (4) it holds that:

$$\Pr[\text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, x_{\text{edel}}, \text{zk}.\pi_{\text{edel}}) = 1 \wedge (x_{\text{edel}}, w_{\text{edel}}) \notin \mathcal{L}_{\text{edel}}] \geq \epsilon_2$$

Now we construct adversary $\mathcal{B}_{\text{zk}}^{\mathcal{E}}$ that breaks the straight-line extractor property of the underlying NIZK scheme as follows:

1. Receive $(n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from $\mathcal{A}$.
2. Receive $\text{zk.crs}_{\text{edel}}^*$ from $\mathcal{C}$.
3. Construct crs similar to experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ where $\text{zk.crs}_{\text{edel}} = \text{zk.crs}_{\text{edel}}^*$, and send it to $\mathcal{A}$.
4. Receive (x_{pb}, b, π) from $\mathcal{A}$.
5. Parse $\pi = (\text{eh}_{\text{pr}}, \text{zk}.\pi_{\text{enc}}, \text{zk}.\pi_{\text{edel}})$.
6. Compute $h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}})$ and $h_{\Sigma} \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S)$.
7. Send $((h_{\text{pb}}, \text{eh}_{\text{pr}}, h_{\Sigma}, b), \text{zk}.\pi_{\text{edel}})$ to $\mathcal{C}$.

Note that $\mathcal{B}_{\text{zk}}^{\mathcal{E}}$ perfectly simulates the experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ for $\mathcal{A}$, thus has the same advantage ϵ_2 in breaking the straight-line extractor property of the underlying NIZK scheme.

Case 3. Suppose towards the contradiction that ϵ_1 is non-negligible. First note that $(x_{\text{enc}}, w_{\text{enc}}) \in \mathcal{L}_{\text{enc}}$ is equivalent with:

$$\text{eh}_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr},1}^*; r_1^*) \quad \wedge \quad h_{\text{pr},1}^* = \text{HT.Hash}(\text{ht.hk}, x_{\text{pr}}^*)$$

and $(x_{\text{edel}}, w_{\text{edel}}) \in \mathcal{L}_{\text{edel}}$ is equivalent with:

$$\text{eh}_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr},2}^*; r_2^*) \quad \wedge \quad \text{Del.Verify}(\text{del.crs}, (h_{\text{pb}}, h_{\text{pr},2}^*, h_{\Sigma}), b, \text{del.}\pi) = 1$$

then by Eq. (5) it holds that

$$\Pr \left[\begin{array}{l} \text{eh}_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr},1}^*; r_1^*) \wedge h_{\text{pr},1}^* = \text{HT.Hash}(\text{ht.hk}, x_{\text{pr}}^*) \wedge \\ \text{eh}_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr},2}^*; r_2^*) \wedge h_{\text{pr},2}^* \neq \text{HT.Hash}(\text{ht.hk}, x_{\text{pr}}^*) \end{array} : \text{Exp}_{\mathcal{E}_{\text{Del}}} \right] \geq \epsilon_3$$

Note that this implies that $h_{\text{pr},1}^* \neq h_{\text{pr},2}^*$, however, $\text{PKE.Enc}(\text{pke.pk}, h_{\text{pr},1}^*; r_1^*) = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr},2}^*; r_2^*)$. This breaks the perfect correctness of the underlying PKE scheme.

Case 4. Suppose towards the contradiction that ϵ_4 is non-negligible, then we construct adversary $\mathcal{B}_{\text{del}}^{\text{sound}}$ that breaks the soundness property of the underlying Del scheme as follows:

1. Receive $(n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from $\mathcal{A}$.
2. Send $(1^\lambda, T)$ to $\mathcal{C}$ and receive del.crs^* from $\mathcal{C}$.
3. Construct crs similar to experiment $\text{Exp}_{\mathcal{E}_{\text{Del}}}$ where $\text{del.crs} = \text{del.crs}^*$, and send it to $\mathcal{A}$.
4. Receive (x_{pb}, b, π) from $\mathcal{A}$.
5. Parse $\pi = (\text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}, \text{zk.}\pi_{\text{edel}})$.
6. Compute $h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}})$ and $h_{\Sigma} \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S)$.
7. Compute $(h_{\text{pr},2}^*, r_2^*, \text{del.}\pi^*) = \text{NIZK.}\mathcal{E}_2(\text{zk.td}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_{\Sigma}, b), \text{zk.}\pi_{\text{edel}})$ and $(x_{\text{pr}}^*, h_{\text{pr},1}^*, r_1^*) \leftarrow \text{NIZK.}\mathcal{E}_1(\text{zk.td}_{\text{enc}}, \text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}})$.
8. Send $(\text{cf}_0 = (x_{\text{pb}}, x_{\text{pr}}^*, 0^S), b, \text{del.}\pi^*)$ to $\mathcal{C}$.

Note that $(h_{\text{pb}}, h_{\text{pr},2}^*, h_{\Sigma}) = \text{Del.Digest}(\text{cf}_0)$, $(x_{\text{pb}}, x_{\text{pr}}^*) \notin \mathcal{L}_{\mathcal{M},T}$ implies that $\text{cf}_0 \notin \mathcal{L}_{\mathcal{M},T}$, and $\mathcal{B}_{\text{del}}^{\text{sound}}$ perfectly simulates the experiment $\text{Exp}_{\mathcal{E}_{\text{Del}}}$ for $\mathcal{A}$. Thus $\mathcal{B}_{\text{del}}^{\text{sound}}$ has the same advantage ϵ_4 in breaking the soundness property of the underlying Del scheme. $\square$

Proof of Input Hiding. We will define the simulator as follows – (1) sample the crs similar to the Setup except that generate $\text{zk.crs}_{\text{enc}}$ and $\text{zk.crs}_{\text{edel}}$ using $\text{zk.crs}_{\text{enc}} \leftarrow \text{NIZK.}\mathcal{S}_1(1^\lambda, 1^{n_{\text{enc}}})$ and $\text{zk.crs}_{\text{edel}} \leftarrow \text{NIZK.}\mathcal{S}_2(1^\lambda, 1^{n_{\text{edel}}})$, and (2) generate the proof by computing $\text{eh}_{\text{pr}} \leftarrow \text{PKE.Enc}(\text{pke.pk}, 0^{|h_{\text{pr}}|})$, $\text{zk.}\pi_{\text{enc}} \leftarrow \text{NIZK.}\mathcal{S}_1(\text{eh}_{\text{pr}})$, $h_{\text{pb}} \leftarrow \text{HT.Hash}(\text{ht.hk}, x_{\text{pb}})$, $h_{\Sigma} \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S)$, and $\text{zk.}\pi_{\text{edel}} \leftarrow \text{NIZK.}\mathcal{S}_2(h_{\text{pb}}, \text{eh}_{\text{pr}}, h_{\Sigma}, b)$ and letting $\pi = (\text{eh}_{\text{pr}}, \text{zk.}\pi_{\text{enc}}, \text{zk.}\pi_{\text{edel}})$.

We will define the following hybrids:

hyb₀. This is the real experiment where the $\mathcal{A}$ receives $(\text{crs}, \text{td}) \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$, then makes queries $(x_{\text{pb}}, x_{\text{pr}})$ and receives $(b, \pi) \leftarrow \text{Prove}(\text{crs}, x_{\text{pb}}, x_{\text{pr}})$.

hyb₁. Similar to hyb₀ except that $\text{zk.crs}_{\text{enc}} \leftarrow \text{NIZK.S}_1(1^\lambda, 1^{n_{\text{enc}}})$ in the crs, and in the query responses, $\text{zk}.\pi_{\text{enc}}$ is generated using $\text{zk}.\pi_{\text{enc}} \leftarrow \text{NIZK.S}_1(\text{eh}_{\text{pr}})$.

hyb₂. Similar to hyb₁ except that $\text{zk.crs}_{\text{edel}} \leftarrow \text{NIZK.S}_2(1^\lambda, 1^{n_{\text{edel}}})$ in the crs, and in the query responses, $\text{zk}.\pi_{\text{edel}}$ is generated using $\text{zk}.\pi_{\text{edel}} \leftarrow \text{NIZK.S}_2(h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b)$.

hyb₃. Similar to hyb₂ except that $\text{eh}_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, 0^{|\text{h}_{\text{pr}}|})$. Note that this is the simulated experiment.

Let $\mathcal{A}$ output a bit $b \in \{0, 1\}$ at the end of each experiment and denote this output by $\text{hyb}_i(\mathcal{A})$.

Lemma 4.3. If NIZK is zero-knowledge, then for any adversary $\mathcal{A}$ it holds that

$$|\Pr[\text{hyb}_0(\mathcal{A}) = 1] - \Pr[\text{hyb}_1(\mathcal{A}) = 1]| \leq \text{negl}(\lambda).$$

Proof. Suppose towards the contradiction that for some adversary $\mathcal{A}$ and some non-negligible function $\epsilon(\cdot)$ it holds that $|\Pr[\text{hyb}_0(\mathcal{A}) = 1] - \Pr[\text{hyb}_1(\mathcal{A}) = 1]| \geq \epsilon(\lambda)$. We construct an adversary $\mathcal{B}_{\text{zk}}$ against the zero-knowledge property of the underlying NIZK scheme as follows:

1. Receive $(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from the adversary.
2. Receive $\text{zk.crs}_{\text{enc}} \leftarrow \text{NIZK.Setup}(1^\lambda, 1^{n_{\text{enc}}})$ from $\mathcal{C}$.
3. Construct the rest of the crs similar to the Setup.
4. On any query $(x_{\text{pb}}, x_{\text{pr}})$ generate the proof similar to $\text{Prove}(\text{crs}, x_{\text{pb}}, x_{\text{pr}})$ except that to generate $\text{zk}.\pi_{\text{enc}}$, make a query to $\mathcal{C}$ on $(\text{eh}_{\text{pr}}, (x_{\text{pr}}, h_{\text{pr}}, r))$.
5. Send π to $\mathcal{A}$ and output whatever $\mathcal{A}$ outputs.

Note that if the challenger uses NIZK.Setup and NIZK.Prove (resp. $\mathcal{S}_1$) in the experiment then $\mathcal{B}_{\text{zk}}$ perfectly simulates hyb₀ (resp. hyb₁). Thus $\mathcal{B}_{\text{zk}}$ has the same advantage $\epsilon(\lambda)$ as $\mathcal{A}$ in breaking the zero-knowledge property of the NIZK scheme. $\square$

Lemma 4.4. If NIZK is zero-knowledge, then for any adversary $\mathcal{A}$ it holds that

$$|\Pr[\text{hyb}_1(\mathcal{A}) = 1] - \Pr[\text{hyb}_2(\mathcal{A}) = 1]| \leq \text{negl}(\lambda).$$

Proof. Suppose towards the contradiction that for some adversary $\mathcal{A}$ and some non-negligible function $\epsilon(\cdot)$ it holds that $|\Pr[\text{hyb}_1(\mathcal{A}) = 1] - \Pr[\text{hyb}_2(\mathcal{A}) = 1]| \geq \epsilon(\lambda)$. We construct an adversary $\mathcal{B}_{\text{zk}}$ against the zero-knowledge property of the underlying NIZK scheme as follows:

1. Receive $(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from the adversary.
2. Receive $\text{zk.crs}_{\text{edel}} \leftarrow \text{NIZK.Setup}(1^\lambda, 1^{n_{\text{edel}}})$ from $\mathcal{C}$.
3. Construct the rest of the crs similar to hyb₁.
4. On any query $(x_{\text{pb}}, x_{\text{pr}})$ generate the proof similar to hyb₁ except that to generate $\text{zk}.\pi_{\text{edel}}$, make a query to $\mathcal{C}$ on $((h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), (x_{\text{pr}}, h_{\text{pr}}, r))$.
5. Send π to $\mathcal{A}$ and output whatever $\mathcal{A}$ outputs.

Note that if the challenger uses NIZK.Setup and NIZK.Prove (resp. $\mathcal{S}_2$) in the experiment then $\mathcal{B}_{zk}$ perfectly simulates hyb_1 (resp. hyb_2). Thus $\mathcal{B}_{zk}$ has the same advantage $\epsilon(\lambda)$ as $\mathcal{A}$ in breaking the zero-knowledge property of the NIZK scheme. $\square$

Lemma 4.5. If PKE is secure, then for any adversary $\mathcal{A}$ it holds that

$$|\Pr[\text{hyb}_2(\mathcal{A}) = 1] - \Pr[\text{hyb}_3(\mathcal{A}) = 1]| \leq \text{negl}(\lambda).$$

Proof. Suppose towards the contradiction that for some adversary $\mathcal{A}$ and some non-negligible function $\epsilon(\cdot)$ it holds that $|\Pr[\text{hyb}_2(\mathcal{A}) = 1] - \Pr[\text{hyb}_3(\mathcal{A}) = 1]| \geq \epsilon(\lambda)$. We construct an adversary $\mathcal{B}_{pke}$ against the security of the underlying PKE scheme as follows:

1. Receive $(1^\lambda, n_{pb}, 1^{n_{pr}}, S, T)$ from the adversary.
2. Receive $\text{pke.pk} \leftarrow \text{PKE.Enc}(1^\lambda)$ from $\mathcal{C}$.
3. Construct the rest of the crs similar to hyb_1 .
4. On any query (x_{pb}, x_{pr}) compute $h_{pr} \leftarrow \text{HT.Hash}(\text{ht.hk}, x_{pr})$ send a query to $\mathcal{C}$ on h_{pr} and receive $e_{h_{pr}}$. Generate the rest of the proof similar to hyb_2 .
5. Send π to $\mathcal{A}$ and output whatever $\mathcal{A}$ outputs.

Note that if the challenger uses $\text{PKE.Enc}(\text{pke.pk}, \cdot)$ (resp. $\text{PKE.Enc}(\text{pke.pk}, 0^{|h_{pr}|})$) in the experiment then $\mathcal{B}_{pke}$ perfectly simulates hyb_2 (resp. hyb_3). Thus $\mathcal{B}_{pke}$ has the same advantage $\epsilon(\lambda)$ as $\mathcal{A}$ in breaking the security of the PKE scheme. $\square$

We conclude the input hiding proof by Lemmas 4.3 to 4.5 and a hybrid argument. $\square$

Corollary 4.6. Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists an encrypted RAM delegation with straight-line extractor, and input hiding. Additionally, if we plug-in our optimally efficient NIZK from Section 5.2, the crs size and the setup running-time will only grow with $\log n_{pr}$.

Remark 4.7. If we use NIZKs with statistical zero-knowledge property and instead of PKEs we use statistically-hiding commitments, then our construction achieves statistical input-hiding property.

4.2 Reusable Encrypted RAM Delegation

We observe that our construction of encrypted RAM delegation is already reusable. In the following we show how to partition the algorithms in Construction 4.1 to match the syntax of reusable encrypted RAM delegation.

Construction 4.8. [Reusable Encrypted RAM Delegation] Let $\text{PKE} = (\text{PKE.Gen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a public-key encryption scheme, $\text{Del} = (\text{Del.Setup}, \text{Del.Digest}, \text{Del.Prove}, \text{Del.Verify})$ be a split memory RAM delegation scheme associated with a hash tree HT, and $\text{NIZK} = (\text{NIZK.Setup}, \text{NIZK.Prove}, \text{NIZK.Verify})$ be a non-interactive zero-knowledge scheme. We construct an encrypted RAM delegation $\mathcal{EDel} = (\text{Setup}, \text{Prove}, \text{Verify})$ as follows:

$\text{Setup}(1^\lambda, n_{pb}, 1^{n_{pr}}, S, T) \rightarrow (\text{crs})$. Same as in Construction 4.1.

$\text{PrivEnc}(\text{crs}, x_{\text{pr}}) \rightarrow (\text{eh}_{\text{pr}}, \pi_{\text{enc}}, \text{aux})$. Perform Items 1 to 3 from the prover's algorithm in Construction 4.1 and output $(\text{eh}_{\text{pr}}, \pi_{\text{enc}} = \text{zk}.\pi_{\text{enc}}, \text{aux} = (x_{\text{pr}}, r))$.

$\text{Prove}(\text{crs}, x_{\text{pb}}, \text{aux}) \rightarrow (b, \pi_{\text{edel}})$. Same as in Construction 4.1 except that it takes $\text{aux} = (x_{\text{pr}}, r)$ as input and doesn't compute $\text{zk}.\pi_{\text{enc}}$.

$\text{VerifyEnc}(\text{crs}, \text{eh}_{\text{pr}}, \pi_{\text{enc}}) \rightarrow 0/1$. Parse $\text{crs} = (\text{del}.\text{crs}, \text{pke}.\text{pk}, \text{zk}.\text{crs}_{\text{enc}}, \text{zk}.\text{crs}_{\text{edel}}, S)$. Output 1 if NIZK. $\text{Verify}(\text{zk}.\text{crs}_{\text{enc}}, \text{eh}_{\text{pr}}, \pi_{\text{enc}}) = 1$; else output 0.

$\text{Verify}(\text{crs}, \text{h}_{\text{pb}}, \text{eh}_{\text{pr}}, b, \pi_{\text{edel}}) \rightarrow 0/1$. Parse crs as above. Let $\text{h}_{\Sigma} \leftarrow \text{HT}.\text{Hash}(\text{ht}.\text{hk}, 0^S)$. Output 1 if $\text{NIZK}.\text{Verify}(\text{zk}.\text{crs}_{\text{edel}}, (\text{h}_{\text{pb}}, \text{eh}_{\text{pr}}, \text{h}_{\Sigma}, b), \pi_{\text{edel}}) = 1$; else output 0.

Theorem 4.9. Assuming split memory RAM delegations, PKEs, and NIZKs, the Construction 4.8 is a reusable encrypted RAM delegation (Definition 3.2) with strong soundness, straight-line extractor, and input hiding.

Completeness. The completeness follows by the construction and the completeness of the underlying Del and NIZK and the correctness of the underlying PKE.

Efficiency. The proof is similar to the proof as in the encrypted RAM delegation. Refer to Section 4.1 for details.

4.2.1 Security Analysis.

Proof of Straight-Line Extractor. The proof is similar to the proof as in the encrypted RAM delegation. Refer to Section 4.1.1 for details. $\square$

Proof of Input Hiding. The proof is similar to the proof as in the encrypted RAM delegation. Refer to Section 4.1.1 for details. $\square$

Proof of Strong Soundness. First define the following hybrids:

hyb_0 . This is the real experiment where the adversary $\mathcal{A}$ on input 1^λ sends $(n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ to the challenger $\mathcal{C}$. Then $\mathcal{C}$ runs $\text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ and sends crs to $\mathcal{A}$. Upon receiving crs , $\mathcal{A}$ outputs $(x_{\text{pb}}, \text{eh}_{\text{pr}}, \pi_{\text{edel}}^{(0)}, \pi_{\text{edel}}^{(1)})$. $\mathcal{A}$ wins if for $b \in \{0, 1\}$, $\text{Verify}(\text{zk}.\text{crs}_{\text{edel}}, \text{h}_{\text{pb}}, \text{eh}_{\text{pr}}, b, \pi_{\text{edel}}^{(b)}) = 1$.

hyb_1 . Similar to hyb_0 except that $\mathcal{C}$ generates $\text{zk}.\text{crs}_{\text{edel}}$ in the crs by running $\text{NIZK}.\mathcal{E}$ and appends $\text{zk}.\text{td}_{\text{edel}}$ to td .

The indistinguishability of the hyb_0 and hyb_1 directly follows from the crs indistinguishability of the underlying NIZK scheme.

To conclude the proof we only need to show that the winning probability of $\mathcal{A}$ in hyb_1 is negligible. Let Setup' be the setup as defined in hyb_1 . Define the experiment $\text{Exp}_{\mathcal{E}^{\text{Del}}}$ be the same as in hyb_1 except that $\mathcal{C}$ additionally computes h_{Σ} and extracts the witness from NIZK proofs as follows:

$$\text{Exp}_{\mathcal{E}\text{Del}} = \left[\begin{array}{l} (n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{crs}, \text{td}) \leftarrow \text{Setup}'(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T), \\ (h_{\text{pb}}, \text{eh}_{\text{pr}}, \pi_{\text{edel}}^{(0)}, \pi_{\text{edel}}^{(1)}) \leftarrow \mathcal{A}(\text{crs}), \\ h_\Sigma \leftarrow \text{HT.Hash}(\text{ht.hk}, 0^S), \\ (h_{\text{pr}}^{(b)}, r^{(b)}, \text{del}.\pi^{(b)}) = \text{NIZK}.\mathcal{E}(\text{zk.td}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), \pi_{\text{edel}}^{(b)}) \text{ for } b \in \{0, 1\} \end{array} \right]$$

For an adversary $\mathcal{A}$ we define the output of the above experiment, $\text{Exp}_{\mathcal{E}\text{Del}}(\mathcal{A})$, to be 1, if $\mathcal{A}$ wins (similar to hyb_2), namely:

$$\left[\text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), \pi_{\text{edel}}^{(b)}) = 1 \text{ for } b \in \{0, 1\} \quad : \quad \text{Exp}_{\mathcal{E}\text{Del}} \right]$$

Let $(x_{\text{edel}}^{(b)}, w_{\text{edel}}^{(b)}) = ((h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, b), (h_{\text{pr}}^{(b)}, r^{(b)}, \text{del}.\pi^{(b)}))$. Define the following variables:

$$\epsilon_1 = \Pr[\text{Exp}_{\mathcal{E}\text{Del}}(\mathcal{A}) = 1] - \Pr \left[\begin{array}{l} (x_{\text{edel}}^{(0)}, w_{\text{edel}}^{(0)}) \in \mathcal{L}_{\text{edel}} \\ \text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, 1), \pi_{\text{edel}}^{(1)}) = 1 \end{array} \quad : \quad \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (7)$$

$$\epsilon_2 = \Pr \left[\begin{array}{l} (x_{\text{edel}}^{(0)}, w_{\text{edel}}^{(0)}) \in \mathcal{L}_{\text{edel}} \\ \text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, 1), \pi_{\text{edel}}^{(1)}) = 1 \end{array} \quad : \quad \text{Exp}_{\mathcal{E}\text{Del}} \right] - \Pr \left[\begin{array}{l} (x_{\text{edel}}^{(b)}, w_{\text{edel}}^{(b)}) \in \mathcal{L}_{\text{edel}} \text{ for } b \in \{0, 1\} \end{array} \quad : \quad \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (8)$$

$$\epsilon_3 = \Pr \left[\begin{array}{l} (x_{\text{edel}}^{(b)}, w_{\text{edel}}^{(b)}) \in \mathcal{L}_{\text{edel}} \text{ for } b \in \{0, 1\} \end{array} \quad : \quad \text{Exp}_{\mathcal{E}\text{Del}} \right] - \Pr \left[\begin{array}{l} h_{\text{pr}}^{(0)} = h_{\text{pr}}^{(1)} \\ \text{Del.Verify}(\text{del.crs}, (h_{\text{pb}}, h_{\text{pr}}^{(b)}, h_\Sigma), b, \text{del}.\pi^{(b)}) = 1 \text{ for } b \in \{0, 1\} \end{array} \quad : \quad \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (9)$$

$$\epsilon_4 = \Pr \left[\begin{array}{l} h_{\text{pr}}^{(0)} = h_{\text{pr}}^{(1)} \\ \text{Del.Verify}(\text{del.crs}, (h_{\text{pb}}, h_{\text{pr}}^{(b)}, h_\Sigma), b, \text{del}.\pi^{(b)}) = 1 \text{ for } b \in \{0, 1\} \end{array} \quad : \quad \text{Exp}_{\mathcal{E}\text{Del}} \right]. \quad (10)$$

Now since $\Pr[\text{Exp}_{\mathcal{E}\text{Del}}(\mathcal{A}) = 1] = \epsilon_1 + \epsilon_2 + \epsilon_3 + \epsilon_4$ we only need to show that for $i \in [4]$, ϵ_i is negligible.

Case 1. Suppose towards the contradiction that ϵ_1 is non-negligible, then by Eq. (7) it holds that:

$$\Pr[\text{NIZK.Verify}(\text{zk.crs}_{\text{edel}}, (h_{\text{pb}}, \text{eh}_{\text{pr}}, h_\Sigma, 0), \pi_{\text{edel}}^{(0)}) = 1 \wedge (x_{\text{edel}}^{(0)}, w_{\text{edel}}^{(0)}) \notin \mathcal{L}_{\text{edel}}] \geq \epsilon_1$$

Now we construct adversary $\mathcal{B}_{\text{zk}}^\mathcal{E}$ that breaks the straight-line extractor property of the underlying NIZK scheme as follows:

1. Receive $(n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from $\mathcal{A}$.
2. Receive $\text{zk.crs}_{\text{edel}}^*$ from $\mathcal{C}$.
3. Construct crs similar to experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ where $\text{zk.crs}_{\text{edel}} = \text{zk.crs}_{\text{edel}}^*$, and send it to $\mathcal{A}$.
4. Receive $(h_{\text{pb}}, eh_{\text{pr}}, \pi_{\text{edel}}^{(0)}, \pi_{\text{edel}}^{(1)})$ from $\mathcal{A}$.
5. Compute $h_{\Sigma} = \text{HT.Hash}(\text{ht.hk}, 0^S)$.
6. Send $((h_{\text{pb}}, eh_{\text{pr}}, h_{\Sigma}, 0), \pi_{\text{edel}}^{(0)})$ to $\mathcal{C}$.

Note that $\mathcal{B}_{\text{zk}}^{\mathcal{E}}$ perfectly simulates the experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ for $\mathcal{A}$, thus has the same advantage ϵ_1 in breaking the straight-line extractor property of the underlying NIZK scheme.

Case 2. Similar to case 1.

Case 3. Suppose towards the contradiction that ϵ_3 is non-negligible. First note that $(x_{\text{edel}}^{(b)}, w_{\text{edel}}^{(b)}) \in \mathcal{L}_{\text{edel}}$ is equivalent with:

$$eh_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr}}^{(b)}; r^{(b)}) \quad \wedge \quad \text{Del.Verify}(\text{del.crs}, (h_{\text{pb}}, h_{\text{pr}}^{(b)}, h_{\Sigma}), b, \text{del}.\pi^{(b)}) = 1$$

then by Eq. (9) it holds that

$$\Pr \left[\begin{array}{l} eh_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr}}^{(0)}; r^{(0)}) \wedge \\ eh_{\text{pr}} = \text{PKE.Enc}(\text{pke.pk}, h_{\text{pr}}^{(1)}; r^{(1)}) \wedge h_{\text{pr}}^{(0)} \neq h_{\text{pr}}^{(1)} \end{array} : \text{Exp}_{\mathcal{E}\text{Del}} \right] \geq \epsilon_3$$

This breaks the perfect correctness of the underlying PKE scheme.

Case 4. Suppose towards the contradiction that ϵ_4 is non-negligible, then we construct adversary $\mathcal{B}_{\text{del}}^{\text{sound}}$ that breaks the strong soundness property of the underlying Del scheme as follows:

1. Receive $(n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T)$ from $\mathcal{A}$.
2. Send $(1^\lambda, T)$ to $\mathcal{C}$ and receive del.crs^* from $\mathcal{C}$.
3. Construct crs similar to experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ where $\text{del.crs} = \text{del.crs}^*$, and send it to $\mathcal{A}$.
4. Receive $(h_{\text{pb}}, eh_{\text{pr}}, \pi_{\text{edel}}^{(0)}, \pi_{\text{edel}}^{(1)})$ from $\mathcal{A}$.
5. Compute $(h_{\text{pr}}^{(b)}, r^{(b)}, \text{del}.\pi^{(b)}) = \text{NIZK}.\mathcal{E}(\text{zk.td}_{\text{edel}}, (h_{\text{pb}}, eh_{\text{pr}}, h_{\Sigma}, b), \pi_{\text{edel}}^{(b)})$ for $b \in \{0, 1\}$.
6. Compute $h_{\Sigma} = \text{HT.Hash}(\text{ht.hk}, 0^S)$.
7. Send $((h_{\text{pb}}, h_{\text{pr}}^{(0)}, h_{\Sigma}), \pi_{\text{edel}}^{(0)}, \pi_{\text{edel}}^{(1)})$ to $\mathcal{C}$.

Note that $\mathcal{B}_{\text{del}}^{\text{sound}}$ perfectly simulates the experiment $\text{Exp}_{\mathcal{E}\text{Del}}$ for $\mathcal{A}$. Thus $\mathcal{B}_{\text{del}}^{\text{sound}}$ has the same advantage ϵ_4 in breaking the strong soundness property of the underlying Del scheme. $\square$

Corollary 4.10. Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists a reusable encrypted RAM delegation with strong soundness, straight-line extractor, and input hiding. Additionally, by using our optimally efficient NIZK from Section 5.2, the crs size and setup's running-time will only grow with $\log n_{\text{pr}}$.

5 Homomorphic Non-Interactive Zero-Knowledge

We begin by recalling the definition of homomorphic NIZKs as introduced in [ADKL19]. Next, we describe some useful tools that we rely on to construct homomorphic NIZKs. The main tool that we rely on are optimally-efficient NIZKs. We provide a simple construction for optimally-efficient NIZKs using encrypted RAM delegation scheme in Section 5.2, and later use them to design fully-homomorphic NIZKs.

5.1 Definitions: Homomorphic NIZKs

We start by defining some useful notation for circuits that we will rely on throughout this section.

Notation: circuits as graphs. We interpret each circuit C as a directed acyclic graph. We say a directed acyclic graph $G = (V, E)$ represents a boolean circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$ if G is defined as follows. In graph G , each vertex $v \in V$ of in-degree 0 is an input node labeled as $i \in [k]$, there are k such vertices. For each remaining vertex of G , it is labeled as a logic gate (NAND gate for simplicity). Each internal wire of the circuit corresponds to a directed edge in G , where the direction of such edge indicates the direction of signal. The node v_{out} which has an out-degree of 0 represents the output gate.

Merge nodes in graphs. Given graph $G = (V, E)$, the merge of vertices $u, v \in V$ is defined as follows: First, for each $w \in V \setminus \{u, v\}$, replace each edge (w, v) (resp. (v, w)) with a new edge (w, u) (resp. (u, w)) if there exists such edges. Next, remove node v , and edges $(u, v), (v, u)$ if there exists such edges.

Split nodes. Given $G = (V, E)$, the split of a vertex $u \in V$ is as follows: The first step is adding a new vertex u' . Next for each vertex v such that $(u, v) \in E$, remove such edge, and add an additional edge (u', v) . Intuitively, the node u is “split” into two disconnected nodes u, u' .

Cutable node. “Cutable node” is considered as a duality term of an edge cut. Given $G = (V, E)$, then vertex $u \in V$ is a cutable node if and only if splitting vertex u in G results in an additional maximally connected component³ in the original graph.⁴ Note that in our work, a node with in-degree or out-degree 0 will not be considered as a cutable node.

Defining homomorphic NIZKs. For some boolean circuit family $\mathcal{C}$ that takes n -bits of input, consider the following NP language:

$$\mathcal{L} = \{(C, b) \in \mathcal{C} \times \{0, 1\} : \exists \omega \in \{0, 1\}^n \text{ s.t. } C(\omega) = b\} \quad (11)$$

A NIZK proof system (Setup, Prove, Verify) for any such language $\mathcal{L}$ is a fully-homomorphic NIZK system if there also exists a polynomial algorithm, Eval, with the following syntax:

Eval(crs, $C, \{(C_j, b_j, \pi_j)\}_{j \in [k]} \rightarrow (C', b, \pi)$. The evaluation algorithm takes as input the CRS crs, k instances $\{(C_i, b_i)\}_{i=1}^k$, their proofs $\{\pi_i\}_{i=1}^k$, and a circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$. It outputs the composed instance (C', b) where $C' = \text{Compose}(C, \{(C_i, b_i)\}_{i=1}^k)$ and a proof π . The Compose procedure is described below.

Compose($C, C_1, \dots, C_k$) $\rightarrow C'$. Let graph G represent the circuit C and graph G_i represent the circuit C_i for all $i \in [k]$. It proceeds as follows:

³The term maximally connected component refers to the connected subgraph to which no vertex can be added and it remains connected.

⁴When we say connected component, we abuse the notation and consider the graph as an undirected one.

1. Define a new graph $G' = (V', E')$ where $V' = (\bigcup_{i=1}^k V_i) \cup V$ and $E' = (\bigcup_{i=1}^k E_i) \cup E$.
2. For all $i \in [k]$, let $u_i \in V_i$ be the output gate of C_i , and $v_i \in V$ be the i -th input gate of C . Next for all $i \in [k]$, it merges every pair of u_i, v_i in graph G' and labels the merged vertex as a NAND gate. It outputs C' which corresponds to graph G' as the resulting circuit.

In addition to the regular completeness, soundness, and zero-knowledge properties for standard NIZKs (Definition 2.9), homomorphic NIZKs must satisfy two additional properties called completeness of Eval and unlinkability defined as follows:

Completeness of Eval. For any PPT adversary $\mathcal{A}$ and for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \exists i \in [k] \text{ s.t. } \text{Verify}(\text{crs}, (C_i, b_i), \pi_i) = 0 \\ \vee \left(\begin{array}{l} \text{Verify}(\text{crs}, (C', b), \pi) = 1 \\ \wedge C' = \text{Compose}(C, C_1, \dots, C_k) \\ \wedge b = C(b_1, \dots, b_k) \end{array} \right) \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (C, \{(C_j, b_j, \pi_j)\}_{j \in [k]}) \leftarrow \mathcal{A}(\text{crs}) \\ (C', b, \pi) \leftarrow \text{Eval}(\text{crs}, C, \{(C_j, b_j, \pi_j)\}_{j \in [k]}) \end{array} \right] = 1.$$

Unlinkability. This property states that in any homomorphic NIZK scheme a proof obtained by Eval to be indistinguishable from a freshly generated proof. Formally, for any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, the following equation holds:

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), \beta \leftarrow \{0, 1\} \\ (C, \{(C_i, b_i, \omega_i, \pi_i)\}_{i \in [k]}) \leftarrow \mathcal{A}(\text{crs}) \\ (C_0^*, b_0^*, \pi_0^*) \leftarrow \text{Eval}(\text{crs}, C, \{(C_i, b_i, \pi_i)\}_{i \in [k]}) \\ C_1^* = \text{Compose}(C, C_1, \dots, C_k) \\ b_1^* = C_1^*(\omega_1, \dots, \omega_k) \\ \pi_1^* \leftarrow \text{Prove}(\text{crs}, (C_1^*, b_1^*), (\omega_1, \dots, \omega_k)) \end{array} : \mathcal{A}(C_\beta^*, b_\beta^*, \pi_\beta^*) = \beta \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where $\mathcal{A}$ is admissible if $\text{Verify}(\text{crs}, (C_i, b_i), \pi_i) = 1$ and $C_i(\omega_i) = b_i$ for all $i \in [k]$.

5.2 Tools Needed: Optimally Efficient NIZK and Rate-1 Message Encodings

In this section, we construct an optimally efficient NIZK using our encrypted RAM delegation scheme and rate-1 message encodings.

Construction 5.1. Let $\text{r1Enc} = (\text{Encode}, \text{Decode})$ be a secure rate-1 message encoding scheme (Definition 2.14) and $\mathcal{EDel} = (\text{Setup}, \text{Prove}, \text{Verify})$ be an encrypted RAM delegation scheme w.r.to HT (Definition 3.1) for machine $\mathcal{M}'$ defined as follows – $\mathcal{M}'$ has $(\lambda, \mathcal{M})$ hardwired, takes (x, ρ) as public input, and sk as private input, computes $w = \text{r1Enc.Decode}(1^\lambda, \rho, \text{sk})$ and output 1 if and only if $\mathcal{M}(x, w) = 1$. We construct an optimally efficient NIZK for the language $\mathcal{L}_{\mathcal{M}, T} = \{(x, w) : \mathcal{M}(x, w, T) = 1\}$ as follows:

Setup $(1^\lambda, T) \rightarrow \text{crs}$. This probabilistic algorithm samples $\mathcal{EDel}$ scheme's crs , $\text{del.crs} \leftarrow \mathcal{EDel.Setup}(1^\lambda, 1^\lambda, T)$. Output $\text{crs} = (\lambda, \text{del.crs})$.

$\text{Prove}(\text{crs}, x, w) \rightarrow \pi$. This probabilistic algorithm parses crs as $(\lambda, \text{del.crs})$ and does the following.

1. Let $(\rho, \text{sk}) \leftarrow \text{r1Enc.Encode}(1^\lambda, w)$. Set $x_{\text{pb}} := (x, \rho)$ and $x_{\text{pr}} := \text{sk}$.
2. Sample $\mathcal{EDel}$ proof $\text{del.}\pi \leftarrow \mathcal{EDel.Prove}(\text{del.crs}, x_{\text{pb}}, x_{\text{pr}})$ and output the proof $\pi = (\rho, \text{del.}\pi)$.

$\text{Verify}(\text{crs}, x, \pi) \rightarrow 0/1$. This poly-time algorithm parses crs as $(\lambda, \text{del.crs})$, π as $(\rho, \text{del.}\pi)$, computes $h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, (x, \rho))$, and outputs $\mathcal{EDel.Verify}(\text{del.crs}, h_{\text{pb}}, 1, \text{del.}\pi)$.

Theorem 5.2. If $\mathcal{EDel}$ is a secure encrypted RAM delegation scheme w.r.to hash tree HT for language $\mathcal{L}_{\mathcal{M}'}$ (Definition 3.1) and r1Enc is a secure rate-1 message encoding scheme (Definition 2.14), then Construction 5.1 is an optimally efficient NIZK scheme for language $\mathcal{L}_{\mathcal{M}}$.

Remark 5.3. If we rely on a multi-theorem NIZK argument and plug-in the optimally efficient NIZK from Construction 5.1 in the reusable $\mathcal{EDel}$ scheme (Definition 3.2), we get an optimal rate $\text{r}\mathcal{EDel}$ scheme.

Proof. We show that Construction 5.1 is correct and efficient as follows:

Correctness. We have that $\text{crs} = (\lambda, \text{del.crs})$ and $\pi = (\rho, \mathcal{EDel.Prove}(\text{del.crs}, (x, \rho), \text{sk}))$ where $(\rho, \text{sk}) \leftarrow \text{r1Enc.Encode}(1^\lambda, w)$. By correctness of $\mathcal{EDel}, \text{r1Enc}$ for machine $\mathcal{M}'$, if $\mathcal{M}(x, w) = 1$, then $\mathcal{M}'((x, \rho), \text{sk}) = 1$ and the verification algorithm outputs 1.

Efficiency. Note that the crs of the NIZK scheme is nothing but the crs for $\mathcal{EDel}$. Hence, $|\text{crs}| = |\text{del.crs}| = \text{poly}(\lambda, \log T)$. Similarly, running time of **Setup** follows from running time of $\mathcal{EDel.Setup}$'s running time. In **Prove**, we are using $\mathcal{EDel.Prove}$ whose running time is $\text{poly}(\lambda, T, (|x| + |w|))$ and r1Enc whose running time $\text{poly}(\lambda, |w|)$. It follows that the running time of **Prove** is $\text{poly}(\lambda, T, |x|, |w|)$. The size of the proof is $|\rho| + |\text{del.}\pi| = |w| + \text{poly}(\lambda, \log T, \log(|x| + |w|))$. The **Verify** algorithm computes using HT which takes $O((|x| + |w|)\lambda)$ time and then runs $\mathcal{EDel.Verify}$. Hence the running time of **Verify** follows from the running time $\mathcal{EDel.Verify}$ and HT.Hash . In summary, Construction 5.1 has the following efficiency properties:

- $|\text{crs}| = \text{poly}(\lambda, \log T)$.
- $|\pi| = |w| + \text{poly}(\lambda, \log T, \log(|x| + |w|))$.
- Running time of **Setup** is $\text{poly}(\lambda, \log T)$.
- Running time of **Prove** is $\text{poly}(\lambda, T, |x|, |w|)$.
- Running time of **Verify** is $O((|x| + |w|)\lambda) + \text{poly}(\lambda, \log T, \log(|x| + |w|))$.

Moreover, we remark that the running time of verifier can be reduced by considering a pre-processing phase where x, ρ (parsed from $\text{del.}\pi$) are hashed independent of **Verify**. This renders the running time of **Verify** to be virtually independent of $|x|$ and $|w|$. In addition, due to the poly-logarithmic dependence $|\text{crs}|$ and **Setup** algorithm's running time T , we can parse λ many crs with exponentially increasing T , i.e., $T = 1, \dots, T = 2^\lambda$ and use the crs that appropriately bounds the running time of $\mathcal{A}$. This is the popular powers-of-2 technique [GKP⁺13] which is used in various contexts to reduce/remove the dependency of certain parameters in the analysis of algorithm which depend poly-logarithmically in the length of said parameters. We can leverage this technique to remove the dependence of T and we follow this convention for the rest of the paper. $\square$

5.2.1 Security Analysis

Here we show that Construction 5.1 satisfies straight-line extraction and computational zero-knowledge.

Proof of straight-line extraction. We show that if $\mathcal{EDel}$ satisfies straight-line extraction and $\mathbf{r1Enc}$ is correct, then Construction 5.1 satisfies straight-line extraction. Let the extractor do the following – (1) generate (crs, td) using $\mathcal{EDel}.\mathcal{E}$. Set $\text{crs} = \text{del.crs}$ and $\text{td} = \text{del.td}$, (2) to extract w^* upon receiving $\pi = (\rho, \text{del}.\pi)$, compute $\text{sk}^* \leftarrow \mathcal{EDel}.\mathcal{E}(\text{td}, \text{del}.\pi)$ and output $w^* = \mathbf{r1Enc}.\text{Decode}(1^\lambda, \rho, \text{sk}^*)$.

Note that the crs indistinguishability is implied by the crs indistinguishability of $\mathcal{EDel}$. We define the experiment Exp_{NIZK} as follows –

$$\text{Exp}_{\text{NIZK}} = \left[\begin{array}{l} T \leftarrow \mathcal{A}(1^\lambda), (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, T), (x, \pi) \leftarrow \mathcal{A}(\text{crs}), \\ \text{Let } \pi = (\rho, \text{del}.\pi), h_{\text{pb}} = \text{HT.Hash}(\text{ht.hk}, (x, \rho)), \\ \text{sk}^* \leftarrow \mathcal{EDel}.\mathcal{E}(\text{td}, \pi), w^* \leftarrow \mathbf{r1Enc}.\text{Decode}(1^\lambda, \rho, \text{sk}^*) \end{array} \right]$$

We say that $\mathcal{A}$ wins if

$$\Pr \left[\begin{array}{l} (x, w^*) \notin \mathcal{L}_{\mathcal{M}} \wedge \\ \mathcal{EDel}.\text{Verify}(\text{del.crs}, h_{\text{pb}}, \text{del}.\pi) = 1 \end{array} : \text{Exp}_{\text{NIZK}} \right] \geq \epsilon(\lambda)$$

for some non-negligible function $\epsilon(\cdot)$. Then we define the following advantages of the adversary $\mathcal{A}$,

$$\text{Adv}_1 = |\Pr[\mathcal{A} \text{ wins}] - \Pr[(x, w^*) \notin \mathcal{L}_{\mathcal{M}} \wedge ((x, \rho), \text{sk}^*) \in \mathcal{L}_{\mathcal{M}'} : \text{Exp}_{\text{NIZK}}]| \quad (12)$$

$$\begin{aligned} \text{Adv}_2 = & |\Pr[(x, w^*) \notin \mathcal{L}_{\mathcal{M}} \wedge ((x, \rho), \text{sk}^*) \in \mathcal{L}_{\mathcal{M}'} : \text{Exp}_{\text{NIZK}}] - \\ & \Pr[(x, w^*) \notin \mathcal{L}_{\mathcal{M}} \wedge w^* = \mathbf{r1Enc}.\text{Decode}(1^\lambda, \rho, \text{sk}^*) : \text{Exp}_{\text{NIZK}}]| \end{aligned} \quad (13)$$

If $\mathcal{A}$ wins in Exp_{NIZK} , then either $\mathcal{A}$ breaks the straight-line extraction property of $\mathcal{EDel}$ (Type 1, $\text{Adv}_1 \geq \epsilon/2$) or it breaks the correctness of $\mathbf{r1Enc}$ (Type 2, $\text{Adv}_2 \geq \epsilon/2$).

Type 1. Suppose Eq. (12) is correct, then it holds that

$$\Pr[((x, \rho), \text{sk}^*) \notin \mathcal{L}_{\mathcal{M}'} \wedge \mathcal{EDel}.\text{Verify}(\text{del.crs}, h_{\text{pb}}, \text{del}.\pi) = 1] \geq \epsilon/2$$

Now we construct an adversary $\mathcal{B}_{\mathcal{EDel}, \mathcal{E}}$ that breaks the straight-line extraction property of $\mathcal{EDel}$ for machine $\mathcal{M}'$ as follows:

1. Receive T from $\mathcal{A}$ and send it to $\mathcal{C}$.
2. Receive crs from $\mathcal{C}$ and pass it on to $\mathcal{A}$.
3. Receive (x, π) from $\mathcal{A}$. Parse π as $(\rho, \text{del}.\pi)$. Send $((x, \rho), \text{del}.\pi)$ to $\mathcal{C}$.

Note that $\mathcal{B}_{\mathcal{EDel}, \mathcal{E}}$ perfectly simulates Exp_{NIZK} for $\mathcal{A}$ thus having advantage $\epsilon/2$ in breaking the straight-line extraction for $\mathcal{EDel}$.

Type 2. Suppose Eq. (13) is correct, note that $((x, \rho), \text{sk}^*) \in \mathcal{L}_{\mathcal{M}'}$ implies that $w^* = \text{r1Enc.Decode}(1^\lambda, \rho, \text{sk}^*)$. However, by Eq. (13), $\mathcal{A}$ can find $w^* \neq \text{r1Enc.Decode}(1^\lambda, \rho, \text{sk}^*)$ with probability $\epsilon/2$, which breaks the perfect correctness of r1Enc . $\square$

Proof of zero-knowledge. We show that if $\mathcal{E}\text{Del}$ is input-hiding and r1Enc is secure, then Construction 5.1 satisfies computational zero-knowledge. We define the following hybrids:

$\text{hyb}_0^{\mathcal{A}}$ This is the original experiment. Namely, in this experiment, the adversary $\mathcal{A}$ sends T to the challenger $\mathcal{C}$. $\mathcal{C}$ runs $\text{crs} \leftarrow \text{Setup}(1^\lambda, T)$ and sends crs to $\mathcal{A}$. $\mathcal{A}$ queries $\mathcal{C}$ with (x, w) and $\mathcal{C}$ responds with $\text{Prove}(\text{crs}, x, w)$ if $\mathcal{M}(x, w) = 1$.

$\text{hyb}_1^{\mathcal{A}}$ This is same as $\text{hyb}_0^{\mathcal{A}}$ except that we use $\mathcal{E}\text{Del.S}$ to simulate $\mathcal{E}\text{Del}$ instantiation.

$\text{hyb}_2^{\mathcal{A}}$ This is same as $\text{hyb}_1^{\mathcal{A}}$ except we sample random $\rho \leftarrow \{0, 1\}^m$ instead of r1Enc.Encode . This is the description of the simulator $\mathcal{S}$.

Let $\text{hyb}_i^{\mathcal{A}}(1^\lambda)$ denote the output of the experiment $\text{hyb}_i^{\mathcal{A}}$.

Claim 5.4. If $\mathcal{E}\text{Del}$ is an input-hiding encrypted RAM delegation scheme, then for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \text{hyb}_0^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{hyb}_1^{\mathcal{A}}(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

Proof. Assuming that there exists an adversary $\mathcal{A}$ which can distinguish between $\text{hyb}_0^{\mathcal{A}}$ and $\text{hyb}_1^{\mathcal{A}}$ such that $|\Pr [1 \leftarrow \text{hyb}_0^{\mathcal{A}}(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_1^{\mathcal{A}}(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{\text{del}}$ against the input-hiding property of $\mathcal{E}\text{Del}$ as follows:

1. $\mathcal{A}$ sends T to $\mathcal{B}_{\text{del}}$. $\mathcal{B}_{\text{del}}$ sends T to $\mathcal{C}$. $\mathcal{B}_{\text{del}}$ passes the crs received from the challenger, $\mathcal{C}$, to $\mathcal{A}$.
2. $\mathcal{A}$ queries (x, w) to $\mathcal{B}_{\text{del}}$. If $\mathcal{M}(x, w) = 1$, query $\mathcal{C}$ with $((x, \rho), \text{sk})$ where $(\rho, \text{sk}) \leftarrow \text{r1Enc.Encode}(1^\lambda, w)$. Otherwise, respond with $\perp$.
3. Send $(\rho, \text{del}.\pi)$ to $\mathcal{A}$ where $\text{del}.\pi$ is send by $\mathcal{C}$. Output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ uses $\mathcal{E}\text{Del.Setup}$ and $\mathcal{E}\text{Del.Prove}$ (resp. $\mathcal{E}\text{Del.S}$) in the experiment then $\mathcal{B}_{\text{del}}$ perfectly simulates $\text{hyb}_0^{\mathcal{A}}$ (resp. $\text{hyb}_1^{\mathcal{A}}$). Thus $\mathcal{B}_{\text{del}}$ has advantage $\epsilon(\lambda)$ in breaking the input-hiding property of $\mathcal{E}\text{Del}$. $\square$

Claim 5.5. If r1Enc is a secure rate-1 message encoding scheme, then for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \text{hyb}_1^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{hyb}_2^{\mathcal{A}}(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

Proof. Assuming that there exists an adversary $\mathcal{A}$ which can distinguish between $\text{hyb}_1^{\mathcal{A}}$ and $\text{hyb}_2^{\mathcal{A}}$ such that $|\Pr [1 \leftarrow \text{hyb}_1^{\mathcal{A}}(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_2^{\mathcal{A}}(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{\text{r1Enc}}$ that breaks the security of r1Enc as follows:

1. $\mathcal{A}$ sends T to $\mathcal{B}_{\text{r1Enc}}$. $\mathcal{B}_{\text{r1Enc}}$ samples $\text{del.crs} \leftarrow \mathcal{E}\text{Del.S}(1^\lambda, 1^\lambda, T)$ and sends del.crs to $\mathcal{A}$.

2. $\mathcal{A}$ queries (x, w) . If $\mathcal{M}(x, w) = 1$, query the challenger, $\mathcal{C}$, with w to receive ρ and send $(\rho, \text{del}.\pi)$ where $\text{del}.\pi \leftarrow \mathcal{E}\text{Del}.\mathcal{S}(\text{del}.\text{crs}, x, \rho)$ to $\mathcal{A}$. Otherwise, output $\perp$.
3. Output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ uses r1Enc.Encode (resp. using random ρ) in the experiment then $\mathcal{B}_{\text{r1Enc}}$ perfectly simulates $\text{hyb}_1^{\mathcal{A}}$ (resp. $\text{hyb}_2^{\mathcal{A}}$). Thus $\mathcal{B}_{\text{r1Enc}}$ has advantage $\epsilon(\lambda)$ in breaking the security of r1Enc . $\square$

$\square$

We state the following corollary of Theorems 5.2, 4.2 and Remarks 2.12, 2.10.

Corollary 5.6. Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists an optimally efficient non-interactive zero-knowledge argument scheme with straight-line extraction for all NP languages.

5.3 Boolean Circuit Decomposition

In the next section, we will provide our construction for fully-homomorphic NIZKs. But first, we define a new circuit decomposition algorithm **Decompose**, which plays an important role of achieving the unlinkability property. At a high level, by splitting each cuttable vertex of G (which corresponds to circuit C), $\text{Decompose}(C) \rightarrow (T, \{C_v\}_{v \in V_T})$ parses circuit C into smallest chunks of circuits $\{C_v\}_{v \in V_T}$ such that their combination forms a tree structure $T = (V_T, E_T)$. Fig. 2 is an example of the circuit decomposition algorithm.

$\text{Decompose}(C) \rightarrow (T, \{C_v\}_{v \in V_T})$.

1. Let directed acyclic graph $G = (V, E)$ represent the boolean circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$.
2. For each cuttable node $u \in V$ of graph G , the **Decompose** algorithm simply splits node u as u, u' , and labels u as an output gate and u' as an input node. After splitting all of the cuttable nodes, it obtains a new graph G' .
3. Next, each connected component of graph G' can be represented as a boolean circuit. Consider each of these boolean circuits as a vertex. Then these vertices will naturally form a tree $T = (V_T, E_T)$, where for each $u, v \in V_T$, u is the parent of v if and only if the output gate of circuit C_v and the input gate of circuit C_u are splits of a single vertex in the previous step.
4. The **Decompose** algorithm outputs $T, \{C_v\}_{v \in V_T}$.

Theorem 5.7. Given any set of boolean circuits $\{C, C_1, \dots, C_\ell\}$ where $C = \{0, 1\}^\ell \rightarrow \{0, 1\}$, let $(T_0, \{C_v\}_{v \in V_{T_0}})$ be the decomposition by $\text{Decompose}(C)$, and for each $i \in [\ell]$, let $(T_i, \{C_v\}_{v \in V_{T_i}})$ be $\text{Decompose}(C_i)$. Define C' as $\text{Compose}(C, (C_1, \dots, C_\ell))$, and let $(T', \{C_v\}_{v \in V_{T'}})$ be $\text{Decompose}(C')$. Then,

$$\bigcup_{i=0}^{\ell} \{C_v\}_{v \in V_{T_i}} = \{C_v\}_{v \in V_{T'}},$$

Proof. We note that on composing $C, \{C_i\}_{i \in [\ell]}$, one is merging the output gate of C_i with the input gate of circuit C and make it a single node in the composed circuit C' . Such node must be a cuttable node, since the node is the only node connecting circuit C and C_i . Thus the decomposer splits all such nodes in C' , and our theorem directly follows. $\square$

Language $\mathcal{L}$

Instance: $\text{nizk.crs}, C, b$

Witness: $\{(b_i, \pi_i)\}_{i \in [k]}$.

Output: To check validity of the instance-witness pair, proceed as follows:

- It runs $\text{Decompose}(C)$ and obtains $(T, \{C_v\}_{v \in V_T})$.
- Let Rt be the root node of T and check if C_{Rt} takes k bits of input. If the check fails, it outputs 0 and aborts.
- Let $v_1, \dots, v_k$ be the k child nodes of Rt in T . For each $i \in [k]$, let circuit C_i be the circuit defined by the subtree with root v_i (note that we are combining all the circuits within that subtree).
- For each $i \in [k]$, if C_i is not empty, it checks if $\text{NIZK.Verify}(\text{nizk.crs}, (\text{nizk.crs}, C_i, b_i), \pi_i) = 1$. If any of the checks fails, it outputs 0 and aborts.
- It outputs 1 if and only if $C_{\text{Rt}}(b_1, \dots, b_k) = b$.

Figure 3: Description of language $\mathcal{L}$ for optimally efficient NIZK

would be generated by the underlying optimally efficient NIZK scheme from a universal language. The unlinkability property immediately holds by the zero-knowledge property of optimally efficient NIZK. Our construction is formally described below.

Construction 5.8. Let $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ be an optimally efficient NIZK scheme (Definition 2.13) for language $\mathcal{L}$ defined in Fig. 3. We construct a homomorphic NIZK scheme for language from Eq. (11) as follows:

$\text{Setup}(1^\lambda) \rightarrow \text{crs}$. The Setup algorithm generates crs with respect to language $\mathcal{L}$ as $\text{nizk.crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$ and outputs $\text{crs} = \text{nizk.crs}$.

$\text{Prove}(\text{crs}, (C, b), \omega) \rightarrow \pi$. Parse crs as nizk.crs and witness ω as $(b_1, \dots, b_\ell) \in \{0, 1\}^\ell \forall i \in [\ell]$, the prover's algorithm sets C_i as an empty circuit, π_i as an empty string. It runs $(C', b', \pi') \leftarrow \text{Eval}(\text{nizk.crs}, C, \{(C_i, b_i, \pi_i)\}_{i \in [\ell]})$ and outputs proof π as π' .

$\text{Eval}(\text{crs}, C, \{(C_i, b_i, \pi_i)\}_{i \in [k]}) \rightarrow (C', b, \pi)$. The algorithm Eval parses crs as nizk.crs and does the following:

1. Set $(T, \{C_v\}_{v \in V_T})$ to be $\text{Decompose}(C)$.
2. For i -th leaf node $v \in V_T$, set $C'_v = C_i$, b_v as b_i , π_v as π_i .
3. Sort all the internal nodes in T in a topological order (ending with the root) and iteratively do the following on all levels starting from the leaves:
 - If $v \in V_T$ has an in-degree ℓ , let $(v_1, \dots, v_\ell)$ be the child nodes of v , i.e., C_v takes $(b_{v_1}, \dots, b_{v_\ell})$ as input. Then compute

$$b_v = C_v(b_{v_1}, \dots, b_{v_\ell}), C'_v = \text{Compose}(C_v, (C'_{v_1}, \dots, C'_{v_\ell})),$$

and generate $\pi_v \leftarrow \text{NIZK.Prove}(\text{nizk.crs}, (\text{nizk.crs}, C'_v, b_v), \{(b_{v_i}, \pi_{v_i})\}_{i \in \ell})$.

4. Let node Rt be the root of T . Output $C' = C'_{\text{Rt}}$, $b = b_{\text{Rt}}$, and $\pi = \pi_{\text{Rt}}$.

$\text{Verify}(\text{crs}, (C, b), \pi) \rightarrow 0/1$. Parse crs as nizk.crs and output $\text{NIZK.Verify}(\text{nizk.crs}, (\text{nizk.crs}, C, b), \pi)$.

Theorem 5.9. If NIZK is a optimally efficient NIZK scheme (Definition 2.13) for language $\mathcal{L}$ (Fig. 3), then Construction 5.8 is a homomorphic NIZK scheme for the NP language defined in Eq. (11).

Completeness. The design described above implements a recursive zero-knowledge proof over the tree of the decomposed circuit. The functionality of the decomposed circuit is equivalent to the original circuit, and the completeness of the above design is implied by the completeness of the underlying NIZK.

Efficiency. The succinctness of the above design follows from the optimal efficiency property of the underlying NIZK which is specified by the efficiency of optimally efficient NIZK.

Lemma 5.10. Assume that NIZK satisfies optimal efficiency property, then for every $\lambda \in \mathbb{N}$, the following holds:

- For $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, $|\text{crs}| \leq \text{poly}(\lambda)$.
- For boolean circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$, the running time of $\text{Prove}(\text{crs}, C, b, \{b_i\}_{i \in [k]})$ is at most $\text{poly}(\lambda, |C|)$.
- For proof $\pi \leftarrow \text{Prove}(\text{crs}, C, b, \{b_i\}_{i \in [k]})$, $|\pi| \leq \text{poly}(\lambda, |C|)$.

Proof. By optimal efficiency of NIZK, the crs size is at most $\text{poly}(\lambda)$ since the running time of the above language $\mathcal{L}$ is no more than 2^λ . Next we consider the running time of Prove , observe that language $\mathcal{L}$ contains a NIZK.Verify step. Again by efficiency of optimally efficient NIZK, the NIZK verifier running time is bounded by $\text{poly}(\lambda, |x|, |\omega|)$ for some $\text{poly}(\cdot, \cdot, \cdot)$. Thus the size of language $\mathcal{L}$ is at most $\text{poly}(\lambda, |C|)$. By the optimal efficiency of NIZK, there is a $\text{poly}(\lambda)$ -bit overhead of the zero knowledge proof. Otherwise, the proof size is exactly equivalent to the witness size. We observe that the maximum number of NIZK proofs applied is fewer than the number of nodes in the $\text{Decompose}(C)$ tree. Thus, the overall overhead size is bounded by $\text{poly}(\lambda, |C|)$. Since the overall witness length $k \leq |C|$, we conclude that $|\pi| \leq \text{poly}(\lambda, |C|)$. $\square$

5.5 Security Analysis

Lemma 5.11 (Argument of knowledge). Assume that NIZK satisfies straight-line extraction, then Construction 5.8 satisfies argument of knowledge.

Proof. We will show how to construct an extractor $\mathcal{E}$ for our scheme. Before doing that, we start by defining a recursive extractor $\mathcal{E}'$ for our scheme.

$\mathcal{E}'(1^\lambda) \rightarrow (\text{crs}, \text{td})$: The extractor first sets up NIZK CRS and trapdoor with respect to language $\mathcal{L}$:

$$(\text{nizk.crs}, \text{nizk.td}) \leftarrow \text{NIZK}.\mathcal{E}(1^\lambda).$$

Next, it outputs crs as nizk.crs and td as $(\text{nizk.crs}, \text{nizk.td})$.

$\mathcal{E}'(\text{td}, (C, b), \pi) \rightarrow (T, \{(C'_v, b_v, \omega_v)\}_{v \in V_T})$. Below is a step-by-step explanation:

1. It first computes $(T, \{C_v\}_{v \in V_T}) = \text{Decompose}(C)$. Next for each leaf node v , it sets $C'_v = C_v$. For each non-leaf node v with child nodes $(v_1, \dots, v_\ell)$, it sets $C'_v = \text{Compose}(C_v, (C'_{v_1}, \dots, C'_{v_\ell}))$. It sets C'_v for each $v \in V_T$ in a topological order.
2. Let Rt be the root node of T , then $C'_{\text{Rt}} = C$ by definition of step 1. It sets π_{Rt} as π and b_{Rt} as b . It parses td as $(\text{nizk.crs}, \text{nizk.td})$, and we note that crs is set as nizk.crs in our scheme.
3. Then for each internal node $v \in V_T$, consider that it has been assigned π_v , b_v and that C_v takes ℓ bits of input. The extractor runs $\text{NIZK.E}(\text{nizk.td}, (\text{nizk.crs}, C'_v, b_v), \pi_v)$ and obtains $\{(b_i, \pi_i)\}_{i \in [\ell]}$. It sets ω_v as $(b_1, \dots, b_\ell)$. Since C'_v takes ℓ bits of input, v also has a number of ℓ child nodes $(v_1, \dots, v_\ell)$. It sets b_{v_i} as b_i and π_{v_i} as π_i for all $i \in [\ell]$.
4. It runs step 3 for each non-leaf node $v \in V_T$ in a reverse topological order. For each leaf node v , it sets ω_v as an empty string. It outputs T and $\{(C'_v, b_v, \omega_v)\}_{v \in V_T}$.

Claim 5.12. Assuming that NIZK satisfies adaptive argument of knowledge, then there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, the following holds for all non-leaf node $v \in V_T$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, (C, b), \pi) = 1 \\ \wedge \omega_v \text{ is not a valid witness for } (\text{crs}, C'_v, b_v) \in \mathcal{L} \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}'(1^\lambda) \\ (C, b, \pi) \leftarrow \mathcal{A}(\text{crs}, \text{td}) \\ (T, \{(C'_v, b_v, \omega_v)\}_{v \in V_T}) \leftarrow \mathcal{E}'(\text{td}, (C, b), \pi) \end{array} \right] \leq \text{negl}(\lambda),$$

where in the above equation $T = (V_T, E_T)$.

Proof. We construct a proof by induction over each $v \in V_T$, in a reverse topological order.

Base Case ($v = \text{Rt}$): We first show that the lemma holds for $v = \text{Rt}$. Assume that $\text{Verify}(\text{crs}, (C, b), \pi) = 1$ and ω_{Rt} is not a valid witness for $(\text{crs}, C'_{\text{Rt}}, b_{\text{Rt}}) \in \mathcal{L}$, where $\omega_{\text{Rt}} \leftarrow \text{NIZK.E}(\text{nizk.td}, (\text{nizk.crs}, C'_{\text{Rt}}, b_{\text{Rt}}), \pi_{\text{Rt}})$. Assume that there exists a PPT adversary $\mathcal{A}$ that breaks the security property in our lemma, we build a reduction algorithm $\mathcal{B}$ that breaks the proof of knowledge property of NIZK.

$\mathcal{A}$ starts by setting language $\mathcal{L}$ and security parameter λ . Reduction algorithm $\mathcal{B}$ then queries the NIZK challenger with 1^λ . The challenger outputs $\text{nizk.crs}, \text{nizk.td}$. $\mathcal{B}$ sets $\text{crs} = \text{nizk.crs}$, $\text{td} = (\text{nizk.crs}, \text{nizk.td})$ and sends crs, td to $\mathcal{A}$. $\mathcal{A}$ outputs (C, b, π) .

Consider that $\mathcal{E}'(\text{td}, (C, b), \pi)$ outputs tree $T = (V_T, E_T)$ and $\{(C'_v, b_v, \omega_v)\}_{v \in V_T}$. Let Rt be the root of T . Then, based on our design of $\mathcal{E}'$, ω_{Rt} is extracted using $\text{NIZK.E}(\text{nizk.td}, (\text{crs}, C'_{\text{Rt}}, b_{\text{Rt}}), \pi_{\text{Rt}})$. Note that $C'_{\text{Rt}} = C$, $b_{\text{Rt}} = b$ and $\pi_{\text{Rt}} = \pi$, thus $\text{Verify}(\text{crs}, (C, b), \pi) = 1$ implies that $\text{NIZK.Verify}(\text{nizk.crs}, (\text{crs}, C'_{\text{Rt}}, b_{\text{Rt}}), \pi_{\text{Rt}}) = 1$. $\mathcal{B}$ could break the proof of knowledge property by outputting $(\text{crs}, C'_{\text{Rt}}, b_{\text{Rt}})$ and π_{Rt} .

Inductive Step for non-leaf node v : Suppose that the lemma holds for every node that comes before node v in the reverse topological order. We show that the lemma also holds for v . Assume that for some PPT adversary $\mathcal{A}$, $\text{Verify}(\text{crs}, (C, b), \pi) = 1$ but ω_v is not a valid witness for $(\text{crs}, C'_v, b_v) \in \mathcal{L}$ where $\omega_v \leftarrow \text{NIZK.E}(\text{nizk.td}, (\text{crs}, C'_v, b_v), \pi_v)$. We build a reduction algorithm $\mathcal{B}$ that breaks the proof of knowledge property of NIZK.

$\mathcal{A}$ sets $\mathcal{L}$ and λ . $\mathcal{B}$ queries the NIZK challenger with 1^λ and the challenger responds with $\text{nizk.crs}, \text{nizk.td}$. $\mathcal{B}$ sets and outputs $\text{crs} = \text{nizk.crs}$ and $\text{td} = (\text{nizk.crs}, \text{nizk.td})$. $\mathcal{A}$ then outputs (C, b, π) . Let u be the parental node of v . Then by the above inductive hypothesis, with all but negligible probability, ω_u is a valid witness for $(\text{crs}, C'_u, b_u) \in \mathcal{L}$, which already implies that

$\text{NIZK.Verify}(\text{nizk.crs}, (\text{nizk.crs}, C'_v, b_v), \pi_v) = 1$. If ω_v is not a valid witness, then $\mathcal{B}$ breaks the argument of knowledge property of nizk by extracting and outputting (crs, C'_v, b_v) . $\square$

The recursive extractor $\mathcal{E}'$ falls short as a sufficient extractor, as it only produces the instance and witness for individual nodes. We introduce the actual extractor $\mathcal{E}$. $\mathcal{E}$ takes as input (C, b, π) , where circuit C takes k input bits. Ideally, $\mathcal{E}$ outputs a valid witness $(b_1, \dots, b_k)$ satisfying $C(b_1, \dots, b_k) = b$.

$\mathcal{E}(\text{td}, (C, b), \pi) \rightarrow (b_1, \dots, b_k)$. It runs $\mathcal{E}'(\text{td}, (C, b), \pi)$ and obtains $T, \{(C'_v, b_v, \omega_v)\}_{v \in V_T}$. Next for all $i \in [k]$, let v_i be the i -th leaf node, it outputs $(b_{v_1}, \dots, b_{v_k})$.

Let $\{C_v\}_{v \in V_T}$ be the output of $\text{Decompose}(C)$. We note that by Claim 5.12, for each internal node v with child nodes $(v_1, \dots, v_\ell)$, it holds that $C_v(b_{v_1}, \dots, b_{v_\ell}) = 1$. Since circuits $\{C_v\}_{v \in V_T}$ is a decomposition of C , we have $C(b_{v_1}, \dots, b_{v_k}) = 1$. Our theorem follows. $\square$

Lemma 5.13 (Unlinkability). Assume that NIZK satisfies zero-knowledge property, then Construction 5.8 satisfies unlinkability.

Proof. Before proceeding to the proof, we define the term atomic circuit: A circuit C is considered atomic if output of $\text{Decompose}(C)$ includes the circuit C itself. We argue the following: For any set of boolean circuits $C, C_1, \dots, C_\ell$, where $C' = \text{Compose}(C, C_1, \dots, C_\ell)$, the output by $\text{Prove}(\text{crs}, C', b, \{b_i\}_{i \in [k]})$ is indistinguishable from $\text{Eval}(\text{crs}, C, \{(C_i, b_i, \pi_i)\}_{i \in [k]})$, thereby unlinkability holds. By Theorem 5.7, whether we decompose $C, C_1, \dots, C_\ell$ or directly decompose C' , the resulting atomic circuit sets are the same. Thus by definition of algorithm Compose , the circuits C'_{Rt} at the roots of both $\text{Prove}(\text{crs}, C', b, \{b_i\}_{i \in [k]})$ and $\text{Eval}(\text{crs}, C, \{(C_i, b_i, \pi_i)\}_{i \in [k]})$ are equivalent to C' . Furthermore, the output proof $\pi = \pi_{\text{Rt}}$ at the final hop by both Prove and Eval is generated using the zero-knowledge proof system NIZK under the same instance $(\text{crs}, C'_{\text{Rt}}, b_{\text{Rt}})$. Since language $\mathcal{L}$ is deterministic, given a fixed instance, our design already satisfies unlinkability by the zero-knowledge property of NIZK. $\square$

Lemma 5.14 (Zero knowledge). Assume that NIZK satisfies zero-knowledge property, then Construction 5.8 satisfies zero-knowledge property.

Proof. Given instance (C, b) , the proof π by our design is itself a NIZK proof π_{Rt} under instance $(C'_{\text{Rt}}, b_{\text{Rt}})$ where $C'_{\text{Rt}} = C$ and $b_{\text{Rt}} = b$. Thus our design immediately satisfies zero-knowledge property. $\square$

We close the section with the following corollaries, which are immediate by Corollary 5.6.

Corollary 5.15. Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists a homomorphic NIZK scheme for any NP language.

6 Complexity-Preserving Semi-Malicious to Malicious Compiler for mrNISC

In this section, we provide the definition, construction, and analysis of a maliciously secure multi-party non-interactive secure computation scheme (mrNISC) for n parties using a semi-maliciously secure mrNISC scheme for n parties and reusable Encrypted RAM Delegation scheme (rEDel) w.r.to HT.

6.1 Definition

Syntax. A multi-party reusable non-interactive secure computation scheme for n parties (NISC) consists for the following polynomial time algorithms:

$\text{Setup}(1^\lambda) \rightarrow \text{crs}$. The probabilistic setup algorithm takes as input the security parameter and outputs the common random string crs .

$\text{InpEnc}(\text{crs}, \text{id}, x; r) \rightarrow (\hat{x}, \tau)$. The probabilistic input encoding algorithm takes as input a crs , identity of the party id , input x , randomness r , and outputs the public encoding of x , $\hat{x}$ and trapdoor τ .

$\text{FuncEnc}(\text{crs}, \text{id}, f, \{\hat{x}_{\text{id}_x}\}_{\text{id}_x \in I}, \tau_{\text{id}}) \rightarrow \hat{f}_{\text{id}}$. The function encoding algorithm takes as input a crs , identity of the party id , description of function f , public encodings for all $\text{id}_x \in I \subseteq [n]$, $\{\hat{x}_{\text{id}_x}\}_{\text{id}_x \in I}$ ($\text{id} \in I$), trapdoor for the id -th party τ_{id} , and outputs the function encoding for the id -th party, $\hat{f}_{\text{id}}$.

$\text{Eval}(\text{crs}, f, \{\hat{f}_{\text{id}}\}_{\text{id} \in I}, \{\hat{x}_{\text{id}}\}_{\text{id} \in I}) \rightarrow y$. The evaluation algorithm takes as input a crs , the function encodings, $\{\hat{f}_{\text{id}}\}_{\text{id} \in I}$ and input encodings $\{\hat{x}_{\text{id}}\}_{\text{id} \in I}$ for all $\text{id} \in I$, and outputs a value y .

Definition 6.1. An NISC scheme for n parties ($\text{Setup}, \text{InpEnc}, \text{FuncEnc}, \text{Eval}$) is required to satisfy the following properties:

Correctness. For any $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $I \subseteq [n]$, efficient computation f , we have:

$$\Pr \left[\begin{array}{l} f(\{x_{\text{id}}\}_{\text{id} \in I}) = \\ \text{Eval}(\text{crs}, f, \{\hat{f}_{\text{id}}\}_{\text{id} \in I}, \\ \{\hat{x}_{\text{id}}\}_{\text{id} \in I}) \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), \\ \forall \text{id} \in I, (\hat{x}_{\text{id}}, \tau_{\text{id}}) \leftarrow \text{InpEnc}(\text{crs}, \text{id}, x_{\text{id}}; r_{\text{id}}), \\ \hat{f}_{\text{id}} \leftarrow \text{FuncEnc}(\text{crs}, \text{id}, f, \{\hat{x}_{\text{id}_x}\}_{\text{id}_x \in I}, \tau_{\text{id}}) \end{array} \right] = 1$$

Static Corruptions, Semi-Malicious, Adaptive Security. For any $\lambda \in \mathbb{N}$, any $n = n(\lambda)$, any $H \subseteq [n]$, $|H| \geq 1$ and admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that:

$$\left| \Pr \left[1 \leftarrow \text{Expt}_0^{\mathcal{C}, \mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Expt}_1^{\mathcal{S}, \mathcal{A}}(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

where the definitions of admissible adversary, $\text{Expt}_0^{\mathcal{C}, \mathcal{A}}$ and $\text{Expt}_1^{\mathcal{S}, \mathcal{A}}$ are provided in Figure 4. If the adversary instead behaves maliciously, we say the scheme is maliciously secure.

Remark 6.2. A semi-maliciously secure mrNISC scheme's crs can be empty (or trivial). For malicious security, mrNISC scheme has to be in the crs model for PPT simulation security.

Theorem 6.3 ([BL20, BJKL21, AJJM21, Shi22]). Assuming $X \in \{\text{SXDH}, \text{LWE}\}$, there exists a mrNISC scheme for n parties for all efficient functions that is semi-maliciously secure.

Admissible Adversary. An admissible adversary is a stateful PPT machine that queries for a single input encoding from all parties (corrupt and honest) and unbounded functional encodings.

$\text{Expt}_0^{\mathcal{C}, \mathcal{A}}(1^\lambda)$. This is the Real experiment parameterized by an honest challenger $\mathcal{C}$. $\mathcal{A}$ submits the set of honest parties H and receives $\text{crs} \leftarrow \text{Setup}(1^\lambda)$ from $\mathcal{C}$. $\mathcal{A}$ makes the following queries in an adaptive manner. After this, output $\mathcal{A}$'s guess b' .

- **HONEST INPUT ENCODING:** $\mathcal{A}$ sends $(\text{id}, x_{\text{id}})$ for $\text{id} \in H$ and receives $\hat{x}_{\text{id}}$ where $(\hat{x}_{\text{id}}, \tau_{\text{id}}) \leftarrow \text{InpEnc}(\text{crs}, \text{id}, x_{\text{id}}; r_{\text{id}})$ for some randomness $r_{\text{id}} \leftarrow \{0, 1\}^\lambda$.
- **CORRUPT INPUT ENCODING:** $\mathcal{A}$ sends $(\text{id}, x_{\text{id}}, r_{\text{id}})$ for $\text{id} \notin H$ to $\mathcal{C}$.
- **FUNCTION ENCODING:** $\mathcal{A}$ sends (id, f, I) for $\text{id} \in H$ such that input encodings for all $\text{idx} \in I$ are queried, and receives $\hat{f}_{\text{id}} \leftarrow \text{FuncEnc}(\text{crs}, \text{id}, f, \{\hat{x}_{\text{idx}}\}_{\text{idx} \in I}, \tau_{\text{id}})$.

$\text{Expt}_1^{\mathcal{S}, \mathcal{A}}(1^\lambda)$. This is the Ideal experiment parameterized by a stateful simulator $\mathcal{S}$. $\mathcal{A}$ submits the set of honest parties H and receives $\text{crs} \leftarrow \mathcal{S}(1^\lambda)$. $\mathcal{A}$ makes the following queries in an adaptive manner. After this, output $\mathcal{A}$'s guess b' .

- **HONEST INPUT ENCODING:** $\mathcal{A}$ sends $(\text{id}, x_{\text{id}})$ for $\text{id} \in H$ and receives $\hat{x}_{\text{id}}$ where $\hat{x}_{\text{id}} \leftarrow \mathcal{S}(\text{id})$.
- **CORRUPT INPUT ENCODING:** $\mathcal{A}$ sends $(\text{id}, x_{\text{id}}, r_{\text{id}})$ for $\text{id} \notin H$.
- **FUNCTION ENCODING:** $\mathcal{A}$ sends (id, f, I) for $\text{id} \in H$ such that input encodings for all $\text{idx} \in I$ are queried. If $\forall \text{idx} \in I \cap H, \text{idx} \neq \text{id}$, function encodings are queried, set $y = f(\{x_{\text{id}}\}_{\text{id} \in I})$. Otherwise, set $y = \perp$. Send $\hat{f}_{\text{id}} \leftarrow \mathcal{S}(\text{id}, f, I, y)$ to $\mathcal{A}$.

Figure 4: Real and Ideal experiments for security of SMSC

6.2 Construction

Construction 6.4 (Malicious NISC). Let $\text{r1Enc} = (\text{Encode}, \text{Decode})$ be a secure rate-1 message encoding scheme (Definition 2.14) and $\text{SMSC} = (\text{InpEnc}, \text{FuncEnc}, \text{Eval})$, be a semi-maliciously secure mrNISC scheme (Definition 6.1). Using a reusable encrypted RAM delegation scheme $\text{rEDel} = (\text{Setup}, \text{PrivEnc}, \text{Prove}, \text{VerifyEnc}, \text{Verify})$ w.r.to HT for language $\mathcal{L}_{\mathcal{M}, T}^{\text{NISC}} = \{(x_{\text{pb}}, x_{\text{pr}}, T) : \mathcal{M}_{\text{NISC}}(x_{\text{pb}}, x_{\text{pr}}, T) = 1\}$ where $\mathcal{M}_{\text{NISC}}$ is defined in Figure 6.4, we provide the construction of an maliciously secure mrNISC scheme NISC for n parties for the function f below.

RAM Machine $\mathcal{M}_{\text{NISC}}$	Function g
Public Input: $\text{id}, f, \{\hat{x}_{\text{id}_x}\}_{\text{id}_x \in I}, \hat{f}_{\text{id}}$ Private Input: x, r, τ Output: Output 1 if and only if $(\hat{x}, \tau) = \text{InpEnc}(\text{id}, x; r)$. $\hat{f}_{\text{id}} = \text{FuncEnc}(\text{id}, f, \{\hat{x}_{\text{id}_x}\}_{\text{id}_x \in I}, \tau; r)$.	Hardwired: $\lambda, \{\rho_{\text{id}}\}_{\text{id} \in I}, f$ Input: $\{\text{sk}_{\text{id}}\}_{\text{id} \in I}$ Output: $\forall \text{id} \in I, x_{\text{id}} = \text{r1Enc.Decode}(1^\lambda, \rho_{\text{id}}, \text{sk}_{\text{id}})$. - Output $f(\{x_{\text{id}}\}_{\text{id} \in I})$.

$\text{Setup}(1^\lambda) \rightarrow \text{crs}$. This probabilistic algorithm samples a reusable encrypted RAM delegation crs for each $\text{id} \in [n]$, $\text{del.crs}_{\text{id}} \leftarrow \text{rEDel.Setup}(1^\lambda, 1^{\ell_{\text{pr}}})$ where ℓ_{pr} is the maximum length of the private input for $\mathcal{M}_{\text{NISC}}$ across all n parties. Also, sample SMSC's crs , $\text{smc.crs} \leftarrow \text{SMSC.Setup}(1^\lambda)$. Output $\text{crs} = (\text{smc.crs}, (\text{del.crs}_{\text{id}})_{\text{id} \in [n]})$.

$\text{InpEnc}(\text{crs}, \text{id}, x; r) \rightarrow (\hat{x}, \tau)$. This probabilistic algorithm parses crs , using smc.crs and $\text{del.crs}_{\text{id}}$ and does the following.

1. Sample $r' \leftarrow \{0, 1\}^\lambda$. Compute $(\rho, \text{sk}) = \text{r1Enc.Encode}(1^\lambda, x)$.
2. Sample input encoding for SMSC for sk , $(\text{smc.s}\hat{\text{sk}}, \text{smc.}\tau) \leftarrow \text{SMSC.InpEnc}(\text{smc.crs}, \text{sk}; r')$.
3. Compute private encoding from rEDel , $(\text{del.eh}_{\text{pr}}, \text{del.}\pi_{\text{enc}}, \text{del.aux}) \leftarrow \text{rEDel.PrivEnc}(\text{del.crs}_{\text{id}}, (\text{sk}, r', \text{smc.}\tau))$.
4. Output $\hat{x} = (\rho, \text{smc.s}\hat{\text{sk}}, \text{del.eh}_{\text{pr}}, \text{del.}\pi_{\text{enc}})$, $\tau = \text{del.aux}$.

$\text{FuncEnc}(\text{crs}, \text{id}, f, \{\hat{x}_{\text{id}_x}\}_{\text{id}_x \in I}, \tau_{\text{id}}) \rightarrow \hat{f}_{\text{id}} / \perp$. This probabilistic algorithm parses the crs , uses smc.crs , $(\text{del.crs}_{\text{id}_x})_{\text{id}_x}$, τ_{id} as $\text{del.aux}_{\text{id}}$, $\hat{x}_{\text{id}_x}$ as $(\rho_{\text{id}_x}, \text{smc.s}\hat{\text{sk}}_{\text{id}_x}, \text{del.eh}_{\text{pr}, \text{id}_x}, \text{del.}\pi_{\text{enc}, \text{id}_x})$ for $\text{id}_x \in I$, and does the following.

1. If for any $\text{id}_x \in I$, $\text{rEDel.VerifyEnc}(\text{del.crs}_{\text{id}_x}, \text{del.eh}_{\text{pr}, \text{id}_x}, \text{del.}\pi_{\text{enc}, \text{id}_x}) = 0$, abort and output $\perp$.
2. Otherwise, sample, using SMSC, function encoding for g defined in Figure 6.4. $\text{smc.g}_{\text{id}} \leftarrow \text{SMSC.FuncEnc}(\text{smc.crs}, \text{id}, g, \{\text{smc.s}\hat{\text{sk}}_{\text{id}_x}\}_{\text{id}_x}, \text{smc.}\tau_{\text{id}}; r'_{\text{id}})$ where $\text{smc.}\tau_{\text{id}}, r'_{\text{id}}$ are derived from $\text{del.aux}_{\text{id}}$.
3. Compute proof, $\text{del.}\pi_{\text{edel}, \text{id}} \leftarrow \text{rEDel.Prove}(\text{del.crs}_{\text{id}}, (\text{id}, g, \{\text{smc.s}\hat{\text{sk}}_{\text{id}_x}\}_{\text{id}_x}, \text{smc.g}_{\text{id}}), \text{del.aux}_{\text{id}})$.
4. Output $\hat{f}_{\text{id}} = (\text{smc.g}_{\text{id}}, \text{del.}\pi_{\text{edel}, \text{id}})$.

$\text{Eval}(\text{crs}, f, \{\hat{x}_{\text{id}}\}_{\text{id} \in I}, \{\hat{f}_{\text{id}}\}_{\text{id} \in I}) \rightarrow y / \perp$. This algorithm parses crs , uses smc.crs , $(\text{del.crs}_{\text{id}})_{\text{id}}$, for each $\text{id} \in I$, $\hat{x}_{\text{id}}$ as $(\rho_{\text{id}}, \text{smc.s}\hat{\text{sk}}_{\text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}})$, $\hat{f}_{\text{id}}$ as $(\text{smc.g}_{\text{id}}, \text{del.}\pi_{\text{edel}, \text{id}})$ and does the following.

1. Construct g and compute for $\text{id} \in I$, $\text{h}_{\text{pb}, \text{id}} = \text{HT.Hash}(\text{ht.hk}, \text{id}, g, \{\text{smc.s}\hat{\text{sk}}_{\text{id}_x}\}_{\text{id}_x \in I}, \text{smc.g}_{\text{id}})$.
2. If for any $\text{id} \in I$, $\text{rEDel.VerifyEnc}(\text{del.crs}_{\text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}}) = 0$ or $\text{rEDel.Verify}(\text{del.crs}_{\text{id}}, \text{h}_{\text{pb}, \text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, 1, \text{del.}\pi_{\text{edel}, \text{id}}) = 0$, abort, output $\perp$.
3. Output $\text{SMSC.Eval}(\text{smc.crs}, g, \{\text{smc.s}\hat{\text{sk}}_{\text{id}}\}_{\text{id}}, \{\text{smc.g}_{\text{id}}\}_{\text{id}})$.

Theorem 6.5. If SMSC is a semi-maliciously secure mrNISC scheme (Definition 6.1), r1Enc is a secure rate-1 message encoding scheme (Definition 2.14), and rEDel is a reusable encrypted RAM delegation scheme w.r.to HT (Definition 3.2) for machine $\mathcal{M}_{\text{NISC}}$, then Construction 6.4 is a complexity-preserving maliciously secure mrNISC scheme.

Proof. We show that Construction 6.4 is correct and efficient as follows:

Correctness. We have that $\text{crs} = (\text{smc.crs}, (\text{del.crs}_{\text{id}_x})_{\text{id}_x \in [n]})$, for each $\text{id} \in I$,

$$\begin{aligned}\hat{x}_{\text{id}} &= (\text{r1Enc.Encode}(1^\lambda, x_{\text{id}}), \text{smc}.\hat{\text{sk}}_{\text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, \text{del}.\pi_{\text{enc}, \text{id}}, \text{smc}.\tau_{\text{id}}) \\ \hat{f}_{\text{id}} &= (\text{SMSC.FuncEnc}(\text{smc.crs}, \text{id}, g, \{\text{smc}.\hat{\text{sk}}_{\text{id}_x}\}_{\text{id}_x \in I}, \text{smc}.\tau_{\text{id}}; r'_{\text{id}}), \\ &\quad \text{rEDel.Prove}(\text{del.crs}_{\text{id}}, x_{\text{pb}, \text{id}}, \text{del.aux}_{\text{id}}))\end{aligned}$$

where $(\text{del.eh}_{\text{pr}, \text{id}}, \text{del}.\pi_{\text{enc}, \text{id}}, \text{del.aux}_{\text{id}}) \leftarrow \text{rEDel.PrivEnc}(\text{del.crs}_{\text{id}}, (\text{sk}_{\text{id}}, r'_{\text{id}}, x))$ and $(\text{smc}.\hat{\text{sk}}_{\text{id}}, \text{smc}.\tau_{\text{id}}) \leftarrow \text{SMSC.InpEnc}(\text{smc.crs}, \text{sk}_{\text{id}}; r'_{\text{id}})$. By completeness of rEDel,

$$\begin{aligned}\text{rEDel.VerifyEnc}(\text{del.crs}_{\text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, \text{del}.\pi_{\text{enc}, \text{id}}) &= 1 \text{ and} \\ \text{rEDel.Verify}(\text{del.crs}_{\text{id}}, h_{\text{pb}, \text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, 1, \\ \text{rEDel.Prove}(\text{del.crs}_{\text{id}}, \text{id}, g, \{\text{smc}.\hat{\text{sk}}_{\text{id}_x}\}_{\text{id}_x}, \text{smc}.\hat{g}_{\text{id}}, \text{del.aux}_{\text{id}})) &= 1\end{aligned}$$

By correctness of SMSC, $y = g(\{\text{sk}_{\text{id}}\}_{\text{id} \in I}) = f(\{x_{\text{id}}\}_{\text{id} \in I})$.

Efficiency. We prove that our transformation is rate-1 in the input x_{id} for input encoding, and complexity-preserving for function encoding.

INPUT ENCODING. As K_{id} and r'_{id} are λ -bit strings, length of $\text{smc}.\hat{\text{sk}}_{\text{id}}$ and $\text{smc}.\tau_{\text{id}}$ is polynomial in λ . Similarly, the length of $\text{del.eh}_{\text{pr}, \text{id}}$ and $\text{del}.\pi_{\text{enc}, \text{id}}$ is polynomial in λ . However, ρ_{id} is the same length as x_{id} (i.e., l). Thus $\hat{x}_{\text{id}}$ is a rate-1 string and τ_{id} is independent of the length of x_{id} .

FUNCTION ENCODING. Note that the size of the function g , $|g| = |f| + \text{poly}(\lambda)$ where the polynomial $\text{poly}(\lambda)$ depends on r1Enc. By efficiency of rEDel, we have that size of $\text{del}.\pi_{\text{edl}, \text{id}}$ is $\text{poly}(\lambda) + \text{poly}(\log |g|, \lambda)$. Hence, the size of $\hat{f}_{\text{id}}$ is the size of function encoding of SMSC for g and some additional polynomial in λ . Thus, the function encoding for f in Construction 6.4 is complexity preserving.

□

6.3 Security Analysis of Construction 6.4

Here, we show that Construction 6.4 is secure against malicious adversaries in the static corruptions, adaptive query setting.

Proof of static corruptions, malicious, adaptive query security. We show that if SMSC, r1Enc are secure, and rEDel is input-hiding, then Construction 6.4 is a maliciously secure mrNISC scheme. We define the following hybrids:

hyb_0^A This is the honest experiment $\text{Expt}_0^{C, A}$ with malicious adversary $\mathcal{A}$.

$\text{hyb}_{1,j}^A$ This is the same as hyb_0^A except that for $j \in [n+1]$, we simulate rEDel instantiations for the first $j-1$ honest parties.

$\text{hyb}_{2,j}^A$ This is the same as $\text{hyb}_{1,n+1}^A$ except that for $j \in [n+1]$, we use the extractor rEDel for the first $j-1$ corrupt parties to extract the corrupt party input and randomness. If these values are not consistent with SMSC function encodings provided by the malicious adversary, abort and output $\perp$.

hyb_3^A This is the same as $\text{hyb}_{2,n+1}^A$ except that we simulate all encodings of SMSC for honest parties.

$\text{hyb}_{4,j}^A$ This is same as hyb_3^A except that for $j \in [n+1]$, we sample random ρ_{id} instead of using r1Enc.Encode for the first $j-1$ honest parties. $\text{hyb}_{4,n+1}^A$ is the description of simulator $\mathcal{S}$.

Let $\text{hyb}_{i(j)}^A(1^\lambda)$ denote the output of the experiment $\text{hyb}_{i(j)}^A$.

Claim 6.6. For every $\lambda \in \mathbb{N}$, any PPT adversary $\mathcal{A}$,

$$\Pr \left[1 \leftarrow \text{hyb}_0^A(1^\lambda) \right] = \Pr \left[1 \leftarrow \text{hyb}_{1,1}^A(1^\lambda) \right]$$

Proof. The proof of this claim is immediate from the distributions of hyb_0^A and $\text{hyb}_{1,1}^A$. $\square$

Claim 6.7. If rEDel is input-hiding, then for any PPT adversary $\mathcal{A}$, for any $j \in [n]$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \text{hyb}_{1,j}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{hyb}_{1,j+1}^A(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

Proof. Note that if $j \in \tilde{H}$ where $\tilde{H} = [n] \setminus H$, $\text{hyb}_{1,j}^A$ and $\text{hyb}_{1,j+1}^A$ are identical. Hence, w.l.o.g. assume $j \in H$. Assuming that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{hyb}_{1,j}^A$ and $\text{hyb}_{1,j+1}^A$ such that $|\Pr [1 \leftarrow \text{hyb}_{1,j}^A(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_{1,j+1}^A(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{\text{del}}$ that breaks the input-hiding property of rEDel as follows:

1. $\mathcal{B}_{\text{del}}$ receives crs from the challenger $\mathcal{C}$. $\mathcal{A}$ submits the number of parties n and the set of honest parties H . If $\text{id} \in H, \text{id} < j$, sample $\text{del.crs}_{\text{id}} \leftarrow \text{rEDel.S}(1^\lambda, 1^{\ell_{\text{pr}}})$. Otherwise if $\text{id} = j$, set $\text{del.crs}_{\text{id}} := \text{crs}$. Otherwise, if $\text{id} > j$, sample $\text{del.crs}_{\text{id}} \leftarrow \text{rEDel.Setup}(1^\lambda, 1^{\ell_{\text{pr}}})$ in accordance with $\mathcal{M}_{\text{NISC}}$. Sample $\text{smc.crs} \leftarrow \text{SMSC.Setup}(1^\lambda)$ and send $(\text{smc.crs}, (\text{del.crs}_{\text{id}})_{\text{id}})$ to $\mathcal{A}$.
2. When $\mathcal{A}$ queries for input encoding of $(\text{id}, x_{\text{id}})$ for $\text{id} \in H$, do the following.
 - Sample $r'_{\text{id}} \leftarrow \{0, 1\}^\lambda$. Compute $(\rho_{\text{id}}, \text{sk}_{\text{id}}) \leftarrow \text{r1Enc.Encode}(1^\lambda, x_{\text{id}})$.
 - Compute $(\text{smc.sk}_{\text{id}}, \text{smc.}\tau_{\text{id}}) \leftarrow \text{SMSC.InpEnc}(\text{smc.crs}, \text{sk}_{\text{id}}; r'_{\text{id}})$.
 - If $\text{id} < j$, $(\text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}}) \leftarrow \text{rEDel.S}(\text{del.crs}_{\text{id}}, 1^{|\text{sk}_{\text{id}}| + \lambda + |\text{smc.}\tau_{\text{id}}|})$. Otherwise, if $\text{id} > j$, $(\text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}}, \text{del.aux}_{\text{id}}) \leftarrow \text{rEDel.PrivEnc}(\text{del.crs}_{\text{id}}, (\text{sk}_{\text{id}}, r_{\text{id}}, \text{smc.}\tau_{\text{id}}))$. Otherwise, if $\text{id} = j$, send $(\text{sk}_{\text{id}}, r_{\text{id}}, \text{smc.}\tau_{\text{id}})$ to $\mathcal{C}$ and receive $(\text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}})$.

Send $\hat{x}_{\text{id}} = (\rho_{\text{id}}, \text{smc.sk}_{\text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}})$ to $\mathcal{A}$.

3. $\mathcal{A}$ submits input encoding for $\text{id} \in \tilde{H}$, parse $\hat{x}_{\text{id}}$ as $(\rho_{\text{id}}, \text{smc.sk}_{\text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}})$. If $\text{rEDel.VerifyEnc}(\text{del.crs}_{\text{id}}, \text{del.eh}_{\text{pr}, \text{id}}, \text{del.}\pi_{\text{enc}, \text{id}}) = 0$, abort and output $\perp$.

4. When $\mathcal{A}$ queries for function encoding of (id, f, I) for $id \in I \cap H$, such that all the input encodings for parties $idx \in I$ are known, do the following:

- If for any $idx \in I \cap \tilde{H}$, if $r\mathcal{E}Del.VerifyEnc(\text{del.crs}_{idx}, \text{del.eh}_{pr,idx}, \text{del.\pi}_{enc,idx}) = 0$, abort and output $\perp$.
- Sample $\text{smc}.\hat{g}_{id} \leftarrow \text{SMSC.FuncEnc}(\text{smc.crs}, id, g, \{\text{smc}.\hat{sk}_{idx}\}_{idx}, \text{smc}.\tau_{id}; r'_{id})$.
- If $id < j$, sample $\text{del}.\pi_{edl,id} \leftarrow r\mathcal{E}Del.\mathcal{S}(\text{del.crs}_{id}, (id, g, \{\text{smc}.\hat{sk}_{idx}\}_{idx}, \text{smc}.\hat{g}_{id}), 1)$. Otherwise, if $id > j$, $\text{del}.\pi_{edl,id} \leftarrow r\mathcal{E}Del.Prove(\text{del.crs}_{id}, (id, g, \{\text{smc}.\hat{sk}_{idx}\}_{idx}, \text{smc}.\hat{g}_{id}), \text{del.aux}_{id})$. Otherwise, if $id = j$, send $((id, g, \{\text{smc}.\hat{sk}_{idx}\}_{idx}, \text{smc}.\hat{g}_{id}), 1)$ to $\mathcal{C}$ and receive $\text{del}.\pi_{edl,id}$.

Send $(\text{smc}.\hat{g}_{id}, \text{del}.\pi_{edl,id})$ to $\mathcal{A}$. Here, we use g as described in Figure 6.4.

5. $\mathcal{A}$ sends the function encoding for $id \in I \cap \tilde{H}$, such that all the input encodings for $idx \in I$ are known, parse $\hat{f}_{id}$ as $(\text{smc}.\hat{g}_{id}, \text{del}.\pi_{edl,id})$. If $r\mathcal{E}Del.Verify(\text{del.crs}_{id}, h_{pb,id}, \text{del.eh}_{pr,id}, 1, \text{del}.\pi_{edl,id}) = 0$, abort and output $\perp$ where $h_{pb,id} = \text{HT.Hash}(\text{ht.hk}, id, g, \{\text{smc}.\hat{x}_{idx}\}_{idx}, \text{smc}.\hat{g}_{id})$.
6. Output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ uses $r\mathcal{E}Del.Setup$, $r\mathcal{E}Del.PrivEnc$, and $r\mathcal{E}Del.Prove$ (resp. $r\mathcal{E}Del.\mathcal{S}$) in the experiment then $\mathcal{B}_{del}$ simulates $\text{hyb}_{1,j}^A$ (resp. $\text{hyb}_{1,j+1}^A$). Thus $\mathcal{B}_{del}$ has advantage $\epsilon(\lambda)$ in breaking input hiding property of $r\mathcal{E}Del$. $\square$

Claim 6.8. For every $\lambda \in \mathbb{N}$, any PPT adversary $\mathcal{A}$,

$$\Pr \left[1 \leftarrow \text{hyb}_{1,n+1}^A(1^\lambda) \right] = \Pr \left[1 \leftarrow \text{hyb}_{2,1}^A(1^\lambda) \right]$$

Proof. The proof of this claim is immediate from the distributions of $\text{hyb}_{1,n+1}^A$ and $\text{hyb}_{2,1}^A$. $\square$

Claim 6.9. If $r\mathcal{E}Del$ satisfies straight-line extraction, then for any PPT adversary $\mathcal{A}$, any $j \in [n+1]$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$, $\Pr [\perp \leftarrow \text{hyb}_{2,j}^A(1^\lambda)] \leq \text{negl}(\lambda)$.

Proof. Note that if $j \in H$, $\text{hyb}_{2,j}^A$ will not abort as there is no extractor involved in the hybrid. Hence, w.l.o.g assume $j \in \tilde{H}$. Assuming that there exists an adversary $\mathcal{A}$ such that $\Pr [\perp \leftarrow \text{hyb}_{2,j}^A(1^\lambda)] = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{del}$ that breaks the straight-line extraction property of $r\mathcal{E}Del$ as follows:

1. $\mathcal{B}_{del}$ receives crs from the challenger $\mathcal{C}$. $\mathcal{A}$ submits the number of parties n and the set of honest parties H . For each $id \in [n]$, if $id \in H$, sample $\text{del.crs}_{id} \leftarrow r\mathcal{E}Del.\mathcal{S}(1^\lambda, 1^{\ell_{pr}})$. Otherwise if $id \in \tilde{H}$ and $id < j$, sample $(\text{del.crs}_{id}, \text{del.td}_{id}) \leftarrow r\mathcal{E}Del.\mathcal{E}(1^\lambda, 1^{\ell_{pr}})$ in accordance with $\mathcal{M}_{NISC}$. Otherwise if $id \in \tilde{H}, id > j$, sample $\text{del.crs}_{id} \leftarrow r\mathcal{E}Del.Setup(1^\lambda, 1^{\ell_{pr}})$. If $id = j$, set $\text{del.crs}_{id} := \text{crs}$. Sample $\text{smc.crs} \leftarrow \text{SMSC.Setup}(1^\lambda)$ and send $(\text{smc.crs}, (\text{del.crs}_{id})_{id})$ to $\mathcal{A}$.
2. When $\mathcal{A}$ queries for input encoding of (id, x_{id}) for $id \in H$, do the following.
 - Sample $r'_{id} \leftarrow \{0, 1\}^\lambda$. Compute $(\rho_{id}, \text{sk}_{id}) \leftarrow \text{r1Enc.Encode}(1^\lambda, x_{id})$.
 - Compute $(\text{smc}.\hat{sk}_{id}, \text{smc}.\tau_{id}) \leftarrow \text{SMSC.InpEnc}(\text{smc.crs}, \text{sk}_{id}; r'_{id})$.
 - Sample $(\text{del.eh}_{pr,id}, \text{del}.\pi_{enc,id}) \leftarrow r\mathcal{E}Del.\mathcal{S}(\text{del.crs}_{id})$.

Send $\hat{x}_{id} = (\rho_{id}, \text{smsec}.\hat{\text{sk}}_{id}, \text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id})$ to $\mathcal{A}$.

3. $\mathcal{A}$ submits input encoding for $id \in \tilde{H}$, parse $\hat{x}_{id}$ as $(\rho_{id}, \text{smsec}.\hat{\text{sk}}_{id}, \text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id})$. If $\text{rE}Del.\text{VerifyEnc}(\text{del.crs}_{id}, \text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id}) = 0$, abort and output $\perp$. Otherwise, if $id < j$, extract $(\text{sk}_{id}, r'_{id}, \text{smsec}.\tau_{id}) \leftarrow \text{rE}Del.\mathcal{E}(\text{del.td}_{id}, \text{del}.\pi_{\text{enc},id})$. If $id = j$, send $(\text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id})$ to $\mathcal{C}$.
4. When $\mathcal{A}$ queries for function encoding of (id, f, I) for $id \in I \cap H$ such that all the input encodings for parties $\text{idx} \in I$ are known, do the following.
 - Sample $\text{smsec}.\hat{g}_{id} \leftarrow \text{SMSC}.\text{FuncEnc}(\text{smsec}.\text{crs}, id, g, \{\text{smsec}.\hat{\text{sk}}_{\text{idx}}\}_{\text{idx}}, \text{smsec}.\tau_{id}; r'_{id})$.
 - Sample $\text{del}.\pi_{\text{edel},id} \leftarrow \text{rE}Del.\mathcal{S}(\text{del.crs}_{id}, (id, g, \{\text{smsec}.\hat{\text{sk}}_{\text{idx}}\}_{\text{idx}}, \text{smsec}.\hat{g}_{id}), 1)$.

Send $(\text{smsec}.\hat{g}_{id}, \text{del}.\pi_{\text{edel},id})$ to $\mathcal{A}$. Here, we use g as described in Figure 6.4.

5. $\mathcal{A}$ sends the function encoding for $id \in I \cap \tilde{H}$, such that all the input encodings for $\text{idx} \in I$ are known, parse $\hat{f}_{id}$ as $(\text{smsec}.\hat{g}_{id}, \text{del}.\pi_{\text{edel},id})$, and do the following:
 - Set $x_{\text{pb},id} := (id, g, \{\text{smsec}.\hat{\text{sk}}_{\text{idx}}\}_{\text{idx}}, \text{smsec}.\hat{g}_{id})$ and compute $\text{h}_{\text{pb},id} = \text{HT}.\text{Hash}(\text{ht.hk}, x_{\text{pb},id})$.
 - For any $id \neq j$, $\text{rE}Del.\text{Verify}(\text{del.crs}_{id}, \text{h}_{\text{pb},id}, \text{del.eh}_{\text{pr},id}, 1, \text{del}.\pi_{\text{edel},id}) = 0$ abort and output $\perp$.
 - If $id < j$, $\mathcal{M}_{\text{NISC}}(x_{\text{pb},id}, (\text{sk}_{id}, r'_{id}, \text{smsec}.\tau_{id})) = 0$, abort and output $\perp$.
 - Otherwise, if $id = j$, send $(x_{\text{pb},id}, \text{del}.\pi_{\text{edel},id})$ to $\mathcal{C}$.

6. Output whatever $\mathcal{A}$ outputs.

Note that $\mathcal{B}_{\text{del}}$ is a valid adversary against the straight-line extractor of $\text{rE}Del$ with advantage $\epsilon(\lambda)$ whenever $\mathcal{A}$ can make trigger an abort in $\text{hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$. $\square$

Claim 6.10. If SMSC is a semi-maliciously secure mrNISC scheme, then for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \text{hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{hyb}_3^{\mathcal{A}}(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

Proof. Assuming that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{hyb}_{2,n+1}^{\mathcal{A}}$ and $\text{hyb}_3^{\mathcal{A}}$ such that $|\Pr [1 \leftarrow \text{hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_3^{\mathcal{A}}(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{\text{SMSC}}$ that breaks the security of SMSC as follows:

1. $\mathcal{A}$ submits the number of parties n and the set of honest parties H . Send n, H to $\mathcal{C}$ and receives $\text{smsec}.\text{crs}$. For $id \in [n]$, if $id \in H$, sample $\text{del.crs}_{id} \leftarrow \text{rE}Del.\mathcal{S}(1^\lambda, 1^{\ell_{\text{pr}}})$. Otherwise, sample $(\text{del.crs}_{id}, \text{del.td}_{id}) \leftarrow \text{rE}Del.\mathcal{E}(1^\lambda, 1^{\ell_{\text{pr}}})$ in accordance with $\mathcal{M}_{\text{NISC}}$. Send $(\text{smsec}.\text{crs}, (\text{del.crs}_{id})_{id})$ to $\mathcal{A}$.
2. When $\mathcal{A}$ queries for input encoding of (id, x_{id}) for $id \in H$, do the following.
 - Compute $(\rho_{id}, \text{sk}_{id}) \leftarrow \text{r1Enc}.\text{Encode}(1^\lambda, x_{id})$.
 - Send (id, sk_{id}) to $\mathcal{C}$ and receive $\text{smsec}.\hat{\text{sk}}_{id}$.
 - Sample $(\text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id}) \leftarrow \text{rE}Del.\mathcal{S}(\text{del.crs}_{id})$.

Send $\hat{x}_{id} = (\rho_{id}, \text{smc}.\hat{\text{sk}}_{id}, \text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id})$ to $\mathcal{A}$.

3. $\mathcal{A}$ submits input encoding for $id \in \tilde{H}$, parse $\hat{x}_{id}$ as $(\rho_{id}, \text{smc}.\hat{\text{sk}}_{id}, \text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id})$. If $\text{rE}.\text{Del}.\text{VerifyEnc}(\text{del.crs}_{id}, \text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id}) = 0$, abort and output $\perp$. Otherwise, extract $(\text{sk}_{id}, r'_{id}, \text{smc}.\tau_{id}) \leftarrow \text{rE}.\text{Del}.\mathcal{E}(\text{del.td}_{id}, \text{del.eh}_{\text{pr},id}, \text{del}.\pi_{\text{enc},id})$. Extract the input for id -th party, $x_{id} = \text{r1Enc}.\text{Decode}(1^\lambda, \rho_{id}, \text{sk}_{id})$. Send $(id, \text{sk}_{id}, r'_{id})$ to $\mathcal{C}$.
4. When $\mathcal{A}$ queries for function encoding of (id, f, I) for honest party $id \in I$ such that all the input encodings for parties $idx \in I$ are known, do the following.
 - Send (id, g, I) to $\mathcal{C}$ to receive $\text{smc}.\hat{g}_{id}$.
 - Sample $\text{del}.\pi_{\text{edel},id} \leftarrow \text{rE}.\text{Del}.\mathcal{S}(\text{del.crs}_{id}, (id, g, \{\text{smc}.\hat{\text{sk}}_{idx}\}_{idx}, \text{smc}.\hat{g}_{id}), 1)$.

Send $(\text{smc}.\hat{g}_{id}, \text{del}.\pi_{\text{edel},id})$ to $\mathcal{A}$. Here, we use g as described in Figure 6.4.

5. $\mathcal{A}$ sends the function encoding for $id \in I \cap \tilde{H}$, such that all the input encodings for $idx \in I$ are known, parse $\hat{f}_{id}$ as $(\text{smc}.\hat{g}_{id}, \text{del}.\pi_{\text{edel},id})$, and do the following:
 - Set $x_{\text{pb},id} := (id, g, \{\text{smc}.\hat{\text{sk}}_{idx}\}_{idx}, \text{smc}.\hat{g}_{id})$ and compute $\text{h}_{\text{pb},id} = \text{HT}.\text{Hash}(\text{ht.hk}, x_{\text{pb},id})$.
 - If $\text{rE}.\text{Del}.\text{Verify}(\text{del.crs}_{id}, \text{h}_{\text{pb},id}, \text{del.eh}_{\text{pr},id}, 1, \text{del}.\pi_{\text{edel},id}) = 0$ or $\mathcal{M}_{\text{NISC}}(x_{\text{pb},id}, (\text{sk}_{id}, r'_{id}, \text{smc}.\tau_{id})) = 0$ abort and output $\perp$.
6. Output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ uses $\text{SMSC}.\text{InpEnc}$ and $\text{SMSC}.\text{FuncEnc}$ (resp. $\text{SMSC}.\mathcal{S}$) in the experiment then $\mathcal{B}_{\text{SMSC}}$ simulates $\text{hyb}_{2,n+1}^A$ (resp. hyb_3^A). Thus $\mathcal{B}_{\text{SMSC}}$ has advantage $\epsilon(\lambda)$ in breaking the security of SMSC . $\square$

Claim 6.11. For every $\lambda \in \mathbb{N}$, any PPT adversary $\mathcal{A}$,

$$\Pr \left[1 \leftarrow \text{hyb}_3^A(1^\lambda) \right] = \Pr \left[1 \leftarrow \text{hyb}_{4,1}^A(1^\lambda) \right]$$

Proof. The proof of this claim is immediate from the distributions of hyb_3^A and $\text{hyb}_{4,1}^A$. $\square$

Claim 6.12. If r1Enc is a secure rate-1 message encoding scheme, then for any PPT adversary $\mathcal{A}$, for any $j \in [n]$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \text{hyb}_{4,j}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{hyb}_{4,j+1}^A(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

Proof. Note that if $j \in \tilde{H}$, $\text{hyb}_{4,j}^A$ and $\text{hyb}_{4,j+1}^A$ are identical. Hence, w.l.o.g assume $j \in H$. Assuming that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{hyb}_{4,j}^A$ and $\text{hyb}_{4,j+1}^A$ such that

$$\left| \Pr \left[1 \leftarrow \text{hyb}_{4,j}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{hyb}_{4,j+1}^A(1^\lambda) \right] \right| = \epsilon(\lambda)$$

for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{\text{r1Enc}}$ that breaks the security of r1Enc as follows:

1. Sample crs similar to hyb_3^A and send it to $\mathcal{A}$.
2. When $\mathcal{A}$ queries for input encoding of (id, x_{id}) for $id \in H$, do the following.

- If $\text{id} < j$, sample $\rho_{\text{id}} \leftarrow \{0, 1\}^{|x_{\text{id}}|}$. Otherwise if $\text{id} > j$, sample $(\rho_{\text{id}}, \text{sk}_{\text{id}}) \leftarrow \text{r1Enc}(1^\lambda, x_{\text{id}})$. Otherwise, if $\text{id} = j$, query the challenger $\mathcal{C}$ with x_{id} to receive ρ_{id} .
- Perform the rest of the steps same as hyb_3^A .

3. Perform the rest of the steps same as hyb_3^A .

If $\mathcal{C}$ uses r1Enc.Encode (resp. sampling ρ_j randomly) in the experiment then $\mathcal{B}_{\text{r1Enc}}$ simulates $\text{hyb}_{4,j}^A$ (resp. $\text{hyb}_{4,j+1}^A$). Thus $\mathcal{B}_{\text{r1Enc}}$ has advantage $\epsilon(\lambda)$ in breaking the security of r1Enc . $\square$

$\square$

6.4 Semi-Maliciously Secure mrNISC with Compact Communication

In this section, we show the construction, efficiency, and security of a semi-maliciously secure mrNISC scheme where the function encodings scale only with the output size, i.e., $|\widehat{f}_i| = L \cdot \text{poly}(\lambda, |I|)$ for any $i \in I$ and L is the output size of f , assuming the hardness of circularly secure LWE. Specifically, the function encodings of the mrNISC scheme depend only on the output size of f and not on the size or depth of the function f . We use a semi-maliciously secure mrNISC scheme for any poly-size function class and a multi-key homomorphic encryption scheme (Definition A.3).

Construction 6.13. Let $\text{mkFHE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Eval}, \text{Dec})$ be a secure multi-key fully homomorphic encryption scheme (Definition A.3) and $\text{SMSC} = (\text{InpEnc}, \text{FuncEnc}, \text{Eval})$, be a semi-maliciously secure mrNISC scheme (Definition 6.1). We provide the construction of a semi-maliciously secure mrNISC scheme with small function encodings below.

$\text{Setup}(1^\lambda) \rightarrow \text{crs}$. This probabilistic algorithm samples $\text{nisc.crs} \leftarrow \text{SMSC.Setup}(1^\lambda)$ and $\text{fhe.pp} \leftarrow \text{mkFHE.Setup}(1^\lambda)$ and outputs $\text{crs} = (\text{nisc.crs}, \text{fhe.pp})$.

$\text{InpEnc}(\text{crs}, \text{id}, x; r) \rightarrow (\widehat{x}, \tau)$. This probabilistic algorithm parses crs as $(\text{nisc.crs}, \text{fhe.pp})$, and does the following.

1. Sample mkFHE keys $(\text{fhe.pk}, \text{fhe.sk}) \leftarrow \text{mkFHE.KeyGen}(\text{fhe.pp})$.
2. Encrypt x using mkFHE, $\text{fhe.ct} \leftarrow \text{mkFHE.Enc}(\text{fhe.pk}, x)$.
3. Generate input encodings $(\text{nisc.sk}, \text{nisc}\tau) \leftarrow \text{SMSC.InpEnc}(\text{nisc.crs}, \text{id}, \text{fhe.sk})$.
4. Output $\widehat{x} = (\text{fhe.pk}, \text{fhe.ct}, \text{nisc.sk})$ and $\tau = \text{nisc}\tau$.

$\text{FuncEnc}(\text{crs}, \text{id}, f, \{\widehat{x}_{\text{id}_x}\}_{\text{id}_x \in I}, \tau_{\text{id}}) \rightarrow \widehat{f}_{\text{id}}$. This algorithm parses crs as $(\text{nisc.crs}, \text{fhe.pp})$, $\widehat{x}_{\text{id}_x}$ as $(\text{fhe.pk}_{\text{id}_x}, \text{fhe.ct}_{\text{id}_x}, \text{nisc.sk}_{\text{id}_x})$ for all $\text{id}_x \in I \subseteq [n]$, $\{\widehat{x}_{\text{id}_x}\}_{\text{id}_x \in I}$ ($\text{id} \in I$), τ_{id} as $\text{nisc}\tau_{\text{id}}$, and does the following.

1. Compute $\text{fhe.ct}_\ell = \text{mkFHE.Eval}(f_\ell, \{\text{fhe.pk}_{\text{id}_x}, \text{fhe.ct}_{\text{id}_x}\}_{\text{id}_x \in I})$ for $\ell \in [L]$ and L is the output size of f . Here, f_ℓ denotes the circuit that outputs the ℓ -th bit of $f(\{x_{\text{id}_x}\}_{\text{id}_x \in I})$.
2. Let g_ℓ denote the function $\text{mkFHE.Dec}(\cdot, \dots, \cdot, \text{fhe.ct}_\ell)$. For each $\ell \in [L]$, sample $\widehat{g}_{\ell, \text{id}} \leftarrow \text{SMSC.FuncEnc}(\text{nisc.crs}, \text{id}, g_\ell, \{\text{nisc.sk}_{\text{id}_x}\}_{\text{id}_x \in I}, \text{nisc}\tau_{\text{id}})$.
3. Output $\widehat{f}_{\text{id}} = \{\text{fhe.ct}_\ell, \widehat{g}_{\ell, \text{id}}\}_{\ell \in [L]}$.

$\text{Eval}(f, \{\widehat{f}_{\text{id}}\}_{\text{id} \in I}, \{\widehat{x}_{\text{id}}\}_{\text{id} \in I}) \rightarrow y$. This algorithm parses $\widehat{f}_{\text{id}}$ as $\{\text{fhe.ct}_{\ell, \text{id}}, \widehat{g}_{\ell, \text{id}}\}_{\ell \in [L]}$, $\widehat{x}_{\text{id}}$ as $(\text{fhe.pk}_{\text{id}}, \text{fhe.ct}_{\text{id}}, \text{nisc.sk}_{\text{id}})$ for each $\text{id} \in I$, does the following.

1. If $\text{fhe.ct}_{\text{id},\ell}$ are not the same for all $\text{id} \in I$, abort and output $\perp$. Otherwise, let fhe.ct_ℓ be the common one for each $\ell \in [L]$. For any $\ell \in [L]$, if $\text{fhe.ct}_\ell \neq \text{Eval}(f_\ell, \{\text{fhe.pk}_{\text{id}}, \text{fhe.ct}_{\text{id}}\}_{\text{id} \in I})$, abort and output $\perp$.
2. Otherwise, for each $\ell \in [L]$, define g_ℓ to be the function $\text{mkFHE.Dec}(\cdot, \dots, \cdot, \text{fhe.ct}_\ell)$. Compute $y_\ell = \text{SMSC.Eval}(g_\ell, \{\widehat{g}_{\ell,\text{id}}\}_{\text{id} \in I}, \{\text{nisc.sk}_{\text{id}}\}_{\text{id} \in I})$.
3. Output $y = (y_1, \dots, y_L)$.

Theorem 6.14. Assuming the hardness of circularly secure LWE, there exists a semi-maliciously secure mrNISC scheme for all efficiently computable functions such that the size of function encodings of NISC is $L \cdot \text{poly}(\lambda, |I|)$ where $|y| = L$ and $y = f(\{x_{\text{id}}\}_{\text{id} \in I})$ in the common random string model. In particular, function encoding sizes are independent of function size $|f|$ and input sizes $|x_{\text{id}}|$.

Proof. We show that Construction 6.13 satisfies correctness and has compact function encoding sizes below.

Correctness. This follows from correctness of SMSC and multi-hop correctness of mkFHE. For instance, for $\ell = 1$, by correctness of SMSC, we have that $y_1 = \text{mkFHE.Dec}(\{\text{fhe.sk}_{\text{id}}\}_{\text{id} \in I}, \text{fhe.ct}_1)$ and $\text{fhe.ct}_1 = \text{mkFHE.Eval}(f_1, \{\text{fhe.pk}_{\text{id}}, \text{fhe.ct}_{\text{id}}\}_{\text{id} \in I})$. Then, by correctness of mkFHE, $y_1 = f_1(\{x_{\text{id}}\}_{\text{id} \in I})$. Thus, $y = f(\{x_{\text{id}}\}_{\text{id} \in I})$.

Function Encoding Size. Consider the case of $\ell = 1$, then size of function g_1 is $\text{poly}(\lambda, |I|)$. Thus, size of $\widehat{g}_{1,\text{id}}$ is $\text{poly}(\lambda, |I|)$ for any $\text{id} \in I$. Similarly $\text{fhe.ct}_{1,\text{id}}$ encrypts one bit under $|I|$ secret keys. So, it's size is $\text{poly}(\lambda, |I|)$. This is the same for any $\ell \in [L]$ where L is the output size of f . Hence, the size of function encoding for any $\text{id} \in I$ is $L \cdot \text{poly}(\lambda, |I|)$. $\square$

Proof of static corruptions, semi-malicious, adaptive query security. We show that Construction 6.13 is semi-maliciously secure according to Definition 6.1. We define the following hybrids:

$\text{hyb}_0^{\mathcal{A}}$ This is the honest experiment $\text{Expt}_0^{\mathcal{C}, \mathcal{A}}$ with semi-malicious adversary $\mathcal{A}$.

$\text{hyb}_1^{\mathcal{A}}$ This is the same as $\text{hyb}_0^{\mathcal{A}}$ except we will use the stateful simulator to simulate all encodings of SMSC.

$\text{hyb}_{2,j}^{\mathcal{A}}$ This is the same as $\text{hyb}_1^{\mathcal{A}}$ except that for $j \in [|H| + 1]$, we set $\text{fhe.ct}_{\text{id}}$ to be encryptions of $0^{|x_{\text{id}}|}$ for the first $j - 1$ entries of the honest party set H . $\text{hyb}_{2,|H|+1}^{\mathcal{A}}$ is the description of simulator $\mathcal{S}$.

Let $\text{hyb}_{i,(j)}^{\mathcal{A}}(1^\lambda)$ denote the output of the experiment $\text{hyb}_{i,(j)}^{\mathcal{A}}$.

Claim 6.15. If SMSC satisfies static corruptions, semi-malicious adversary, adaptive queries, then for any PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\left| \Pr [1 \leftarrow \text{hyb}_0^{\mathcal{A}}(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_1^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda)$$

Proof. Assuming that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{hyb}_0^{\mathcal{A}}$ and $\text{hyb}_1^{\mathcal{A}}$ such that $|\Pr [1 \leftarrow \text{hyb}_0^{\mathcal{A}}(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_1^{\mathcal{A}}(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{\text{SMSC}}$ that breaks the security of SMSC as follows:

1. $\mathcal{A}$ submits the number of parties n and the set of honest parties H . Send n, H to $\mathcal{C}$ to receive nisc.crs . Sample $\text{fhe.pp} \leftarrow \text{mkFHE.Setup}(1^\lambda)$. Send $(\text{nisc.crs}, \text{fhe.pp})$ to $\mathcal{A}$.
 2. When $\mathcal{A}$ queries for input encoding of $(\text{id}, x_{\text{id}})$ for $\text{id} \in H$, do the following.
 - Compute $(\text{fhe.pk}_{\text{id}}, \text{fhe.sk}_{\text{id}}) \leftarrow \text{mkFHE.KeyGen}(\text{fhe.pp})$.
 - Encrypt x_{id} as $\text{fhe.ct}_{\text{id}} \leftarrow \text{mkFHE.Enc}(\text{fhe.pk}_{\text{id}}, x_{\text{id}})$.
 - Send $(\text{id}, \text{fhe.sk}_{\text{id}})$ to $\mathcal{C}$ and receive $\text{nisc.s}_{\text{id}}$.
- Send $\hat{x}_{\text{id}} = (\text{fhe.pk}_{\text{id}}, \text{fhe.ct}_{\text{id}}, \text{nisc.s}_{\text{id}})$ to $\mathcal{A}$.
3. $\mathcal{A}$ submits input for $\text{id} \in [n] \setminus H$ as $(\text{id}, x_{\text{id}}, \rho_{\text{id}})$. Using ρ_{id} , sample $(\text{fhe.pk}_{\text{id}}, \text{fhe.sk}_{\text{id}})$, encrypt x_{id} as $\text{fhe.ct}_{\text{id}} \leftarrow \text{mkFHE.Enc}(\text{fhe.pk}_{\text{id}}, x_{\text{id}})$, and derive the randomness used in SMSC , ρ'_{id} . Send $(\text{id}, \text{fhe.sk}_{\text{id}}, \rho'_{\text{id}})$ to $\mathcal{C}$.
 4. $\mathcal{A}$ queries for function encoding of (id, f, I) . Sample $\text{fhe.ct}_\ell = \text{mkFHE.Eval}(f, \{\text{fhe.pk}_{\text{id}_x}, \text{fhe.ct}_{\text{id}_x}\}_{\text{id}_x \in I})$ for $\ell \in [L]$ and send $(\text{id}, g_1, I), \dots, (\text{id}, g_L, I)$ to $\mathcal{C}$ to receive $\hat{g}_{\ell, \text{id}}$ for $\ell \in [L]$. Here, $g_\ell = \text{mkFHE.Dec}(\cdot, \dots, \cdot, \text{fhe.ct}_\ell)$. Send $\{\text{fhe.ct}_\ell, \hat{g}_{\ell, \text{id}}\}_{\ell \in [L]}$ to $\mathcal{A}$.
 5. Output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ is an honest challenger and uses SMSC.InpEnc and SMSC.FuncEnc (resp. SMSC.S) in the experiment then $\mathcal{B}_{\text{SMSC}}$ simulates hyb_0^A (resp. hyb_1^A). Thus $\mathcal{B}_{\text{SMSC}}$ has advantage $\epsilon(\lambda)$ in breaking the security of SMSC . $\square$

Claim 6.16. For every $\lambda \in \mathbb{N}$, any PPT adversary $\mathcal{A}$,

$$\Pr [1 \leftarrow \text{hyb}_1^A] = \Pr [1 \leftarrow \text{hyb}_{2,1}^A]$$

Proof. The proof of this claim is immediate from the distributions of hyb_1^A and $\text{hyb}_{2,1}^A$. $\square$

Lemma 6.17. If mkFHE satisfies semantic security, then for any PPT adversary $\mathcal{A}$, for any $j \in [|H|]$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$:

$$\left| \Pr [1 \leftarrow \text{hyb}_{2,j}^A(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_{2,j+1}^A(1^\lambda)] \right| \leq \text{negl}(\lambda)$$

Proof. W.l.o.g., assume that $j \in H$. Assuming that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{hyb}_{2,j}^A$ and $\text{hyb}_{2,j+1}^A$ for some $j \in [|H|]$ such that $|\Pr [1 \leftarrow \text{hyb}_{2,j}^A(1^\lambda)] - \Pr [1 \leftarrow \text{hyb}_{2,j+1}^A(1^\lambda)]| = \epsilon(\lambda)$ for some non-negligible function $\epsilon(\cdot)$, we construct an adversary $\mathcal{B}_{\text{mkFHE}}$ that breaks the semantic security of mkFHE as follows:

1. $\mathcal{A}$ submits the number of parties n and the set of honest parties H . Sample $\text{nisc.crs} \leftarrow \text{SMSC.S}(1^\lambda, n, H)$. Receive $\text{fhe.pp}, \text{fhe.pk}$ from $\mathcal{C}$. Send $(\text{nisc.crs}, \text{fhe.pp})$ to $\mathcal{A}$.
2. When $\mathcal{A}$ queries for input encoding of $(\text{id}, x_{\text{id}})$ for $\text{id} \in H$, do the following.
 - If id is the i -th entry in H and $i \neq j$, sample $(\text{fhe.pk}_{\text{id}}, \text{fhe.sk}_{\text{id}}) \leftarrow \text{mkFHE.KeyGen}(\text{fhe.pp})$. Otherwise, set $\text{fhe.pk}_{\text{id}} := \text{fhe.pk}$.
 - If $i < j$, encrypt $\text{fhe.ct}_{\text{id}} \leftarrow \text{mkFHE.Enc}(\text{fhe.pk}_{\text{id}}, 0^{|x_{\text{id}}|})$. Otherwise, if $i > j$, encrypt x_{id} as $\text{fhe.ct}_{\text{id}} \leftarrow \text{mkFHE.Enc}(\text{fhe.pk}_{\text{id}}, x_{\text{id}})$. Otherwise, send $(x_{\text{id}}, 0^{|x_{\text{id}}|})$ to $\mathcal{C}$ to receive $\text{fhe.ct}_{\text{id}}$.

- Sample $\text{nisc}.\widehat{\text{sk}}_{\text{id}} \leftarrow \text{SMSC}.\mathcal{S}(\text{id})$.

Send $\widehat{x}_{\text{id}} = (\text{fhe}.\text{pk}_{\text{id}}, \text{fhe}.\text{ct}_{\text{id}}, \text{nisc}.\widehat{\text{sk}}_{\text{id}})$ to $\mathcal{A}$.

3. $\mathcal{A}$ submits input for $\text{id} \in [n] \setminus H$ as $(\text{id}, x_{\text{id}}, \rho_{\text{id}})$. Using ρ_{id} , sample $(\text{fhe}.\text{pk}_{\text{id}}, \text{fhe}.\text{sk}_{\text{id}})$, encrypt x_{id} as $\text{fhe}.\text{ct}_{\text{id}} \leftarrow \text{mkFHE}.\text{Enc}(\text{fhe}.\text{pk}_{\text{id}}, x_{\text{id}})$, and derive the randomness used in SMSC , ρ'_{id} . Compute $\text{SMSC}.\mathcal{S}(\text{id}, \text{fhe}.\text{sk}_{\text{id}}, \rho'_{\text{id}})$.
4. $\mathcal{A}$ queries for function encoding of (id, f, I) . Sample $\text{fhe}.\widetilde{\text{ct}}_{\ell} = \text{mkFHE}.\text{Eval}(f, \{\text{fhe}.\text{pk}_{\text{id}_x}, \text{fhe}.\text{ct}_{\text{id}_x}\}_{\text{id}_x \in I})$ for $\ell \in [L]$ and sample $\widehat{g}_{\ell, \text{id}} \leftarrow \text{SMSC}.\mathcal{S}(\text{id}, g_{\ell}, I)$. Here, $g_{\ell} = \text{mkFHE}.\text{Dec}(\cdot, \dots, \cdot, \text{fhe}.\widetilde{\text{ct}}_{\ell})$. Send $\{\text{fhe}.\widetilde{\text{ct}}_{\ell}, \widehat{g}_{\ell, \text{id}}\}_{\ell \in [L]}$ to $\mathcal{A}$.
5. Output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ encrypts x_j (resp. $0^{|x_j|}$) in the experiment then $\mathcal{B}_{\text{mkFHE}}$ simulates $\text{hyb}_{2,j}^{\mathcal{A}}$ (resp. $\text{hyb}_{2,j+1}^{\mathcal{A}}$). Thus $\mathcal{B}_{\text{mkFHE}}$ has advantage $\epsilon(\lambda)$ in breaking the semantic security of mkFHE . $\square$

$\square$

Corollary 6.18. Assuming $X \in \{\text{LWE}\} \cup (\{\text{SXDH}\} \times \{O(1)\text{-LIN}, \text{sub-exp-DDH}\})$, there exists a mrNISC scheme for any n parties and all efficiently computable functionalities.

We state the following corollary of Theorems 6.5, 4.2, 6.14, and 6.3.

Corollary 6.19. Assuming the hardness of circularly secure LWE , there exists a maliciously secure mrNISC scheme for any n parties (NISC) such that:

- The size of input encodings of NISC is $|x| + \text{poly}(\lambda)$.
- The size of function encodings of NISC is $|y| \cdot \text{poly}(\lambda, |I|)$ where $y = f(\{x_{\text{id}}\}_{\text{id} \in I})$.

7 Multi-Hop zkBARG with Somewhere-Extraction

Composable or multi-hop BARGs were introduced by Devadas et al. [DGKV22] who gave a construction for rate-1 multi-hop seBARG from rate-1 seBARG. Informally, a multi-hop BARG allows for the composition of multiple BARG proofs (which could themselves be composed BARG proofs) in a succinct manner. In this section, we extend the notion of a somewhere-extractable zkBARG to the *multi-hop* setting, and describe a construction based on a rate-1 seBARG and optimally efficient NIZK. At a high-level, a multi-hop somewhere-extractable zkBARG allows succinct batching of multiple zkBARGs. The number of *hops* (i.e., number of times zkBARGs can be successively batched) can be any polynomial, and the batch size in each hop can be set arbitrarily. Each hop increases the proof size by an *additive* $\text{poly}(\lambda)$ factor. Our construction essentially combines the technique of [DGKV22]—compose proofs by creating a batch argument with the proofs as the witness—and appends a NIZK proof at the end of each level of composition with the seBARG proof as the witness. Naturally, the zero-knowledge property follows directly from that of the NIZK, but the key observation is that if the underlying seBARG scheme and the NIZK proof system are rate-1, then the composed proof is also rate-1. Further, if the seBARG scheme is somewhere-extractable, and the NIZK proof system is straight-line extractable then one can show, by the process of induction, that the resulting zkBARG scheme is also somewhere-extractable.

7.1 Definition

Formally, our syntax for multi-hop zkBARG (mzkBARG) follows that of multi-hop seBARG defined by [DGKV22]. More precisely, in addition to the algorithms of a plain zkBARG, we additionally require combining algorithm called **ComposeProof**.

Syntax. A multi-hop zkBARG scheme for an NP language $\mathcal{L}$ consists of the following polynomial time algorithms:

Setup($1^\lambda, d, (i_1, \dots, i_d)$) $\rightarrow$ **crs**. This is a probabilistic setup algorithm. It takes as input the security parameter 1^λ , the maximum number of hops $d \in [2^\lambda]$ (i.e., the number of batch-compositions), and a sequence of d extraction indices⁵ $I = (i_1, \dots, i_d) \in [2^\lambda]^d$. It outputs the crs **crs** which consists of d strings $\text{crs} = (\text{crs}_1, \dots, \text{crs}_d)$.

Prove($\text{crs}_1, (x_1, \dots, x_k), (w_1, \dots, w_k)$) $\rightarrow$ π . The poly-time prover algorithm that takes as input crs_1 (which is the first string in **crs**), any (unbounded) number of instance-witness pairs of arbitrary size⁶. It runs in time $\text{poly}(\lambda, (|x_i|, |w_i|)_{i \in [k]})$ and outputs a proof π .

ComposeProof((crs_i) $_{i \in [d']}$, ($X^{(1)}, \dots, X^{(\ell)}$), ($\pi^{(1)}, \dots, \pi^{(\ell)}$)) $\rightarrow$ π . This proof combiner is a poly-time algorithm that takes as input the (crs_i) $_{i \in [d']}$, a sequence of arbitrarily many *instance-trees* $X^{(1)}, \dots, X^{(\ell)}$ (as defined below), of maximal depth $d' - 1$, and corresponding mzkBARG proofs $\pi^{(1)}, \dots, \pi^{(\ell)}$. It outputs a (combined) mzkBARG proof π .

Verify((crs_i) $_{i \in [d']}$, X, π) $\rightarrow \{0, 1\}$. The poly-time verification algorithm takes as input (crs_i) $_{i \in [d']}$, an instance-tree X of depth d' , and a (combined) proof π , and outputs 0/1 (corresponding to reject or accept).

Definition 7.1 (Instance Tree). An instance-tree X is a tree of varying arity, where each leaf node v is associated with an instance x_v , and each intermediate node u corresponds to a mzkBARG proof that certifies the validity of the sub-tree rooted at u (these mzkBARG proofs are not included in X).

Following the notation of Devadas et al. [DGKV22], a tree T is comprised of leaves, denoted by v , associated with a parameter $n_v = n_v(\lambda)$. A *poly-size tree*, is a tree T with at most $\text{poly}(\lambda)$ nodes and $n_v = n_v(\lambda) \leq \text{poly}(\lambda)$ for every leaf v . For any tree T , we denote by $\text{path}(T)$, the set of all the possible paths from the root to a leaf in T . An instance-tree X is said to be *consistent* with T if X has the exact same tree structure as T and for each leaf $v \in T$ the instance (leaf) x_v in X is of size n_v . For any instance-tree X that is consistent with some tree T and any $(i_1, \dots, i_d) \in \text{path}(T)$ we let $X_{i_1, \dots, i_d}$ denote the instance (in the leaf) corresponding to the path $(i_1, \dots, i_d)$. Finally, for a collection of instance-trees $X^{(1)}, \dots, X^{(\ell)}$ we denote by $(X^{(1)}, \dots, X^{(\ell)})$ the instance-tree that combines all the ℓ instance-trees $X^{(1)}, \dots, X^{(\ell)}$ by adding a root with arity ℓ , whose i 'th child is the root of $X^{(i)}$.

Definition 7.2. A rate-1 multi-hop somewhere-extractable zkBARG scheme $\text{mzkBARG} = (\text{Setup}, \text{Prove}, \text{ComposeProof}, \text{Verify})$ for an NP language $\mathcal{L}$ is required to satisfy the following properties:

⁵The j 'th extraction index i_j is interpreted as saying that, from an accepting zkBARG proof π created via j -compositions, we can efficiently extract the i_j 'th witness, which is itself an accepting zkBARG proof π' created via $(j - 1)$ -compositions.

⁶Note that neither the batch size nor the instance size are fixed at setup time in the multi-hop setting

Efficiency. For every $i \in [d]$, the size of crs_i is at most $\text{poly}(\lambda)$, and the size of a (combined) proof corresponding to an instance-tree X of depth d' is at most $m + d' \cdot \text{poly}(\lambda, \log |X|)$, where m is the maximal witness length of all the leaf instances in X .

Completeness. For any $\lambda \in \mathbb{N}$, any $d \in [2^\lambda]$, any instance-tree X of size $\leq 2^\lambda$ and depth $d' \leq d$, and any corresponding valid witness W ,

$$\Pr \left[\text{Verify}((\text{crs}_i)_{i \in [d]}, X, \pi) = 1 \quad : \quad \begin{array}{l} (\text{crs}_i)_{i \in [d]} \leftarrow \text{Setup}(1^\lambda, d, (i_1, \dots, i_d)), \\ \pi \leftarrow \text{Compose}((\text{crs}_i)_{i \in [d]}, X, W) \end{array} \right] = 1$$

where $\text{Compose}((\text{crs}_i)_{i \in [d]}, X, W)$ is defined inductively on d' as follows:

- If $d' = 1$ then parse $X = (x_1, \dots, x_\ell)$ and $W = (w_1, \dots, w_\ell)$ and output $\text{Compose}(\text{crs}_1, X, W) = \text{Prove}(\text{crs}_1, x_1, \dots, x_\ell, w_1, \dots, w_\ell)$.
- If $d' > 1$ then parse $X = (X^{(1)}, \dots, X^{(\ell)})$ and $W = (W^{(1)}, \dots, W^{(\ell)})$, where $W^{(i)}$ is the witness corresponding to the sub-tree instance $X^{(i)}$. Denote by d_i the depth of $X^{(i)}$. For every $i \in [\ell]$ compute by induction

$$\pi^{(i)} = \text{Compose}((\text{crs}_j)_{j \in [d_i]}, X^{(i)}, W^{(i)})$$

and output

$$\pi = \text{ComposeProof}((\text{crs}_i)_{i \in [d]}, X^{(1)}, \dots, X^{(\ell)}, \pi^{(1)}, \dots, \pi^{(\ell)}).$$

Index hiding. For any poly-size adversary $\mathcal{A}$, any polynomial $d = \text{poly}(\lambda)$, any poly-size tree T of depth $d' \leq d$, and any sets of indices $I_0 = (i_{0,1}, \dots, i_{0,d})$, $I_1 = (i_{1,1}, \dots, i_{1,d}) \in \text{path}(T)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\mathcal{A}(\text{crs}) = b \quad : \quad \begin{array}{l} b \leftarrow \{0, 1\}, \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, d, I_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Somewhere argument of knowledge. There exists a stateful PPT extractor $\mathcal{E}$ such that for any poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for any polynomial $d = \text{poly}(\lambda)$, any poly-size tree T of depth $d' \leq d$, and any set of indices $(i_1, \dots, i_{d'}) \in \text{path}(T)$ and $i_{d'+1}, \dots, i_d \in [2^\lambda]$, for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, X, \pi) = 1 \\ \wedge X \text{ is consistent with } T \\ \wedge W^* \text{ is not a valid witness for } X_{i_1, \dots, i_{d'}} \in \mathcal{L} \end{array} \quad : \quad \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ (X, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ W^* \leftarrow \mathcal{E}(\text{td}, X, \pi) \end{array} \right] \leq \text{negl}(\lambda).$$

Zero-knowledge. There exist PPT simulators $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3)$ sharing state-information for any poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for any polynomial $d = \text{poly}(\lambda)$, any poly-size tree T of depth $d' \leq d$, and any set of indices $(i_1, \dots, i_{d'}) \in \text{path}(T)$ and $i_{d'+1}, \dots, i_d \in [2^\lambda]$, for every $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[1 \leftarrow \mathcal{A}^{\text{Prove}(\text{crs}, \cdot, \cdot), \text{ComposeProof}(\text{crs}, \cdot, \cdot)}(\text{crs}) \quad : \quad \text{crs} \leftarrow \text{Setup}(1^\lambda, d, (i_1, \dots, i_d)) \right] - \Pr \left[1 \leftarrow \mathcal{A}^{\mathcal{O}^{\mathcal{S}_2}(\text{crs}_1, \cdot, \cdot), \mathcal{O}^{\mathcal{S}_3}(\text{crs}, \cdot, \cdot)}(\text{crs}) \quad : \quad \text{crs} \leftarrow \mathcal{S}_1(1^\lambda, d) \right] \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}^{\mathcal{S}_2}((x_1, \dots, x_k), (w_1, \dots, w_k))$ outputs $\mathcal{S}_2(x_1, \dots, x_k)$ if for every $i \in [k]$, w_i is a valid witness for $x_i \in \mathcal{L}$ and $\perp$ otherwise; and $\mathcal{O}^{\mathcal{S}_3}((X^{(1)}, \dots, X^{(\ell)}), (\pi^{(1)}, \dots, \pi^{(\ell)}))$ outputs $\mathcal{S}_3(X^{(1)}, \dots, X^{(\ell)})$ if for every $i \in [k]$, $\text{Verify}((\text{crs}_i)_{i \in [d_i]}, X^{(i)}, \pi^{(i)}) = 1$ and $\perp$ otherwise;

7.2 Construction

We now give a construction for rate-1 multi-hop somewhere-extractable zkBARG from any rate-1 seBARG and an optimally efficient NIZK. Our multi-hop zkBARG construction, denoted **mzkBARG**, for some NP language $\mathcal{L}$ uses the single-hop seBARG primitive, denoted **s-seBARG**, for the language $\mathcal{L}'$ containing $\mathcal{L}$. Essentially, an **s-seBARG** is an seBARG where the number of instances and the input length are not a priori bounded. The existence of **s-seBARG** was shown by [DGKV22] and we recall their construction next.

Single-hop seBARG. A single-hop seBARG is a seBARG where the number of batched instances k , and the input length n can be determined when batching the arguments. The construction of **s-seBARG** from a regular seBARG follows from the observation made in [DGKV22] that (i) one can always pad inputs of unequal length; and more importantly, (ii) it is possible to run the setup algorithm for zkBARG, which grows only polylogarithmically in k and n , once for each $k, n \in \{2^i\}_{i \in [\lambda]}$ and allow the prover to choose the appropriate **crs** based on the actual k and n .

Multi-hop zkBARG. We will now use an **s-seBARG** for the language $\mathcal{L}'$ and NIZK for the language $\mathcal{L}''$. Here, any instance in $\mathcal{L}'$ is an instance-tree X of some depth d' , along with $(\text{crs}_i)_{i \in [d']}$ (if $d' = 0$ then there is no **crs** associated with the instance) and a valid witness is a valid aggregated proof corresponding to $(\text{crs}_i)_{i \in [d']}$ (where if $d' = 0$ then a valid proof is simply a valid witness corresponding to $\mathcal{L}$); and any instance of $\mathcal{L}''$ is the input to the **s-seBARG.Verify** circuit, and the witness is the corresponding **s-seBARG** proof.

Setup $(1^\lambda, d, I) \rightarrow \text{crs}$. Parse $I = (i_1, \dots, i_d)$ and for every $j \in [d]$, sample $\text{barg.crs}_j \leftarrow \text{s-seBARG.Setup}(1^\lambda, i_j)$ and $\text{nizk.crs}_j \leftarrow \text{NIZK.Setup}(1^\lambda)$. Finally, output $\text{crs} = (\text{nizk.crs}_j, \text{barg.crs}_j)_{j \in [d]}$.

Prove $(\text{crs}_1, (x_1, \dots, x_k), (w_1, \dots, w_k)) \rightarrow \pi$. Parse crs_1 as nizk.crs_1 and barg.crs_1 . Compute

$$\text{barg.}\pi \leftarrow \text{s-seBARG.Prove}(\text{barg.crs}_1, (x_1, \dots, x_k), (w_1, \dots, w_k))$$

and output,

$$\text{nizk.}\pi \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_1, (\text{barg.crs}_1, (x_1, \dots, x_k)), \text{barg.}\pi)$$

as the proof π .

ComposeProof $(\text{crs}, (X^{(1)}, \dots, X^{(\ell)}), (\pi^{(1)}, \dots, \pi^{(\ell)})) \rightarrow \pi$. For every $i \in [\ell]$, denote by d_i the depth of $X^{(i)}$ and let $d' = \max\{d_i\} + 1$. If $d' > d$ then the abort. Otherwise, parse $\text{crs} = ((\text{nizk.crs}_1, \text{barg.crs}_1), \dots, (\text{nizk.crs}_{d'}, \text{barg.crs}_{d'}, \dots))$ and compute

$$\text{barg.}\pi \leftarrow \text{s-seBARG.Prove}\left(\text{barg.crs}_{d'}, \left((\text{barg.crs}_j)_{j \in [d_i]}, X^{(i)}\right)_{i \in [\ell]}, \left(\pi^{(i)}\right)_{i \in [\ell]}\right)$$

where a valid witness corresponding to $((\text{barg.crs}_j)_{j \in [d_i]}, X^{(i)}) \in \mathcal{L}'$ is $\pi^{(i)}$ such that

$$\text{Verify}\left((\text{barg.crs}_j)_{j \in [d_i]}, X^{(i)}, \pi^{(i)}\right) = 1$$

where if $d_i = 0$ then $\text{Verify}(\perp, X^{(i)}, \pi^{(i)}) = 1$ if and only if $\pi^{(i)}$ is a valid witness for $X^{(i)} \in \mathcal{L}$.

Finally, output

$$\text{nizk}.\pi \leftarrow \text{NIZK.Prove} \left(\text{nizk.crs}_{d'}, \left(\text{barg.crs}_{d'}, \left((\text{barg.crs}_j)_{j \in [d_i]}, X^{(i)} \right)_{i \in [\ell]} \right), \text{barg}.\pi \right)$$

as the proof π .

$\text{Verify}(\text{crs}, X, \pi) \rightarrow 0/1$. Parsing $X = (X^{(1)}, \dots, X^{(\ell)})$, let $d_i \in [d]$ be the depth of the instance-tree $X^{(i)}$. If $d' \neq \max\{d_i\}_{i \in [\ell]} + 1$ then output 0. Otherwise parse $\text{crs} = ((\text{nizk.crs}_1, \text{barg.crs}_1), \dots, (\text{nizk.crs}_{d'}, \text{barg.crs}_{d'}, \dots))$ and then output

$$\text{NIZK.Verify} \left(\text{nizk.crs}_{d'}, \left(\text{barg.crs}_{d'}, \left((\text{barg.crs}_j)_{j \in [d_i]}, X^{(i)} \right)_{i \in [\ell]} \right), \pi \right)$$

Theorem 7.3. If s-seBARG is a rate-1 single-hop seBARG and NIZK is an optimally efficient NIZK scheme (with straight-line extraction) then Construction 7.2 is a rate-1 multi-hop somewhere extractable zkBARG scheme.

Lemma 7.4 (Completeness). If s-seBARG and NIZK are complete, then Construction 7.2 is complete.

Proof. This follows directly from the construction. $\square$

Lemma 7.5 (Efficiency). If s-seBARG and NIZK are rate-1, then Construction 7.2 is rate-1.

Proof. This follows directly from the construction. $\square$

7.3 Security Analysis

Lemma 7.6 (Index hiding). If s-seBARG is index hiding, then Construction 7.2 is index hiding.

Proof. This follows directly from the construction. $\square$

Lemma 7.7 (Somewhere argument of knowledge). If s-seBARG is somewhere argument of knowledge and NIZK is straight-line extractable, then Construction 7.2 is somewhere argument of knowledge.

Proof. Let $\text{s-seBARG}.\mathcal{E}$ (resp. $\text{NIZK}.\mathcal{E}$) denote the PPT extractor corresponding to the underlying s-seBARG (resp. NIZK) scheme. We define an extractor $\mathcal{E}$ that given (td, X, π) does the following:

1. Parse $\text{td} = ((\text{nizk.td}_1, \text{barg.td}_1), \dots, (\text{nizk.td}_d, \text{barg.td}_d))$ and for $j \in [d]$, let each barg.td_j consist of index i_j . We also assume w.l.o.g. that each nizk.td_j implicitly contains $\text{nizk.crs}_{i < j}$ (similarly for barg.td_j).
2. Parse $X = (X^{(1)}, \dots, X^{(\ell)})$, and let d' denote the depth of X .
3. For every $j \in [d']$ we denote by d_j the depth of $X^{(i_{d'}, \dots, i_{d'-j+1})}$, the subtree of X obtained by going down from the root on the path $(i_{d'}, \dots, i_{d'-j+1})$. Let $j^* \in [d']$ be the smallest index such that $d_j = 0$ for every $j \geq j^*$ (i.e., j^* is the length of the path $(i_{d'}, \dots, i_1)$ in X until we reach a leaf).

4. Run the NIZK extractor to compute

$$\text{barg}.\pi^{(i_{d'})} \leftarrow \text{NIZK}.\mathcal{E} \left(\text{nizk}.\text{td}_{d'}, \left(X^{(1)}, \dots, X^{(\ell)} \right), \pi \right).$$

5. Then, run the s-seBARG extractor to compute

$$\pi^{(i_{d'})} \leftarrow \text{s-seBARG}.\mathcal{E} \left(\text{barg}.\text{td}_{d'}, \left(X^{(1)}, \dots, X^{(\ell)} \right), \text{barg}.\pi^{(i_{d'})} \right).$$

Intuitively, if π is a valid mzkBARG proof for X then with overwhelming probability $\pi^{(i_{d'})}$ is a valid mzkBARG proof for the instance-tree $X^{(i_{d'})}$.

6. For every $j \in [j^* - 1]$ parse $X^{(i_{d'}, \dots, i_{d'-j+1})} = (X^{(1)}, \dots, X^{(\ell_j)})$, and then inductively compute (starting with $j = 1$)

$$\text{barg}.\pi^{(i_{d'-j})} \leftarrow \text{NIZK}.\mathcal{E} \left(\text{nizk}.\text{td}_{d_j}, \left(X^{(1)}, \dots, X^{(\ell_j)} \right), \pi^{(i_{d'}, \dots, i_{d'-j+1})} \right).$$

and,

$$\pi^{(i_{d'}, \dots, i_{d'-j})} \leftarrow \text{s-seBARG}.\mathcal{E} \left(\text{barg}.\text{td}_{d_j}, \left(X^{(1)}, \dots, X^{(\ell_j)} \right), \text{barg}.\pi^{(i_{d'}, \dots, i_{d'-j})} \right)$$

7. Output $W^* = \pi^{(i_{d'}, \dots, i_{d'-j^*+1})}$.

Now, fix any PPT adversary $\mathcal{A}$ and any polynomial $d = \text{poly}(\lambda)$. We will argue by induction that for every $d' \leq d$ there exists a negligible function $\mu_{d'}$, such that for any poly-size tree T of depth d' , and any set of indices $(i_1, \dots, i_{d'}) \in \text{path}(T)$ and $i_{d'+1}, \dots, i_d \in [2^\lambda]$ it holds that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}((\text{crs}_j)_{j \in [d']}, X, \pi) = 1 \\ \wedge X \text{ is consistent with } T \\ \wedge W^* \text{ is not a valid witness for } X_{i_1, \dots, i_{d'}} \in \mathcal{L} \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ (X, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ W^* \leftarrow \mathcal{E}(\text{td}, X, \pi) \end{array} \right] \leq \mu_{d'}(\lambda). \quad (14)$$

Base case: $d' = 1$: Let us define the following two types of possible PPT adversaries:

Type 1. For this type of adversary, the following probability is non-negligible,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}_1, (x_1, \dots, x_k), \pi) = 1 \wedge \\ \text{Verify}(\text{crs}_1, (x_1, \dots, x_k), \text{barg}.\pi) \neq 1 \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ ((x_1, \dots, x_k), \pi) \leftarrow \mathcal{A}(\text{crs}) \\ \text{barg}.\pi \leftarrow \text{NIZK}.\mathcal{E}(\text{nizk}.\text{td}_1, (x_1, \dots, x_k), \pi) \\ w^* \leftarrow \text{s-seBARG}.\mathcal{E}(\text{td}_1, (x_1, \dots, x_k), \text{barg}.\pi) \\ \text{parse crs} = (\text{crs}_1, \dots, \text{crs}_d) \end{array} \right] \quad (15)$$

Then, we give a reduction $\mathcal{B}_{\text{NIZK}.\mathcal{E}}$ that breaks straight-line extraction of NIZK as follows:

1. On receiving $\overline{\text{crs}}$ from the challenger, $\mathcal{B}_{\text{NIZK}.\mathcal{E}}$ sets $\text{nizk}.\text{crs}_1 = \overline{\text{crs}}$.
2. It then generates the rest of the crs honestly and sends it to the type 1 adversary.
3. Finally it outputs whatever the adversary outputs.

It is clear that if the type 1 adversary has non-negligible probability for (15), then $\mathcal{B}_{\text{NIZK}.\mathcal{E}}$ breaks straight-line extraction of NIZK with the same probability.

Type 2. For this type of adversary, the following probability is non-negligible,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}_1, (x_1, \dots, x_k), \pi) = 1 \wedge \\ \text{Verify}(\text{crs}_1, (x_1, \dots, x_k), \text{barg}.\pi) = 1 \\ \wedge w^* \text{ is not a valid witness for } x_{i_1} \in \mathcal{L} \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ ((x_1, \dots, x_k), \pi) \leftarrow \mathcal{A}(\text{crs}) \\ \text{barg}.\pi \leftarrow \text{NIZK}.\mathcal{E}(\text{nizk}.\text{td}_1, (x_1, \dots, x_k), \pi) \\ w^* \leftarrow \text{s-seBARG}.\mathcal{E}(\text{td}_1, (x_1, \dots, x_k), \text{barg}.\pi) \\ \text{parse crs} = (\text{crs}_1, \dots, \text{crs}_d) \end{array} \right] \quad (16)$$

Then, we give a reduction $\mathcal{B}_{\text{seBARG}.\mathcal{E}}$ that breaks somewhere argument of knowledge of seBARG as follows:

1. On receiving $\overline{\text{crs}}$ from the challenger, $\mathcal{B}_{\text{seBARG}.\mathcal{E}}$ sets $\text{barg}.\text{crs}_1 = \overline{\text{crs}}$.
2. It then generates the rest of the crs honestly and sends it to the type 2 adversary.
3. On receiving $((x_1, \dots, x_k), \pi)$ from the adversary, it runs the NIZK extractor, to obtain $\text{barg}.\pi$.
4. It then outputs $((x_1, \dots, x_k), \text{barg}.\pi)$.

It is clear that if the type 2 adversary has non-negligible probability for (16), then $\mathcal{B}_{\text{seBARG}.\mathcal{E}}$ breaks somewhere argument of knowledge of the underlying seBARG scheme with the same probability.

Intermediate step: Before proceeding with the induction step, we must first show that there exists a negligible function ν such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}((\text{crs}_j)_{j \in [d']}, X, \pi) = 1 \wedge \\ X \text{ is consistent with } T \wedge \\ \left(\begin{array}{l} \text{Verify}((\text{crs}_j)_{j \in [d_1]}, X^{(i_{d'})}, \text{barg}.\pi^{(i_{d'})}) \neq 1 \\ \vee \text{Verify}((\text{crs}_j)_{j \in [d_1]}, X^{(i_{d'})}, \pi^{(i_{d'})}) \neq 1 \end{array} \right) \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ (X, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ \text{barg}.\pi^{(i_{d'})} \leftarrow \text{NIZK}.\mathcal{E}(\text{nizk}.\text{td}_{d'}, X, \pi) \\ \pi^{(i_{d'})} \leftarrow \text{s-seBARG}.\mathcal{E}(\text{td}_{d'}, X, \text{barg}.\pi^{(i_{d'})}) \\ \text{parse crs} = (\text{crs}_1, \dots, \text{crs}_d) \end{array} \right] \leq \nu(\lambda). \quad (17)$$

Let us again define two types of possible PPT adversaries:

Type 1. For this type of adversary, the following probability is non-negligible,

$$\Pr \left[\begin{array}{l} \text{Verify}((\text{crs}_j)_{j \in [d']}, X, \pi) = 1 \wedge \\ X \text{ is consistent with } T \wedge \\ \text{Verify}((\text{crs}_j)_{j \in [d_1]}, X^{(i_{d'})}, \text{barg}.\pi^{(i_{d'})}) \neq 1 \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ (X, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ \text{barg}.\pi^{(i_{d'})} \leftarrow \text{NIZK}.\mathcal{E}(\text{nizk}.\text{td}_{d'}, X, \pi) \\ \pi^{(i_{d'})} \leftarrow \text{s-seBARG}.\mathcal{E}(\text{td}_{d'}, X, \text{barg}.\pi^{(i_{d'})}) \\ \text{parse crs} = (\text{crs}_1, \dots, \text{crs}_d) \end{array} \right] \quad (18)$$

Then, we give a reduction $\mathcal{B}_{\text{NIZK}.\mathcal{E}}$ that breaks straight-line extraction of NIZK as follows:

1. On receiving $\overline{\text{crs}}$ from the challenger, $\mathcal{B}_{\text{NIZK}.\mathcal{E}}$ sets $\text{nizk}.\text{crs}_{d'} = \overline{\text{crs}}$.

2. It then generates the rest of the crs honestly and sends it to the type 1 adversary.
3. Finally it outputs whatever the adversary outputs.

It is clear that if the type 1 adversary has non-negligible probability for (18), then $\mathcal{B}_{\text{NIZK}, \mathcal{E}}$ breaks straight-line extraction of NIZK with the same probability.

Type 2. For this type of adversary, the following probability is non-negligible,

$$\Pr \left[\begin{array}{l} \text{Verify}((\text{crs}_j)_{j \in [d']}, X, \pi) = 1 \wedge \\ X \text{ is consistent with } T \wedge \\ \text{Verify}((\text{crs}_j)_{j \in [d_1]}, X^{(i_{d'})}, \text{barg}.\pi^{(i_{d'})}) \neq 1 \\ \wedge \text{Verify}((\text{crs}_j)_{j \in [d_1]}, X^{(i_{d'})}, \pi^{(i_{d'})}) \neq 1 \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ (X, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ \text{barg}.\pi^{(i_{d'})} \leftarrow \text{NIZK}.\mathcal{E}(\text{nizk}.\text{td}_{d'}, X, \pi) \\ \pi^{(i_{d'})} \leftarrow \text{s-seBARG}.\mathcal{E}(\text{td}_{d'}, X, \text{barg}.\pi^{(i_{d'})}) \\ \text{parse crs} = (\text{crs}_1, \dots, \text{crs}_d) \end{array} \right] \quad (19)$$

Then, we give a reduction $\mathcal{B}_{\text{seBARG}, \mathcal{E}}$ that breaks somewhere argument of knowledge of seBARG as follows:

1. On receiving $\overline{\text{crs}}$ from the challenger, $\mathcal{B}_{\text{seBARG}, \mathcal{E}}$ sets $\text{barg}.\text{crs}_{d'} = \overline{\text{crs}}$.
2. It then generates the rest of the crs honestly and sends it to the type 2 adversary.
3. On receiving (X, π) from the adversary, it runs the NIZK extractor, to obtain $\text{barg}.\pi^{(i_{d'})}$.
4. It then outputs $(X, \text{barg}.\pi^{(i_{d'})})$.

It is clear that if the type 2 adversary has non-negligible probability for (19), then $\mathcal{B}_{\text{seBARG}, \mathcal{E}}$ breaks somewhere argument of knowledge of the underlying seBARG scheme with the same probability.

Induction step: Now, suppose that Equation (14) holds for every $j < d'$ and we prove that it holds for d' . To that end, consider the poly-size adversary $\mathcal{A}'$ that given crs does the following:

1. Parse $\text{crs} = ((\text{nizk}.\text{crs}_1, \text{barg}.\text{crs}_1), \dots, (\text{nizk}.\text{crs}_d, \text{barg}.\text{crs}_d))$.
2. Generate $(\text{barg}.\text{crs}^*, \text{barg}.\text{td}^*) \leftarrow \text{s-seBARG}.\mathcal{E}(1^\lambda, i_{d'})$ and $(\text{nizk}.\text{crs}^*, \text{nizk}.\text{td}^*) \leftarrow \text{NIZK}.\mathcal{E}(1^\lambda)$.
3. Let $\text{crs}^* = ((\text{nizk}.\text{crs}_1, \text{barg}.\text{crs}_1), \dots, (\text{nizk}.\text{crs}_{d'-1}, \text{barg}.\text{crs}_{d'-1}), (\text{nizk}.\text{crs}^*, \text{barg}.\text{crs}^*), (\text{nizk}.\text{crs}_{d'+1}, \text{barg}.\text{crs}_{d'+1}), \dots, (\text{nizk}.\text{crs}_d, \text{barg}.\text{crs}_d))$.
4. Compute $(X, \pi) \leftarrow \mathcal{A}(\text{crs}^*)$.
5. Compute $\text{barg}.\pi^{(i_{d'})} \leftarrow \text{NIZK}.\mathcal{E}(\text{nizk}.\text{td}^*, X, \pi)$ and $\pi^{(i_{d'})} \leftarrow \text{s-seBARG}.\mathcal{E}(\text{td}_{d'}, X, \text{barg}.\pi^{(i_{d'})})$.
6. Output $(X^{(i_{d'})}, \pi^{(i_{d'})})$.

By the induction hypothesis there exists a negligible function μ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}((\text{crs}_j)_{j \in [d_1]}, X^{(i_{d'})}, \pi^{(i_{d'})}) = 1 \\ \wedge X^{(i_{d'})} \text{ is consistent with } T^{(i_{d'})} \\ \wedge \text{Verify}((\text{crs}_j)_{j \in [d_2]}, X^{(i_{d'}, i_{d'-1})}, \pi^{(i_{d'}, i_{d'-1})}) \neq 1 \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{E}(1^\lambda, d, (i_1, \dots, i_d)) \\ (X^{(i_{d'})}, \pi^{(i_{d'})}) \leftarrow \mathcal{A}'(\text{crs}) \\ \pi^{(i_{d'}, i_{d'-1})} \leftarrow \mathcal{E}(\text{td}^*, X^{(i_{d'})}, \pi^{(i_{d'})}) \end{array} \right] \leq \mu(\lambda).$$

This, together with Equation (17), implies the induction step (i.e., Equation (14)). $\square$

Lemma 7.8 (Zero-knowledge). If NIZK satisfies zero-knowledge, then Construction 7.2 satisfies

Proof. We define the following hybrids:

hyb_0^A This is the original zero-knowledge experiment described in Definition 7.2.

hyb_1^A This is same as hyb_0 except that we use $\text{NIZK}.\mathcal{S}$ to simulate NIZK instantiation. This is the description of the simulator $\mathcal{S}$.

Let $\text{hyb}_i^A(1^\lambda)$ denote the output of the experiment hyb_i^A .

Claim 7.9. If NIZK satisfies zero-knowledge, then there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$:

$$\left| \Pr \left[1 \leftarrow \text{hyb}_0^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{hyb}_1^A(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

Proof. This follows directly from the zero-knowledge property of NIZK. $\square$

$\square$

Corollary 7.10. Assuming $X \in \{\text{LWE}, O(1)\text{-LIN}, \text{sub-exp-DDH}\}$, there exists a rate-1 multi-hop somewhere-extractable zkBARG scheme.

8 Publicly-Verifiable Succinct Delegation of Committed Programs

In this section, we show that reusable encrypted RAM delegation implies a stronger variant of publicly-verifiable succinct delegation of committed programs introduced by Ghosal et al. [GSW23] for fixed programs. The problem of sDel considers the scenario where a client delegates the computation of its program to a server, so that a third-party user, holding a commitment to the client's program, can later request the server for the output said program on some public input. Importantly, the third-party user need not trust the server is additionally required to provide a proof of correct computation with respect to the committed program.

As we explain below, our sDel construction satisfies a stronger notion soundness as well as straight-line extraction. The stronger soundness property is essentially the no-equivocation property for rEDel proofs, namely that an adversary cannot furnish proofs of contradictory outputs for the same committed program. Moreover, our construction directly yields the zero-knowledge version of sDel, called zksDel where the verifier does not learn anything about the committed program.

8.1 Definition

We recall the syntax of `zksDel` given in [GSW23], with a mild modification. Specifically, instead of a “program authoring” algorithm `ProgAuth` that *outputs* the program and its commitment, we consider the program to be the input to an encoding algorithm that outputs the commitment and some auxiliary information. This separates the logic of the program from the algorithm generating its commitment and allows us to define a delegation scheme with respect to a program.

Syntax. A `zksDel` scheme with respect to a program $P \in \{0, 1\}^m$ consists of the following polynomial time algorithms:

`Setup`($1^\lambda, n, 1^m, T$) $\rightarrow$ `crs`. This is a probabilistic setup algorithm. It takes as input the security parameter 1^λ , the input length n to the program, the size m of the program, the maximum runtime T of the program and outputs a common reference string `crs`.

`ProgEnc`(`crs`, P) $\rightarrow$ (c_P , `aux`). The poly-time program encoding algorithm that takes as input `crs` and the program $P \in \{0, 1\}^m$, $m \leq 2^\lambda \in \mathbb{N}$. It outputs a commitment c_P to P and some auxiliary information `aux`, which includes P itself.

`Prove`(`crs`, x , `aux`) $\rightarrow$ (y , π). The poly-time prover algorithm that takes as input `crs`, some program input $x \in \{0, 1\}^n$ and some auxiliary information `aux`. It outputs a value $y \in \{0, 1\}$ and a proof π .

`Verify`(`crs`, x , y , c_P , π) $\rightarrow \{0, 1\}$. The poly-time verification algorithm takes as input `crs`, some program input x , a program output y , commitment c_P and the proof π . It outputs either 1 (accept) or 0 (reject).

Definition 8.1. A `zksDel` scheme $\text{zksDel} = (\text{Setup}, \text{ProgEnc}, \text{Prove}, \text{Verify})$ is required to satisfy the following properties:

Completeness. For any $\lambda, n, m \in \mathbb{N}$, for all $x \in \{0, 1\}^n$ and for all $P \in \{0, 1\}^m$ such that $n, m < 2^\lambda$,

$$\Pr \left[P(x) = y \wedge \text{Verify}(\text{crs}, x, y, c_P, \pi) = 1 \quad : \quad \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, n, 1^m, T), \\ (c_P, \text{aux}) \leftarrow \text{ProgEnc}(\text{crs}, P), \\ (y, \pi) \leftarrow \text{Prove}(\text{crs}, x, \text{aux}) \end{array} \right] = 1$$

Efficiency. For the completeness experiment above,

- Setup’s running time and the size of the `crs` are $\text{poly}(\lambda, \log T, \log n, m)$.
- The size of c_P is $\text{poly}(\lambda, m)$.
- Prover’s running time is $\text{poly}(\lambda, T, n, m)$.
- The size of π is $\text{poly}(\lambda, \log T, \log n, \log m)$.
- The Verifier’s running time is $\text{poly}(\lambda, \log T, \log n, m)$.

$\text{Expt}_0^{\mathcal{C}, \mathcal{A}}(1^\lambda, n, 1^m, T)$. This is the Real experiment parameterized by an honest challenger $\mathcal{C}$. $\mathcal{A}$ receives $\text{crs} \leftarrow \text{Setup}(1^\lambda, n, 1^m, T)$ from $\mathcal{C}$. $\mathcal{A}$ makes the following queries in an adaptive manner. After this, $\mathcal{A}$ outputs guess b' . Output b' .

- **PROGRAM ENCODING:** $\mathcal{A}$ sends program P of length m and runtime T , and receives (c_P, i) where $(c_P, \text{aux}_i) \leftarrow \text{ProgEnc}(\text{crs}, P)$ and i is an index that after each query of this type is incremented.
- **PROVE:** $\mathcal{A}$ sends (x, i) and receives (y, π) where $(y, \pi) \leftarrow \text{Prove}(\text{crs}, x, \text{aux}_i)$ and $y = P(x)$ (for P corresponding to i).

$\text{Expt}_1^{\mathcal{S}, \mathcal{A}}(1^\lambda, n, 1^m, T)$. This is the Ideal experiment parameterized by a stateful simulator $\mathcal{S}$. $\mathcal{A}$ receives $\text{crs} \leftarrow \mathcal{S}(1^\lambda, n, 1^m, T)$ from $\mathcal{C}$. $\mathcal{A}$ makes the following queries in an adaptive manner. After this, $\mathcal{A}$ outputs guess b' . Output b' .

- **PROGRAM ENCODING:** $\mathcal{A}$ sends program P of length m and runtime T , and receives (c_P, i) where $c_P \leftarrow \mathcal{S}(\text{crs})$ and i is an index that after each query of this type is incremented.
- **PROVE:** $\mathcal{A}$ sends (x, i) , and receives (y, π) where $(y, \pi) \leftarrow \mathcal{S}(\text{crs}, x)$ and $y = P(x)$ (for P corresponding to i), otherwise, receives $\perp$.

Figure 5: Real and Ideal experiments for the zero-knowledge property of the succinct delegation of scheme for committed programs.

Zero-knowledge. There exists a poly-time stateful simulator $\mathcal{S}$ such that for any poly-time adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n, m \in \mathbb{N}$ with $n, m \leq 2^\lambda$:

$$\left| \Pr \left[1 \leftarrow \text{Expt}_0^{\mathcal{C}, \mathcal{A}}(1^\lambda, 1^n, 1^m) \right] - \Pr \left[1 \leftarrow \text{Expt}_1^{\mathcal{S}, \mathcal{A}}(1^\lambda, 1^n, 1^m) \right] \right| \leq \text{negl}(\lambda)$$

where definitions of $\text{Expt}_0^{\mathcal{C}, \mathcal{A}}$ and $\text{Expt}_1^{\mathcal{S}, \mathcal{A}}$ are provided in Figure 5.

Straight-line Extraction. There exists a PPT extractor $\mathcal{E}$ such that for any stateful poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for $\lambda \in \mathbb{N}$

$$\Pr \left[\begin{array}{l} P^*(x) \neq y \wedge \text{Verify}(\text{crs}, x, y, c_P, \pi) = 1 \\ (n, m, T) \leftarrow \mathcal{A}(1^\lambda) \\ \text{crs} \leftarrow \mathcal{E}(1^\lambda, n, 1^m, T) \\ (c_P, x, y, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ P^* \leftarrow \mathcal{E}(c_P) \end{array} \right] \leq \text{negl}(\lambda).$$

such that $\text{crs}_1 \leftarrow \mathcal{E}(1^\lambda, n, 1^m, T)$ and $\text{crs}_2 \leftarrow \text{Setup}(1^\lambda, n, 1^m, T)$ are indistinguishable.

Strong Soundness. For any stateful PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, x, 0, c_P, \pi^{(0)}) = 1 \\ \wedge \text{Verify}(\text{crs}, x, 1, c_P, \pi^{(1)}) = 1 \end{array} : \begin{array}{l} (n, m, T) \leftarrow \mathcal{A}(1^\lambda), \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, n_{\text{pb}}, 1^{n_{\text{pr}}}, S, T) \\ (c_P, x, \pi^{(0)}, \pi^{(1)}) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda) .$$

We note that the aforementioned straight-line extraction property implies the following weaker property given in [GSW23]:

Soundness. For any stateful poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for $\lambda, n, m \in \mathbb{N}$ with $n = n(\lambda)$ and $m = m(\lambda)$, and every $P \in \{0, 1\}^m$,

$$\Pr \left[P(x) \neq y \wedge \text{Verify}(\text{crs}, x, y, c_P, \pi) = 1 \quad : \quad \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (c_P, \text{aux}) \leftarrow \text{ProgEnc}(\text{crs}, P) \\ (x, y, \pi) \leftarrow \mathcal{A}(\text{crs}, c_P, \text{aux}) \end{array} \right] \leq \text{negl}(\lambda).$$

8.2 Construction

We now give a construction for zksDel for program P from rEDel for the RAM machine $\mathcal{M}_P$ defined as below, and associated with a hash tree HT with hash key ht.hk .

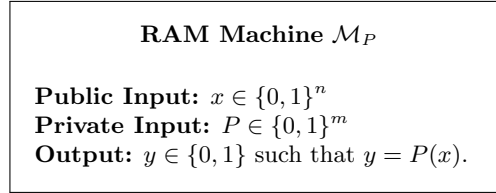


Figure 6: RAM Machine for program P

$\text{Setup}(1^\lambda, n, 1^m, T) \rightarrow \text{crs}$. Sample $\text{crs} \leftarrow \text{rEDel.Setup}(1^\lambda, n, 1^m, T)$ and output it.

$\text{ProgEnc}(\text{crs}, P) \rightarrow (c_P, \text{aux})$. Generate $(\text{eh}_P, \pi_P, \text{aux}) \leftarrow \text{rEDel.PrivEnc}(\text{crs}, P)$, where $\text{aux} = (P, r)$, and output $c_P = (\text{eh}_P, \pi_P)$ and aux .

$\text{Prove}(\text{crs}, x, \text{aux}) \rightarrow (y, \pi)$. Compute $(y, \pi) \leftarrow \text{rEDel.Prove}(\text{crs}, x, \text{aux})$ and output (y, π) .

$\text{Verify}(\text{crs}, x, y, c_P, \pi) \rightarrow \{0, 1\}$. Parse c_P as (eh_P, π_P) , compute the hash $h \leftarrow \text{HT.Hash}(\text{ht.hk}, x)$ and output

$$\text{rEDel.VerifyEnc}(\text{crs}, \text{eh}_P, \pi_P) \wedge \text{rEDel.Verify}(\text{crs}, h, \text{eh}_P, y, \pi) \quad .$$

Theorem 8.2. If rEDel for $\mathcal{M}_P$ is secure reusable encrypted RAM delegation scheme then Construction 8.2 is a secure publicly-verifiable zero-knowledge succinct delegation of committed programs scheme for the program P .

Proof. We show that Construction 8.2 satisfies all desired properties.

Completeness. This follows directly from completeness of rEDel for $\mathcal{M}_P$.

Efficiency. This follows directly from the efficiency of rEDel for $\mathcal{M}_P$.

Zero-knowledge. It follows by construction that if rEDel is input-hiding, then Construction 8.2 is zero-knowledge. $\square$

Straight-line Extraction. It follows by construction that if rEDel is an straight-line extractable, then Construction 8.2 is also straight-line extractable.

Strong Soundness. It follows by construction that if rEDel is strongly sound, then Construction 8.2 is also strongly sound.

References

- [AB09] Shweta Agrawal and Dan Boneh. Homomorphic macs: Mac-based integrity for network coding. In *Applied Cryptography and Network Security: 7th International Conference, ACNS 2009, Paris-Rocquencourt, France, June 2-5, 2009. Proceedings 7*, pages 292–305. Springer, 2009.
- [ABC⁺07] Giuseppe Ateniese, Randal C. Burns, Reza Curtmola, Joseph Herring, Lea Kissner, Zachary N. J. Peterson, and Dawn Song. Provable data possession at untrusted stores. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007: 14th Conference on Computer and Communications Security*, pages 598–609, Alexandria, Virginia, USA, October 28–31, 2007. ACM Press.
- [ACG24] Abtin Afshar, Jiaqi Cheng, and Rishab Goyal. Multi-hop multi-key homomorphic signatures with context hiding from standard assumptions. Cryptology ePrint Archive, Paper 2024/931, 2024.
- [ACJ17] Prabhanjan Ananth, Aloni Cohen, and Abhishek Jain. Cryptography with updates. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 445–472, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.
- [ADKL19] Prabhanjan Ananth, Apoorvaa Deshpande, Yael Tauman Kalai, and Anna Lysyanskaya. Fully homomorphic nizk and niwi proofs. In *Theory of Cryptography Conference*, pages 356–385. Springer, 2019.
- [ADOS22] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 421–452, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [AGM18] Shashank Agrawal, Chaya Ganesh, and Payman Mohassel. Non-interactive zero-knowledge proofs for composite statements. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 643–673, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [AGP14] Prabhanjan Ananth, Vipul Goyal, and Omkant Pandey. Interactive proofs under continual memory leakage. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part II*, volume 8617 of *Lecture Notes in Computer*

- Science*, pages 164–182, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany.
- [AJJM21] Prabhanjan Ananth, Abhishek Jain, Zhengzhong Jin, and Giulio Malavolta. Unbounded multi-party computation from learning with errors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 754–781. Springer, 2021.
 - [AJL⁺19] Prabhanjan Ananth, Aayush Jain, Huijia Lin, Christian Matt, and Amit Sahai. Indistinguishability obfuscation without multilinear maps: New paradigms via low degree weak pseudorandomness and security amplification. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 284–332, August 2019.
 - [AJLA⁺12] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold fhe. In *Advances in Cryptology–EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings 31*, pages 483–501. Springer, 2012.
 - [AN11] Tolga Acar and Lan Nguyen. Homomorphic proofs and applications, 2011.
 - [BCC⁺09] Mira Belenkiy, Jan Camenisch, Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Hovav Shacham. Randomizable proofs and delegatable anonymous credentials. In Shai Halevi, editor, *Advances in Cryptology – CRYPTO 2009*, volume 5677 of *Lecture Notes in Computer Science*, pages 108–125, Santa Barbara, CA, USA, August 16–20, 2009. Springer Berlin Heidelberg, Germany.
 - [BCG⁺17] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: Optimizations and applications. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 2105–2122, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
 - [BCG⁺19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 489–518, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
 - [BCGY24] Foteini Baldimtsi, Jiaqi Cheng, Rishab Goyal, and Aayush Yadav. Non-interactive blind signatures: Post-quantum and stronger security. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part II*, volume 15485 of *Lecture Notes in Computer Science*, pages 70–104, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
 - [BDS25] Pedro Branco, Nico Döttling, and Akshayaram Srinivasan. Rate-1 statistical non-interactive zero-knowledge. In *Annual International Cryptology Conference*, pages 514–547. Springer, 2025.

- [BDSMP91] Manuel Blum, Alfredo De Santis, Silvio Micali, and Giuseppe Persiano. Noninteractive zero-knowledge. *SIAM Journal on Computing*, 20(6):1084–1118, 1991.
- [BF11a] Dan Boneh and David Mandell Freeman. Homomorphic signatures for polynomial functions. In *Advances in Cryptology–EUROCRYPT 2011: 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings 30*, pages 149–168. Springer, 2011.
- [BF11b] Dan Boneh and David Mandell Freeman. Linearly homomorphic signatures over binary fields and new tools for lattice-based signatures. In *International Workshop on Public Key Cryptography*, pages 1–16. Springer, 2011.
- [BFKW09] Dan Boneh, David Freeman, Jonathan Katz, and Brent Waters. Signing a linear subspace: Signature schemes for network coding. In *Public Key Cryptography–PKC 2009: 12th International Conference on Practice and Theory in Public Key Cryptography, Irvine, CA, USA, March 18-20, 2009. Proceedings 12*, pages 68–87. Springer, 2009.
- [BFS14] Xavier Boyen, Xiong Fan, and Elaine Shi. Adaptively secure fully homomorphic signatures based on lattices. Cryptology ePrint Archive, Report 2014/916, 2014.
- [BGG⁺18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part I*, volume 10991 of *Lecture Notes in Computer Science*, pages 565–596, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 337–367, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part I*, volume 9814 of *Lecture Notes in Computer Science*, pages 509–539, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
- [BGI17] Elette Boyle, Niv Gilboa, and Yuval Ishai. Group-based secure computation: Optimizing rounds, communication, and computation. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 163–193, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.
- [BGI⁺18] Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. In Anna R. Karlin, editor, *ITCS 2018: 9th Innovations in Theoretical Computer Science Conference*, volume 94, pages 21:1–21:21, Cambridge, MA, USA, January 11–14, 2018. Leibniz International Proceedings in Informatics (LIPIcs).

- [BGLS03] Dan Boneh, Craig Gentry, Ben Lynn, and Hovav Shacham. Aggregate and verifiably encrypted signatures from bilinear maps. In *Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings 22*, pages 416–432. Springer, 2003.
- [BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 309–325, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery.
- [BHK17] Zvika Brakerski, Justin Holmgren, and Yael Kalai. Non-interactive delegation and batch np verification from standard computational assumptions. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, pages 474–482, 2017.
- [BJKL21] Fabrice Benhamouda, Aayush Jain, Ilan Komargodski, and Huijia Lin. Multiparty reusable non-interactive secure computation from LWE. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 724–753. Springer, 2021.
- [BKP⁺24] Nir Bitansky, Chethan Kamath, Omer Paneth, Ron D. Rothblum, and Prashant Nalini Vasudevan. Batch proofs are statistically hiding. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *56th Annual ACM Symposium on Theory of Computing*, pages 435–443, Vancouver, BC, Canada, June 24–28, 2024. ACM Press.
- [BKS19] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 3–33, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.
- [BL20] Fabrice Benhamouda and Huijia Lin. Mr NISC: Multiparty reusable non-interactive secure computation. In Rafael Pass and Krzysztof Pietrzak, editors, *Theory of Cryptography*, pages 349–378, Cham, 2020. Springer International Publishing.
- [BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 253–273, March 2011.
- [BV11a] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In Rafail Ostrovsky, editor, *52nd Annual Symposium on Foundations of Computer Science*, pages 97–106, Palm Springs, CA, USA, October 22–25, 2011. IEEE Computer Society Press.
- [BV11b] Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from ring-LWE and security for key dependent messages. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 505–524, Santa Barbara, CA, USA, August 14–18, 2011. Springer Berlin Heidelberg, Germany.

- [BWW24] Eli Bradley, Brent Waters, and David J. Wu. Batch arguments to NIZKs from one-way functions. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part II*, volume 15365 of *Lecture Notes in Computer Science*, pages 431–463, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland.
- [CG25] Jiaqi Cheng and Rishab Goyal. Boosting SNARKs and rate-1 barrier in arguments of knowledge. In Keren Censor-Hillel, Fabrizio Grandoni, Joël Ouaknine, and Gabriele Puppis, editors, *ICALP 2025: 52nd International Colloquium on Automata, Languages and Programming*, volume 334 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 56:1–56:20, Aarhus, Denmark, July 8–11, 2025. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik.
- [CGKS23] Matteo Campanelli, Chaya Ganesh, Hamidreza Khoshakhlagh, and Janno Siim. Impossibilities in succinct arguments: Black-box extraction and more. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 465–489, July 2023.
- [CHKV25] Henry Corrigan-Gibbs, Alexandra Henzinger, Yael Tauman Kalai, and Vinod Vaikuntanathan. Somewhat homomorphic encryption from linear homomorphism and sparse LPN. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 3–33, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- [CJJ21] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. Non-interactive batch arguments for np from standard assumptions. In *Annual International Cryptology Conference*, pages 394–423. Springer, 2021.
- [CJJ22] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. SNARGs for $\mathcal{P}$ from LWE. In *62nd Annual Symposium on Foundations of Computer Science*, pages 68–79, Denver, CO, USA, February 7–10, 2022. IEEE Computer Society Press.
- [CKLM12] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Sarah Meiklejohn. Malleable proof systems and applications. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 281–300, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [CKLM13a] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Sarah Meiklejohn. Malleable signatures: Complex unary transformations and delegatable anonymous credentials. *Cryptology ePrint Archive*, Report 2013/179, 2013.
- [CKLM13b] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Sarah Meiklejohn. Succinct malleable NIZKs and an application to compact shuffles. In Amit Sahai, editor, *TCC 2013: 10th Theory of Cryptography Conference*, volume 7785 of *Lecture Notes in Computer Science*, pages 100–119, Tokyo, Japan, March 3–6, 2013. Springer Berlin Heidelberg, Germany.

- [CLTV15] Ran Canetti, Huijia Lin, Stefano Tessaro, and Vinod Vaikuntanathan. Obfuscation of probabilistic circuits and applications. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015: 12th Theory of Cryptography Conference, Part II*, volume 9015 of *Lecture Notes in Computer Science*, pages 468–497, Warsaw, Poland, March 23–25, 2015. Springer Berlin Heidelberg, Germany.
- [CM15] Michael Clear and Ciaran McGoldrick. Multi-identity and multi-key leveled fhe from learning with errors. In *Advances in Cryptology–CRYPTO 2015: 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16–20, 2015, Proceedings, Part II 35*, pages 630–656. Springer, 2015.
- [CM21] Geoffroy Couteau and Pierre Meyer. Breaking the circuit size barrier for secure computation under quasi-polynomial LPN. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 842–870, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [CW23] Jeffrey Champion and David J. Wu. Non-interactive zero-knowledge from non-interactive batch arguments. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 38–71, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [DGKV22] Lalita Devadas, Rishab Goyal, Yael Kalai, and Vinod Vaikuntanathan. Rate-1 non-interactive arguments for batch-np and applications. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1057–1068. IEEE, 2022.
- [DGMR21] Nico Döttling, Vipul Goyal, Giulio Malavolta, and Justin Raizes. Interaction-preserving compilers for secure computation. *Cryptology ePrint Archive*, 2021.
- [DHRW16] Yevgeniy Dodis, Shai Halevi, Ron D. Rothblum, and Daniel Wichs. Spooky encryption and its applications. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 93–122, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
- [DIJL23] Quang Dao, Yuval Ishai, Aayush Jain, and Huijia Lin. Multi-party homomorphic secret sharing and sublinear MPC from sparse LPN. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 315–348, August 2023.
- [FGJS17] Nelly Fazio, Rosario Gennaro, Tahereh Jafarikhah, and William E. Skeith, III. Homomorphic secret sharing from paillier encryption. In Tatsuaki Okamoto, Yong Yu, Man Ho Au, and Yannan Li, editors, *ProvSec 2017: 11th International Conference on Provable Security*, volume 10592 of *Lecture Notes in Computer Science*, pages 381–399, Xi’an, China, October 23–25, 2017. Springer, Cham, Switzerland.
- [FMNP16] Dario Fiore, Aikaterini Mitrokotsa, Luca Nizzardo, and Elena Pagnin. Multi-key homomorphic authenticators. In *International conference on the theory and application of cryptology and information security*, pages 499–530. Springer, 2016.

- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.
- [GGI⁺15] Craig Gentry, Jens Groth, Yuval Ishai, Chris Peikert, Amit Sahai, and Adam Smith. Using fully homomorphic hybrid encryption to minimize non-interactive zero-knowledge proofs. *Journal of Cryptology*, 28(4):820–843, 2015.
- [GKP⁺13] Shafi Goldwasser, Yael Tauman Kalai, Raluca Ada Popa, Vinod Vaikuntanathan, and Nikolai Zeldovich. How to run turing machines on encrypted data. In *Advances in Cryptology–CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part II*, pages 536–553. Springer, 2013.
- [GMW87] O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, STOC ’87, page 218–229, New York, NY, USA, 1987. Association for Computing Machinery.
- [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Non-interactive zaps and new techniques for nizk. In *Advances in Cryptology–CRYPTO 2006: 26th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 2006. Proceedings 26*, pages 97–111. Springer, 2006.
- [GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Advances in Cryptology–CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I*, pages 75–92. Springer, 2013.
- [GSW23] Riddhi Ghosal, Amit Sahai, and Brent Waters. Non-interactive publicly-verifiable delegation of committed programs. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 575–605, May 2023.
- [GU24] Romain Gay and Bogdan Ursu. On instantiating unleveled fully-homomorphic signatures from falsifiable assumptions. In Qiang Tang and Vanessa Teague, editors, *PKC 2024: 27th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 14601 of *Lecture Notes in Computer Science*, pages 74–104, Sydney, NSW, Australia, April 15–17, 2024. Springer, Cham, Switzerland.
- [GVW15a] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Predicate encryption for circuits from LWE. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 503–523, August 2015.
- [GVW15b] Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. Leveled fully homomorphic signatures from standard lattices. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th Annual ACM Symposium on Theory of Computing*, pages 469–477, Portland, OR, USA, June 14–17, 2015. ACM Press.
- [GW11] Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In *Proceedings of the forty-third annual ACM symposium on Theory of computing*, pages 99–108, 2011.

- [Han23] Lucjan Hanzlik. Non-interactive blind signatures for random messages. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 722–752, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [HW15] Pavel Hubacek and Daniel Wichs. On the communication complexity of secure function evaluation with long output. In Tim Roughgarden, editor, *ITCS 2015*, pages 163–172. ACM, January 2015.
- [JMSW02] Robert Johnson, David Molnar, Dawn Song, and David Wagner. Homomorphic signature schemes. In *Cryptographers’ track at the RSA conference*, pages 244–262. Springer, 2002.
- [Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*, STOC ’92, page 723–732, New York, NY, USA, 1992. Association for Computing Machinery.
- [KLVW23] Yael Kalai, Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. Boosting batch arguments and ram delegation. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1545–1552, 2023.
- [KR09] Yael Tauman Kalai and Ran Raz. Probabilistically checkable arguments. In *Annual International Cryptology Conference*, pages 143–159. Springer, 2009.
- [KRR14] Yael Tauman Kalai, Ran Raz, and Ron D Rothblum. How to delegate computations: the power of no-signaling proofs. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, pages 485–494, 2014.
- [LATV12] Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, pages 1219–1234, 2012.
- [Mer87] Ralph C Merkle. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*, pages 369–378. Springer, 1987.
- [Mic94a] S. Micali. Cs proofs. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 436–453, 1994.
- [Mic94b] Silvio Micali. A secure and efficient digital signature algorithm. Technical Memo MIT/LCS/TM-501b, Massachusetts Institute of Technology, Laboratory for Computer Science, April 1994.
- [MW16] Pratyay Mukherjee and Daniel Wichs. Two round multiparty computation via multi-key fhe. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II 35*, pages 735–763. Springer, 2016.

- [OPWW15] Tatsuaki Okamoto, Krzysztof Pietrzak, Brent Waters, and Daniel Wichs. New realizations of somewhere statistically binding hashing and positional accumulators. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 121–145, November / December 2015.
- [OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 678–708, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [PP22] Omer Paneth and Rafael Pass. Incrementally verifiable computation via rate-1 batch arguments. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1045–1056. IEEE, 2022.
- [PR17] Omer Paneth and Guy N Rothblum. On zero-testable homomorphic encryption and publicly verifiable non-interactive arguments. In *Theory of Cryptography: 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part II 15*, pages 283–315. Springer, 2017.
- [RAD⁺78] Ronald L Rivest, Len Adleman, Michael L Dertouzos, et al. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.
- [RS21] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 687–717, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
- [Shi22] Sina Shiehian. mrnisc from lwe with polynomial modulus. In *International Conference on Security and Cryptography for Networks*, pages 481–493. Springer, 2022.
- [Val08] Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19-21, 2008. Proceedings 5*, pages 1–18. Springer, 2008.
- [WW24] Hoeteck Wee and David J. Wu. Succinct functional commitments for circuits from k-Lin. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 280–310, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *27th annual symposium on foundations of computer science (Sfcs 1986)*, pages 162–167. IEEE, 1986.

A Additional Definitions

In this section we provide additional pre-requisite definitions used in this work.

A.1 Somewhere Extractable Hash Families

In what follows we recall the definition of a somewhere extractable (SEH) hash family based on prior works [HW15, OPWW15].

Syntax. A somewhere extractable hash family SEH consists of the following polynomial time algorithms:

$\text{Gen}(1^\lambda, N, I) \rightarrow (\text{hk}, \text{td})$. This is a probabilistic setup algorithm that takes as input a security parameter 1^λ in unary, a message length N , and a subset $I \subseteq [N]$. It outputs a hash key hk .

$\text{Hash}(\text{hk}, x) \rightarrow \mathbf{v}$. This is a deterministic algorithm that takes as input a hash key hk generated by $\text{Gen}(1^\lambda, N, I)$ and an input $x \in \{0, 1\}^N$, and outputs a hash value $\mathbf{v} \in \{0, 1\}^{|I| \cdot \text{poly}(\lambda)}$.

$\text{Open}(\text{hk}, x, S) \rightarrow ((b_j)_{j \in S}, \rho)$. This is a deterministic algorithm that takes as input a hash key hk generated by $\text{Gen}(1^\lambda, N, I)$, an input $x \in \{0, 1\}^N$ and set of indices $S \subseteq [N]$, outputs bits b_j for $j \in S$ and an opening $\rho \in \{0, 1\}^{\leq |I| \cdot |S| \cdot \text{poly}(\lambda, \log N)}$.

$\text{Verify}(\text{hk}, \mathbf{v}, S, (b_j)_{j \in S}, \rho) \rightarrow 0/1$. This is a deterministic algorithm that takes as input a hash key hk , a hash value $\mathbf{v} \in \{0, 1\}^{|I| \cdot \text{poly}(\lambda, \log N)}$, set of indices $S \subseteq [N]$, a sequence of bits b_j for $j \in S$ and an opening ρ , $|\rho| \leq |I| \cdot |S| \cdot \text{poly}(\lambda, \log N)$, and outputs 0 or 1.

$\text{Extract}(\text{td}, \mathbf{v}) \rightarrow (b_j^*)_{j \in I}$. This is a deterministic extraction algorithm that takes as input the trapdoor td generated by $\text{Gen}(1^\lambda, N, I)$, a hash value $\mathbf{v} \in \{0, 1\}^{|I| \cdot \text{poly}(\lambda, \log N)}$, and outputs extracted sequence of bits $(b_j^*)_{j \in I}$.

Definition A.1 (SEH). A somewhere extractable hash family $\text{SEH} = (\text{Gen}, \text{Hash}, \text{Open}, \text{Verify}, \text{Extract})$ is required to satisfy the following properties:

Efficiency. The size of the hash key hk and the hash value $\mathbf{v}$ is at most $|I| \cdot \text{poly}(\lambda, \log N)$.

Index Hiding. For any PPT adversary $\mathcal{A}$, any polynomial $N = N(\lambda)$, and any $I_0, I_1 \subseteq [N]$ such that $|I_0| = |I_1|$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[b \leftarrow \mathcal{A}(\text{hk}) : \begin{array}{l} b \leftarrow \{0, 1\} \\ (\text{hk}, \text{td}) \leftarrow \text{Gen}(1^\lambda, N, I_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

Opening Completeness. For any $\lambda \in \mathbb{N}$, any $N \leq 2^\lambda$, any subset $I \subseteq [N]$, any index $j \in [N]$, and any $x \in \{0, 1\}^N$,

$$\Pr \left[\begin{array}{l} \forall j \in S, b_j = x_j \\ \wedge \text{Verify}(\text{hk}, \mathbf{v}, S, (b_j)_{j \in S}, \rho) = 1 \end{array} : \begin{array}{l} (\text{hk}, \text{td}) \leftarrow \text{Gen}(1^\lambda, N, I), \\ \mathbf{v} = \text{Hash}(\text{hk}, x), \\ ((b_j)_{j \in S}, \rho) = \text{Open}(\text{hk}, x, S), \end{array} \right] = 1.$$

Somewhere Statistically Binding w.r.to Opening. For any $\lambda \in \mathbb{N}$, any $N \leq 2^\lambda$, any subset $I \subseteq [N]$, and any (all powerful) adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{hk}, \text{v}, I, (b_j)_{j \in I}, \rho) = 1 \\ \wedge b_j \neq b_j^*, \text{ for any } j \in I \end{array} : \begin{array}{l} (\text{hk}, \text{td}) \leftarrow \text{Gen}(1^\lambda, N, I), \\ (\text{v}, (b_j)_{j \in I}, \rho) \leftarrow \mathcal{A}(\text{hk}), \\ (b_j^*)_{j \in I} = \text{Extract}(\text{td}, \text{v}) \end{array} \right] \leq \text{negl}(\lambda).$$

Remark A.2 ([KLW23, HW15, OPWW15]). Assuming $X \in \{\text{QR}, \text{DCR}, \text{DDH}, \text{LWE}, O(1)\text{-LIN}\}$, there exists an SEH family.

A.2 Multi-Key Fully Homomorphic Encryption

We provide the syntax and definitions for a multi-key fully homomorphic encryption (mkFHE) for message space $\mathcal{M} \in \{0, 1\}^*$ used in our mrNISC constructions.

Syntax. A multi-key fully homomorphic encryption scheme (mkFHE) consists of the following polynomial-time algorithms.

Setup(1^λ) $\rightarrow$ PP. The probabilistic setup algorithm takes as input the security parameter λ and outputs the public parameters PP. The following algorithms take PP as an implicit parameters unless mentioned.

KeyGen(PP) $\rightarrow$ (pk, sk). The probabilistic key generation algorithm takes as input the public parameters PP and outputs the public and secret key pair (pk, sk).

Enc(pk, m) $\rightarrow$ ct. The probabilistic encryption algorithm takes as input the public key pk, (multi-bit) message $m \in \{0, 1\}^*$, and outputs the ciphertext ct.

Eval($f, \{\text{pk}_{\text{id}}, \text{ct}_{\text{id}}\}_{\text{id} \in [n]}$) $\rightarrow$ $\tilde{\text{ct}}$. This possibly randomized algorithm takes as input the description of the function f , the public keys and ciphertexts for n parties $\{\text{pk}_{\text{id}}, \text{ct}_{\text{id}}\}_{\text{id} \in [n]}$, and outputs the evaluated ciphertext $\tilde{\text{ct}}$.

Dec($\text{sk}_1, \dots, \text{sk}_n, \tilde{\text{ct}}$) $\rightarrow y$. The deterministic decryption algorithm takes as input secret keys for n parties $\{\text{sk}_{\text{id}}\}_{\text{id} \in [n]}$, an evaluated ciphertext $\tilde{\text{ct}}$, and outputs the value y .

Definition A.3 (mkFHE). An mkFHE scheme (Setup, KeyGen, Enc, Eval, Dec) for message space $\{0, 1\}^*$, is required to satisfy the following properties:

Multi-Hop Correctness. For any $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $x, x_1, \dots, x_n \in \{0, 1\}^*$, and n -ary P/Poly circuit f :

$$\Pr \left[\begin{array}{l} x = \text{Dec}(\text{sk}, \text{ct}) \end{array} : \begin{array}{l} \text{PP} \leftarrow \text{Setup}(1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{PP}), \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x) \end{array} \right] = 1$$

$$\Pr \left[\begin{array}{l} f(x_1, \dots, x_n) = \\ \text{Dec}(\text{sk}_1, \dots, \text{sk}_n, \tilde{\text{ct}}) \end{array} : \begin{array}{l} \text{PP} \leftarrow \text{Setup}(1^\lambda), \\ \forall \text{id} \in [n], (\text{pk}_{\text{id}}, \text{sk}_{\text{id}}) \leftarrow \text{KeyGen}(\text{PP}), \\ \forall \text{id} \in [n], \text{ct}_{\text{id}} \leftarrow \text{Enc}(\text{pk}_{\text{id}}, x_{\text{id}}), \\ \tilde{\text{ct}} = \text{Eval}(f, \{\text{pk}_{\text{id}}, \text{ct}_{\text{id}}\}_{\text{id} \in [n]}) \end{array} \right] = 1$$

Compactness. For any $\lambda \in N$, $n = n(\lambda)$, $x_1, \dots, x_n \in \{0, 1\}^*$, and n -ary P/Poly circuit f , there exists a fixed polynomial $\text{poly}(\cdot, \cdot, \cdot)$ such that $|\tilde{\text{ct}}| = \text{poly}(\lambda, n, |f(x_1, \dots, x_n)|)$ where $\text{PP} \leftarrow \text{Setup}(1^\lambda), \forall \text{id} \in [n], (\text{pk}_{\text{id}}, \text{sk}_{\text{id}}) \leftarrow \text{KeyGen}(\text{PP}), \text{ct}_{\text{id}} \leftarrow \text{Enc}(\text{pk}_{\text{id}}, x_{\text{id}}), \tilde{\text{ct}} = \text{Eval}(f, \{\text{pk}_{\text{id}}, \text{ct}_{\text{id}}\}_{\text{id} \in [n]})$.

Semantic Security. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that $\forall \lambda \in \mathbb{N}$,

$$\left| \Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}(\text{ct}) \quad : \quad \begin{array}{l} b \leftarrow \{0, 1\}, \text{PP} \leftarrow \text{Setup}(1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{PP}), \\ (m_0, m_1) \leftarrow \mathcal{A}(1^\lambda, \text{PP}, \text{pk}), \\ \text{ct}_0 \leftarrow \text{Enc}(\text{pk}, m_0), \text{ct}_1 \leftarrow \text{Enc}(\text{pk}, m_1) \end{array} \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

Theorem A.4 ([LATV12, CM15, MW16]). Assuming the hardness of **LWE** with sub-exponential modulus-to-noise ratio and circular security, there exists an **mkFHE** scheme for all P/Poly circuits and $\mathcal{M} = \{0, 1\}^*$.

B Split Memory RAM Delegation

In this section, we state and provide a short proof of a property of RAM Delegation that [CJJ22, KLVW23] satisfies implicitly. Note that in this section we use ‘memory’ and ‘configuration’ interchangeably and we let both of them include the input as well the work tape. We show that the RAM delegation of [CJJ22, KLVW23] can be done when the memory is hashed into multiple hash values and can be verified using these multiple hash values. One advantage of this approach as we demonstrate is that the running time of the prover of RAM delegation scheme can be dependent only on the portion of the memory it uses as work tape (which of polynomial-size computations, is polynomial).

We require an additional property from the collision resistant hash family as building block: for $\text{HT.Hash}(\text{ht.hk}, 0^*)$, the output value $\text{h} = 0^{\lambda+1}$. This can be achieved using any collision resistant hash tree **HT**. Using **HT** we can define a “punctured” hashing algorithm as follows:

$\text{Hash}(\text{ht.hk}, x) \rightarrow \text{h}$. If x is an all-zero string, output $\text{h} = 0^{\lambda+1}$. Otherwise, output $\text{h} = 1 \parallel \text{HT.Hash}(\text{ht.hk}, x)$.

As remarked in [CG25], completeness and collision resistance follow from **HT**. One useful implication of this “punctured” hash evaluation is that the RAM machine need not worry about the part of memory which is not needed. Hence, when we start with an all zero configuration, we need not provide the hash values for all the configurations as they are a string of zeros anyway. Thus, the running time of RAM machine only grows polynomially in the size of the work tape it uses depending on the number partitions specified and hash values changed. We provide a RAM delegation construction that obeys Definition 2.17 which splits the memory into n parts below.

B.1 Split Memory RAM Delegation Construction

We provide the construction of split memory RAM delegation in accordance with Definition 2.17 as follows. We remark that $\tilde{\mathcal{L}}$ (specified below) is the language for Batch Index Circuit-SAT and $C^{\mathcal{M}}$ doesn’t require the entire description of $\mathcal{M}$ hardwired. As shown by Choudhuri et al. [CJJ22], a state transition circuit which ensures that transition between st_{i-1} and st_i are consistent

with $\mathcal{M}$'s description is sufficient. For the sake of brevity, we ignore this technical subtlety in the construction. In addition, since we are using batch arguments for index languages and the statements are clear from the context, we avoid repeating the index statements as input to the batch argument's algorithms. However, we will add the language circuit to the inputs of the batch argument's algorithms as it will help with the analysis.

Construction B.1 (Split Memory RAM Delegation). Let $\text{SEH} = (\text{Gen}, \text{Hash}, \text{Open}, \text{Verify}, \text{Extract})$ be a somewhere extractable hash family (Definition A.1), $\text{HT} = (\text{Gen}, \text{Hash}, \text{Open}, \text{Write}, \text{VerifyRead}, \text{VerifyWrite})$ be a hash tree, $\text{seBARG} = (\text{Setup}, \text{Prove}, \text{Verify})$ be a (publicly-verifiable, non-interactive) batch argument scheme for the language $\tilde{\mathcal{L}} = \{C^{\mathcal{M}} : \exists w_i \text{ such that } C^{\mathcal{M}}(i, w_i) = 1, \forall i \in [k]\}$ where $C^{\mathcal{M}}$ is defined in Figure 7. We provide the construction of a RAM delegation scheme for machine $\mathcal{M}$ with strong soundness below.

Description of circuit $C^{\mathcal{M}}$

Hardwired: $T, S, \text{seh.hk}^0, \text{seh.hk}^1, \text{ht.hk}, h_0^*, v^0, v^1, \text{out}$

Instance: i

Witness: $w_i = (\hat{w}_{i-1}, \rho_{i-1}^0, \rho_{i-1}^1, \hat{w}_i, \rho_i^0, \rho_i^1, b_{\text{rd},i}, \ell_{\text{rd},i}, j_i, u_{\text{rd},i}, \text{op}_{\text{rd},i}, b_{\text{wr},i}, \ell_{\text{wr},i}, j'_i, u_{\text{wr},i}, \text{op}_{\text{wr},i})$

Output: Output 1 if and only if all of these checks pass.

1. Parse $\hat{w}_{i-1} = (\text{st}_{i-1}, h_{i-1})$, and $\hat{w}_i = (\text{st}_i, h_i)$.
2. If $i = 1$, check that st_0 is the initial state. Also, check $h_0 = h_0^*$.
3. If $i = T$, check that st_T is accepting/rejecting state based on $\text{out} = 1/0$ respectively.
4. $\text{SEH.Verify}(\text{seh.hk}^0, v^0, \mathbb{I}_{i-1}, \hat{w}_{i-1}, \rho_{i-1}^0) \stackrel{?}{=} 1$ and $\text{SEH.Verify}(\text{seh.hk}^1, v^1, \mathbb{I}_{i-1}, \hat{w}_{i-1}, \rho_{i-1}^1) \stackrel{?}{=} 1$.
5. $\text{SEH.Verify}(\text{seh.hk}^0, v^0, \mathbb{I}_i, \hat{w}_i, \rho_i^0) \stackrel{?}{=} 1$ and $\text{SEH.Verify}(\text{seh.hk}^1, v^1, \mathbb{I}_i, \hat{w}_i, \rho_i^1) \stackrel{?}{=} 1$.
6. $\mathcal{M}$ in st_{i-1} should read from $\ell_{\text{rd},i}$ and write $b_{\text{wr},i}$ to $\ell_{\text{wr},i}$ and transitions to st_i . In addition, $j_i, j'_i \in [n]$ are partitions that accommodate indices $\ell_{\text{rd},i}$ and $\ell_{\text{wr},i}$ respectively.
7. $u_{\text{rd},i} \stackrel{?}{=} \ell_{\text{rd},i} - (S^{(1)} + \dots + S^{(j_i-1)})$ and $u_{\text{wr},i} \stackrel{?}{=} \ell_{\text{wr},i} - (S^{(1)} + \dots + S^{(j'_i-1)})$.
8. $\text{HT.VerifyRead}(\text{ht.hk}, h_{i-1}^{(j_i)}, u_{\text{rd},i}, b_{\text{rd},i}, \text{op}_{\text{rd},i}) \stackrel{?}{=} 1$.
9. $\text{HT.VerifyWrite}(\text{ht.hk}, h_{i-1}^{(j'_i)}, u_{\text{wr},i}, b_{\text{wr},i}, h_i^{(j'_i)}, \text{op}_{\text{wr},i}) \stackrel{?}{=} 1$.

Figure 7: Circuit for verifying RAM memory transition.

$\text{Setup}(1^\lambda, T, S) \rightarrow \text{crs}$. The probabilistic setup algorithm takes as input the security parameter, a vector of split sizes $S = (S^{(1)}, \dots, S^{(n)})$, running time T for machine $\mathcal{M}$, and does the following.

1. Sample two SEH hash keys, $\text{seh.hk}^0, \text{seh.hk}^1 \leftarrow \text{SEH.Gen}(1^\lambda, m \cdot (T+1), \mathbb{I}_1)$ where $m = |\hat{w}_i|$ (later specified in Prove) and $\mathbb{I}_i = \{(i-1) \cdot m + 1, \dots, i \cdot m\}$ for $i \in [T]$.
2. Sample crs for seBARG, $\text{barg.crs} \leftarrow \text{seBARG.Setup}(1^\lambda, T, 1^{|C^{\mathcal{M}}|}, 1)$.
3. Sample a HT hash key, $\text{ht.hk} \leftarrow \text{HT.Gen}(1^\lambda)$.

Output $\text{crs} = (T, S, \text{seh.hk}^0, \text{seh.hk}^1, \text{barg.crs}, \text{ht.hk})$.

$\text{Digest}(\text{crs}, x) \rightarrow \mathbf{h}$. The deterministic digest algorithm parses crs , uses ht.hk , parses x as $(x^{(1)}, \dots, x^{(n)})$, such that $|x^{(j)}| = S^{(j)}$ for each $j \in [n]$, computes $\mathbf{h}^{(j)} = \text{HT.Hash}(\text{ht.hk}, x^{(j)})$, and outputs $\mathbf{h} = (\mathbf{h}^{(1)}, \dots, \mathbf{h}^{(n)})$.

$\text{Prove}(\text{crs}, \text{cf}_0) \rightarrow (\text{out}, \pi)$. The probabilistic prover algorithm parses crs as $(T, S, \text{seh.hk}^0, \text{seh.hk}^1, \text{barg.crs}, \text{ht.hk})$ and does the following.

1. Let out denote the output, $((\text{st}_1, \text{cf}_1), \dots, (\text{st}_T, \text{cf}_T))$ denote the states and configurations resultant from running $\mathcal{M}(\text{cf}_0, T)$, where cf_i for $i \in [0, T]$ includes the input and work tape.
2. For each $i \in [0, T]$, let $\mathbf{h}_i = \text{Digest}(\text{crs}, \text{cf}_i)$.
3. For each $i \in [T]$:
 - Let $\ell_{\text{rd},i}, \ell_{\text{wr},i}$ denote the location that $\mathcal{M}$ reads from, writes to memory when it is in state st_{i-1} . Let $\ell_{\text{rd},i}, \ell_{\text{wr},i}$ belong to some partitions indexed by $j_i, j'_i \in [n]$ respectively.
 - Let $u_{\text{rd},i}$ denote the index of $\ell_{\text{rd},i}$ in the partition j_i . That is $u_{\text{rd},i} = \ell_{\text{rd},i} - (S^{(1)} + \dots + S^{(j_i-1)})$. Similarly, calculate $u_{\text{wr},i}$.
 - Compute $(b_{\text{rd},i}, \text{op}_{\text{rd},i}) = \text{HT.Open}(\text{ht.hk}, x_{i-1}^{(j_i)}, u_{\text{rd},i})$ and $(\tilde{\mathbf{h}}_i^{(j'_i)}, \text{op}_{\text{wr},i}) = \text{HT.Write}(\text{ht.hk}, x_{i-1}^{(j'_i)}, u_{\text{wr},i}, b_{\text{wr},i})$ where $x_{i-1}^{(j_i)}$ and $x_{i-1}^{(j'_i)}$ are the j_i -th and j'_i -th partitions of cf_{i-1} .
4. For $i \in [0, T]$, set $\hat{w}_i := (\text{st}_i, \mathbf{h}_i)$.
5. Let $\hat{\mathbf{w}} = (\hat{w}_i)_{i \in [0, T]}$, compute $\mathbf{v}^0 = \text{SEH.Hash}(\text{seh.hk}^0, \hat{\mathbf{w}})$, $\mathbf{v}^1 = \text{SEH.Hash}(\text{seh.hk}^1, \hat{\mathbf{w}})$.
6. For $i \in [0, T]$, $(\hat{w}_i, \rho_i^0) = \text{SEH.Open}(\text{seh.hk}^0, \hat{\mathbf{w}}, \mathbb{I}_i)$, $(\hat{w}_i, \rho_i^1) = \text{SEH.Open}(\text{seh.hk}^1, \hat{\mathbf{w}}, \mathbb{I}_i)$.
7. For $i \in [T]$, let $w_i := (\hat{w}_{i-1}, \rho_{i-1}^0, \rho_{i-1}^1, \hat{w}_i, \rho_i^0, \rho_i^1, b_{\text{rd},i}, \ell_{\text{rd},i}, j_i, u_{\text{rd},i}, \text{op}_{\text{rd},i}, b_{\text{wr},i}, \ell_{\text{wr},i}, j'_i, u_{\text{wr},i}, \text{op}_{\text{wr},i})$.
8. Let $\text{barg.}\pi \leftarrow \text{seBARG.Prove}(\text{barg.crs}, C^{\mathcal{M}}, (w_i)_{i \in [T]})$.
9. Set $\pi := (\mathbf{v}^0, \mathbf{v}^1, \text{barg.}\pi)$ and output (out, π) .

$\text{Verify}(\text{crs}, \text{h}_0^*, \text{out}, \pi) \rightarrow 0/1$. The deterministic verification algorithm parses crs as $(T, S, \text{seh.hk}^0, \text{seh.hk}^1, \text{barg.crs}, \text{ht.hk})$, parses π as $(\mathbf{v}^0, \mathbf{v}^1, \text{barg.}\pi)$, and outputs $\text{seBARG.Verify}(\text{barg.crs}, C^{\mathcal{M}}, \text{barg.}\pi)$.

Theorem B.2. If SEH is a secure somewhere extractable hash family (Definition A.1), HT is a sound hash tree (Definition 2.6), and seBARG is a secure somewhere extractable batch argument scheme for language $\tilde{\mathcal{L}}$ (Definition 2.1), then Construction B.1 is a RAM delegation scheme for machine $\mathcal{M}$ w.r.to HT that satisfies strong soundness.

Proof. We show that Construction B.1 is correct, efficient, and collision resistant as follows:

Completeness. Completeness of RAM delegation scheme follows from the completeness of HT, seBARG, and opening completeness of SEH.

Efficiency. Assuming that HT, SEH, seBARG are efficient, the RAM delegation scheme satisfies the following properties:

1. Sizes of the crs and proof π are $\text{poly}(\lambda, n, \log T)$.
2. Running time of Setup is $\text{poly}(\lambda, n, \log T)$.
3. Running time of Digest is $\text{poly}(n, \lambda)$ and the size of digest is $\text{poly}(n, \lambda)$.
4. Running time of Prove is $\text{poly}(\lambda, T, n)$.
5. Running time of Verify is $\text{poly}(\lambda, n, \log T)$.

Collision Resistance. It is easy to see that if HT is collision resistant, then so is the RAM delegation scheme. If there exists an adversary that can produce two configurations cf, cf' that break collision resistance with non-negligible probability ϵ , then note that there is at least one partition indexed by $j \in [n]$ such that the j -th partition of cf is not equal to the j -th partition of cf' . We can reduce this adversary to an adversary that breaks the collision resistance of HT by using this $x^{(j)}$ with ϵ probability. $\square$

B.2 Security Analysis

Here, we show that Construction B.1 is secure against PPT adversaries. We provide a sketch of the proof and it is easy to see that full proof follows from [CJJ22, KLVW23].

Proof of strong soundness. We show that if HT satisfies reading and writing soundness, SEH satisfies index-hiding, straight-line extraction, and seBARG satisfies index-hiding and somewhere extraction, then Construction B.1 satisfies strong soundness. Assume towards a contradiction that there exists a PPT adversary $\mathcal{A}$ that can break the strong soundness property with some non-negligible probability $\epsilon(\lambda)$ as defined in Definition 2.17 for $T = T(\lambda), n = n(\lambda), S(\lambda) = S = (S^{(1)}, \dots, S^{(n)})$. In other words,

$$\Pr \left[\begin{array}{l} \forall b \in \{0, 1\}, \text{seBARG.Verify}(\text{barg.crs}, \\ C_b, \text{barg}.\pi_b) = 1 \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, T, S), \\ (h_0, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] = \epsilon(\lambda)$$

where C_b is $C^\mathcal{M}$ (Figure 7) with $\text{out} = b$. In what follows, we will assume that Setup_k is the same as Setup except that we use $k, \mathbb{I}_k, \mathbb{I}_{k+1}$ for $k \in [T]$ in seBARG.Setup , β -th SEH.Gen , and $(1 - \beta)$ -th SEH.Gen respectively where $\beta = (k - 1) \bmod 2$. By index-hiding property of SEH, seBARG, there exists a negligible function $\mu(\cdot)$ such that for any PPT adversary $\mathcal{A}$, $\lambda \in \mathbb{N}$ and $k \in [T]$,

$$\Pr \left[\begin{array}{l} \forall b \in \{0, 1\}, \\ \text{seBARG.Verify}(\text{barg.crs}, C_b, \\ \text{barg}.\pi_b) = 1 \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}_k(1^\lambda, T, S), \\ (h_0, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \geq \epsilon(\lambda) - \mu(\lambda)$$

Now, we proceed to inductively argue for k that there exists a negligible function $\zeta(\cdot)$ such that for any PPT adversary $\mathcal{A}$, $k \in [T], \lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \forall b \in \{0, 1\}, \\ \text{seBARG.Verify} \\ (\text{barg.crs}, C_b, \text{barg}.\pi_b) = 1 \wedge \\ \widehat{w}_{0,k} = \widehat{w}_{1,k} \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}_k(1^\lambda, T, S), \\ (\mathbf{h}_0, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{crs}) \\ \forall b \in \{0, 1\}, \widehat{w}_{b,k} \leftarrow \\ \text{SEH.Extract}(\text{seh.td}^\beta, v_b^\beta) \end{array} \right] \geq \epsilon(\lambda) - k \cdot \zeta(\lambda) \quad (20)$$

where $\beta \equiv (k-1) \bmod 2$. When $k = T$, Equation 20 claims that an accepting state and rejecting configurations are identical with non-negligible probability which is a contradiction. Hence, we will prove by induction on k that above equation holds.

Base Case. We need to start from $\widehat{w}_{b,0}$, however, since the Setup_k is only defined for $k \geq 1$, for the base case we just consider $k = 1$ for setup and extract $\widehat{w}_{0,0}$ and $\widehat{w}_{1,0}$ from SEH. This holds as there is a unique starting hash value $\mathbf{h}_0$ and starting state $\mathbf{st}_0$.

Induction. Assume that Equation 20 holds for k . Now, we will switch to using $\text{seBARG}.\mathcal{E}$ instead of normal instantiation. By somewhere extraction property of seBARG , there exists a negligible function $\nu(\cdot)$ such that for any PPT adversary $\mathcal{A}$, $k \in [T]$, $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \widehat{w}_{0,k} = \widehat{w}_{1,k} \wedge \\ \forall b \in \{0, 1\}, \\ \text{seBARG.Verify} \\ (\text{barg.crs}, C_b, \text{barg}.\pi_b) = 1 \\ \wedge C_b(k, w_{b,k}) = 1 \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}_k(1^\lambda, T, S), \\ (\mathbf{h}_0, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{crs}) \\ \forall b \in \{0, 1\}, \\ w_{b,k} \leftarrow \text{seBARG}.\mathcal{E}(\text{barg.td}, C_b, \text{barg}.\pi_b), \\ \widehat{w}_{b,k} \leftarrow \text{SEH.Extract}(\text{seh.td}^\beta, v_b^\beta) \end{array} \right] \geq \epsilon(\lambda) - k \cdot \zeta(\lambda) - \nu(\lambda)$$

Parse $w_{b,k}$ to find $\mathbf{st}_{b,k-1}$, $\mathbf{st}_{b,k}$, $\mathbf{h}_{b,k-1}$, $\mathbf{h}_{b,k}$. Next we additionally extract from $(1 - \beta)$ -th instantiation SEH and by assuming the somewhere statistical binding property of SEH, there exists a negligible function $\nu_1(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \widehat{w}_{0,k} = \widehat{w}_{1,k} \wedge \\ \forall b \in \{0, 1\}, \\ \text{seBARG.Verify} \\ (\text{barg.crs}, C_b, \text{barg}.\pi_b) = 1 \\ \wedge C_b(k, w_{b,k}) = 1 \wedge \\ \widehat{w}_{b,k} = (\mathbf{st}_{b,k}, \mathbf{h}_{b,k}) \wedge \\ \widehat{w}_{b,k+1} = (\mathbf{st}_{b,k+1}, \mathbf{h}_{b,k+1}) \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}_k(1^\lambda, T, S), \\ (\mathbf{h}_0, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{crs}) \\ \forall b \in \{0, 1\}, \\ w_{b,k} \leftarrow \text{seBARG}.\mathcal{E}(\text{barg.td}, C_b, \text{barg}.\pi_b), \\ \widehat{w}_{b,k} \leftarrow \text{SEH.Extract}(\text{seh.td}^\beta, v_b^\beta), \\ \widehat{w}_{b,k+1} \leftarrow \text{SEH.Extract}(\text{seh.td}^{1-\beta}, v_b^{1-\beta}) \end{array} \right] \geq \epsilon(\lambda) - k \cdot \zeta(\lambda) - \nu(\lambda) - \nu_1(\lambda)$$

By relying on the reading soundness and writing soundness of HT, we can see that there exists a negligible function $\nu_2(\cdot)$ such that for any PPT adversary $\mathcal{A}$, $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \widehat{w}_{0,k+1} = \widehat{w}_{1,k+1} \wedge \\ \forall b \in \{0,1\}, \\ \text{seBARG.Verify} \\ (\text{barg.crs}, C_b, \text{barg}.\pi_b) = 1 \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}_k(1^\lambda, T, S), \\ (\text{h}_0, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{crs}) \\ \forall b \in \{0,1\}, \\ w_{b,k} \leftarrow \text{seBARG}.\mathcal{E}(\text{barg.td}, C_b, \text{barg}.\pi_b), \\ \widehat{w}_{b,k} \leftarrow \text{SEH.Extract}(\text{seh.td}^\beta, v_b^\beta), \\ \widehat{w}_{b,k+1} \leftarrow \text{SEH.Extract}(\text{seh.td}^{1-\beta}, v_b^{1-\beta}) \end{array} \right] \\ \geq \epsilon(\lambda) - k \cdot \zeta(\lambda) - \nu(\lambda) - \nu_1(\lambda) - \nu_2(\lambda)$$

Now, we will stop extracting using **seBARG** and β -th **SEH** and instead extract them on $k+1$ and $\mathbb{I}_{k+2}$ respectively. By the index hiding properties of **seBARG** and **SEH**, there exists a negligible function $\nu_3(\cdot)$ such th for any PPT $\mathcal{A}, \lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \widehat{w}_{0,k+1} = \widehat{w}_{1,k+1} \wedge \\ \forall b \in \{0,1\}, \\ \text{seBARG.Verify} \\ (\text{barg.crs}, C_b, \text{barg}.\pi_b) = 1 \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}_{k+1}(1^\lambda, T, S), \\ (\text{h}_0, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{crs}) \\ \forall b \in \{0,1\}, \\ \widehat{w}_{b,k+1} \leftarrow \text{SEH.Extract}(\text{seh.td}^{1-\beta}, v_b^{1-\beta}) \end{array} \right] \\ \geq \epsilon(\lambda) - k \cdot \zeta(\lambda) - \nu(\lambda) - \nu_1(\lambda) - \nu_2(\lambda) - \nu_3(\lambda)$$

Note that $(1 - \beta) \equiv ((k+1) - 1) \pmod{2}$. By setting $\zeta(\lambda) = \nu(\lambda) + \sum_{l=1}^3 \nu_l(\lambda)$, we prove the induction step as desired. $\square$